

American International University-Bangladesh (AIUB)



Department of Computer Science

Faculty of Science and Technology

ADVANCE DATABASE MANAGEMENT SYSTEM

Semester: SUMMER 2024-2025

Section: [A]

Supervised By

Juena Ahmed Noshin

Topic Name :

Outdoor Sports Facility Management System

Student Details

Student Name	Student ID	Contributed Topic	Percentage%
MD. SHADMAN SHAKIB ALAM	22-46262-1	Introduction, User Interface, Normalization, Table creation, Adv PL/SQL	25%
RIMON PRAMANIK	22-46957-1	Project Proposal, Table creation, Normalization, Adv PL/SQL	25%
LAMIA TAHSIN	22-46345-1	Scenario, ER diagram, Table creation, Data insertion, Basic PL/SQL	25%
MARUF AHMEND	22-46354-1	Schema diagram, Basic PL/SQL , Conclusion, Data insertion	25%

Submission Date: August 23, 2025

TABLE OF CONTENTS

1. INTRODUCTION	3
2. PROJECT PROPOSAL	4-6
3. USER INTERFACE: (Figma)	7-14
4. SCENARIO DESCRIPTION	15
5. ER DIAGRAM	16
6. NORMALIZATION	17-26
7. SCHEMA DIAGRAM	27
8. TABLE CREATION	28-40
9. DATA INSERTING	40-55
10. BASIC PL/SQL	55-77
11. ADVANCE PL/SQL	78-98
12. CONCLUSION	99

1. INTRODUCTION:

The **Outdoor Sports Facility Management System** is a comprehensive platform built to simplify the management of sports facilities, enhance player experience, and streamline event coordination. Designed for both sports organizations and individual players, this system empowers users to easily book turfs, create and manage teams, schedule matches, and view their profile all from a single, user-friendly interface.

The **Home Page** serves as a gateway to the platform's core features, providing easy navigation for users to **Join**, **Create a Team**, or **Book Turf**. The design is clean and intuitive, ensuring easy access for players and administrators to the tools they need.

Key Features:

- ✓ **Turf Management:** Players can view available turfs and book them for matches or practice sessions. The **Turf View** feature allows users to explore the turf in detail before making a booking.
- ✓ **Team Creation & Management:** Players can form or join teams easily. The platform allows team creation, member invitations, and roster management. Teams can view available slots and schedule practice sessions or matches.
- ✓ **Booking & Payment:** The **Booking Summary** page provides players with an overview of their upcoming bookings. The **Payment Options** and **Payment Success** pages offer secure and smooth transaction processing, with multiple payment methods available for flexibility.
- ✓ **Event Scheduling:** Organize and manage sports events like tournaments and matches effortlessly. The **Booking Summary** page keeps players and administrators updated with event schedules and details.
- ✓ **Profile Management:** Users can manage their personal information and track their activities, including match history and performance.
- ✓ **Favorites:** Users can save their favorite turfs and bookings for quick access in the future, enhancing their experience with easy links to frequently used services.

The system integrates **Nest.js** for backend efficiency, **Next.js** for a responsive and dynamic frontend, and a secure **SQL/PLSQL** database for robust data management. The platform ensures real-time updates, reliability, and scalability to meet growing user needs.

By using this platform, sports organizations can efficiently manage their grounds, schedules, and player data, while players enjoy a seamless and engaging sports experience whether they're booking a turf for practice, forming a team, or participating in a match.

2. PROJECT PROPOSAL :

Project Overview:

The Outdoor Sports facility Management System is a comprehensive, user-friendly platform designed to simplify the management of sports facilities, enhance player experience, and streamline event coordination. This platform will serve both sports organizations and individual players, allowing users to easily book turfs, create teams, manage events, and access their profiles all from a single interface.

With a clean and intuitive design, the system enables players to view real-time turf availability, reserve slots for practice or matches, and manage team activities. Sports organizations will benefit from the efficient management of turf bookings, player data, and event scheduling.

Objective:

The primary goal of the Outdoor Sports Facility Management System is to provide a seamless and efficient solution for managing outdoor sports activities. The system will achieve this by:

- ✓ Allowing players to easily book available turfs for matches or practice sessions.
- ✓ Enabling players to create and join teams, as well as manage team schedules.
- ✓ Providing administrators and sports organizations with tools to manage turf availability, player data, bookings, and payments.
- ✓ Offering a secure and smooth payment gateway for all bookings and event participation.

Scope of the Project:

1. Target Audience:

- **Sports Organizations:** Clubs, event organizers, coaches, and administrators who need an efficient platform to manage turf availability, player data, and event schedules.
- **Players:** Individuals who want to easily book turfs, participate in events, form or join teams, and manage their profiles.

2. Key Features:

- **Turf Management:** Players can view available turfs, check their condition, and book them for practice or matches. The Turf View feature allows users to explore turfs before making bookings.

- **Team Creation & Management:** Players can create or join teams, invite members, and manage rosters. Teams can view available slots and schedule practice sessions or matches.
- **Booking & Payment System:** The Booking Summary page provides players with an overview of upcoming bookings. The Payment Options and Payment Success pages ensure secure transactions, with multiple payment methods available.
- **Event Scheduling:** Admins and players can organize sports events such as tournaments and matches. The system automatically updates players with event details and schedules.
- **Profile Management:** Users can update their profiles, view team participation, and manage personal information.
- **Favorites:** Players can save their favorite turfs and bookings for easy future access.

3. Technology Stack:

- **Frontend:** Next.js for a dynamic and responsive user interface.
- **Backend:** Nest.js for handling backend logic and managing API requests.
- **Database:** SQL/PLSQL for secure, scalable data management.
- **Payment Integration:** bKash, Nagad, or other payment gateways for secure and smooth transactions.

Problem Statement:

Currently, sports organizations and players face challenges in managing turf bookings, organizing events, and tracking player data manually. This often leads to inefficiencies, double bookings, and poor communication. Players also struggle to find and join teams that match their skill levels, and administrators face difficulties managing large amounts of data.

Proposed Solution:

The Outdoor Sports Management System addresses these challenges by:

- Providing a platform where users can check turf availability and book slots in real-time.
- Allowing players to create or join teams, view team rosters, and schedule events or practices.
- Offering a smooth and secure payment system for booking turfs and registering for events.
- Allowing administrators to manage events, bookings, player data, and payments with ease.

Expected Outcomes:

1. For Sports Organizations:

- Simplified turf and event management.
- Enhanced communication with players and teams.
- Reduced administrative workload and errors.

2. For Players:

- Easy access to turf availability and event schedules.
- Streamlined team creation and participation.
- Personalized experience with player profiles and team management.

Timeline:

The project will be completed in the following phases:

- Phase 1: Planning & Requirements Gathering (1 weeks)
- Phase 2: System Design & Database Schema (2 weeks)
- Phase 3: Backend & Frontend Development (3 weeks)
- Phase 4: Integration & Testing (1 weeks)
- Phase 5: Deployment & Final Review (1 weeks)

Budget & Resources

The project requires resources for:

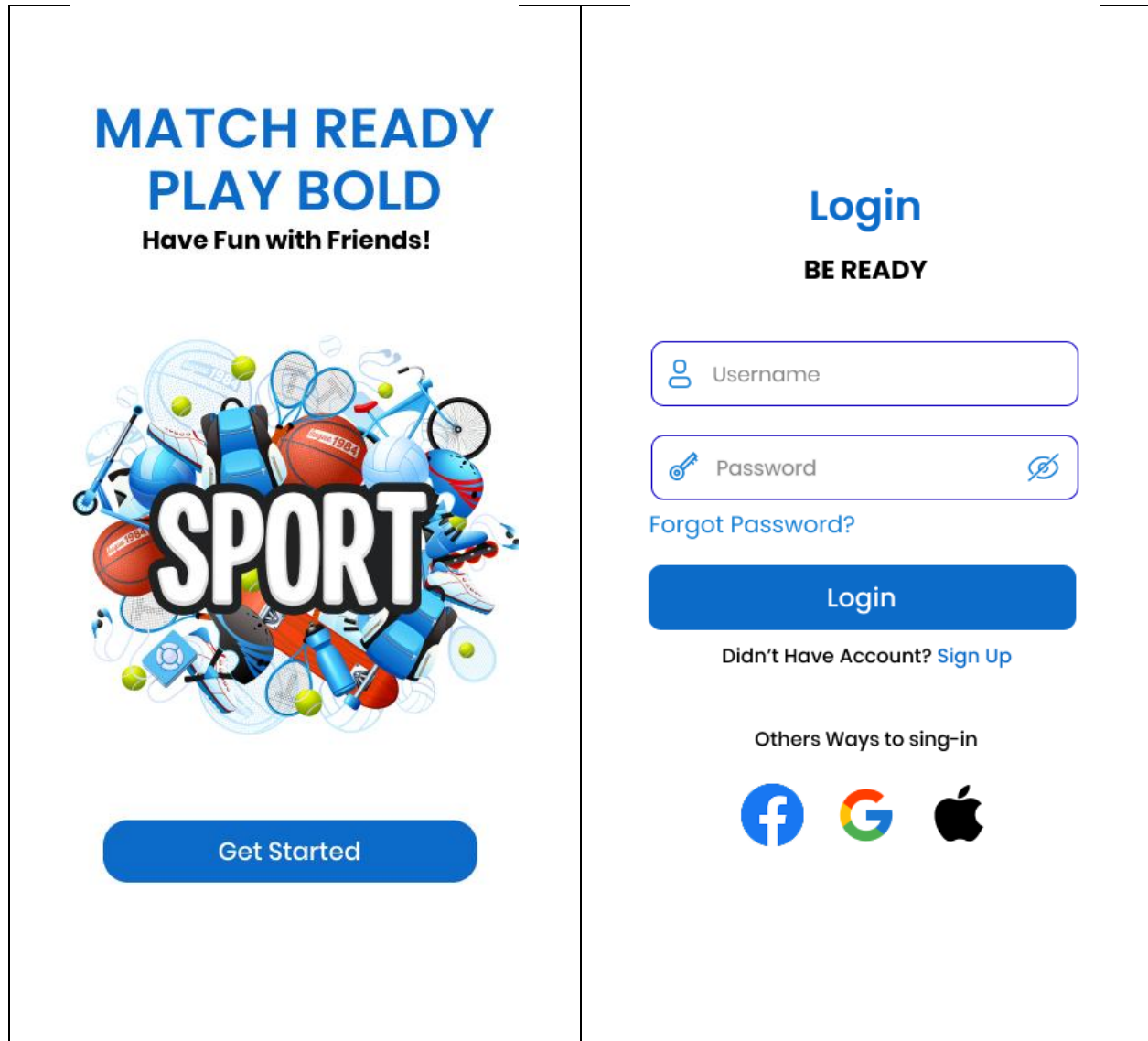
- Software tools and technology (Next.js, Nest.js, SQL/PLSQL).
- Web hosting and domain registration.
- Payment gateway integration.
- Development and testing resources.

The estimated budget will cover these resources, with the project expected to be completed over the course of 2 months.

3. USER INTERFACE: (Figma)

Figma Live link:

<https://www.figma.com/design/gu6MmmA9LLIFNvthsJOYzK/Outdoor-sports-management?node-id=0-1&p=f&t=R19DcoDAKIZGseXV-0>



Sign - Up



Sign-Up

Didn't Have Account? [Sign Up](#)

Others Ways to sing-in



[← Forgot Password](#)

Verify with OTP to change Password
sent to 0181****938



☐ Auto fetching OTP...

Submit

Didn't receive it? Retry in 00:60

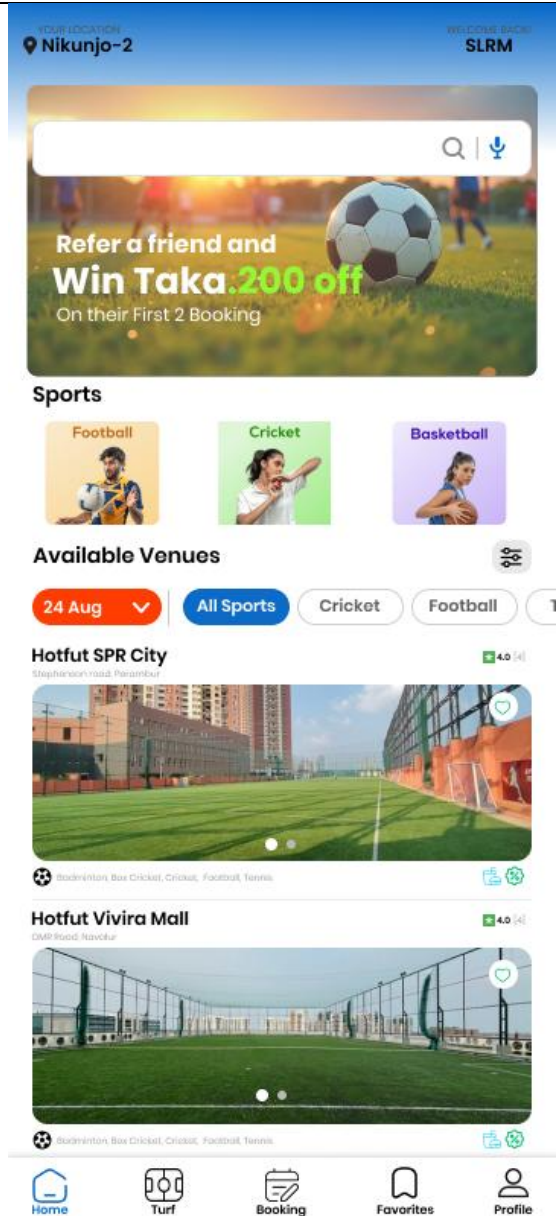
← Save Password

Verify with OTP to change Password
sent to 0181****938

 New Password 

 Confirm Password 

Save Password



YOUR LOCATION

Nikunjo-2

WELCOME BACK!

SLRM

Q

All Trufs

14 Sep

All Sports

Cricket

Football

Hotfut SPR City

Stephenson road, Panambur

4.0

(4)

Badminton, Box Cricket, Cricket, Football, Tennis

Hotfut Vivira Mall

OMV Road, Navalur

4.0

(4)

Badminton, Box Cricket, Cricket, Football, Tennis

Play Champs Badminton

Barathi Nagar, Kolamur

4.0

(4)

Badminton, Box Cricket, Cricket, Football, Tennis

Turfworld Vivira Mall

OMV Road, Navalur

4.0

(4)

Home

Turf

Booking

Favorites

Profile

SKY LAND TURF

Ayanavaram

4.0

(4)

13,2nd Cross St, near asia Bank, VP Colony Branch, ICF South Colony,Ayanavaram, uttor badda

Get Direction

Flatt Tk.200 off On all slots

Flatt Tk.200 off On all slots

Flatt Tk.200 off On all slots

Available Sports

Box Cricket

Cricket

Football

Venue info

Pitch

• 1 Nets

Equipment Provided

Artificial Turf

Amenities

UPI Accepted

Card Accepted

Free Parking

Toilets

Showers

Showers

PROCEED TO SELECT A SLOT

← SKY LAND TURF, Ayanavaram

FORMAT

Box Cricket

NO. OF PITCHES

1

Sun
24 Aug

Mon
25 Aug

Tue
26 Aug

Wed
27 Aug

Thu
28 Aug

Flatt Tk.200 off
On all slots

Twilight

Morning

Noon

Evening

6pm

7pm

8pm

8pm

9pm

10pm

11pm

12pm

Offer applied You are saving Tk.200

Tk.2,800 Tk.3,000

9 pm - 10 pm • 2 Slots

PROCEED →

← SKY LAND TURF, Ayanavaram

03:00

Aug 24 09:00pm - 10:00pm

Box Cricket

5 v 5 Pitches

Bill Details

Slot Cost Tk.3,000

Venue Offer -Tk.200

Service Fee Tk.0

Total Tk.2,800

Payment Options

Advance

Tk.200

Full Amount

Tk.2,800

What will you be playing?

4 v 4

5 v 5

6 v 6

7 v 7

8 v 8

9 v 9

How many players?

8

9

10

10

11

12

13

14

Tk.280 / ♂

Cancellation Policy

Cancel before 25th Aug, 4:00 pm to avail a refund.
Service fee is non-refundable.

If you cancel after the above time, you will lose the
entire amount paid,

Full Amount

Tk.2,800

PROCEED TO PAY →



UPI



Bkash



Pay on Bkash



Nagad



Paytm



[Add New UPI ID](#)

Add New UPI ID
You need to have a registered UPI ID

Credit % Debit Cards



[Add New Card](#)

Save and pay via Cards

Other Options



Net Banking

Select from list of bank



Paid Successful



Congratulations!

Your Slot has been booked!

TICKET

Scan this QR on the Scanner Machine When
you are in Entrance



NAME	ID
slmr	Surendhar_PV
MOBILE	Sport Name
018166049**	Box Cricket
Turf Ground Name	Address
Sky Land Truf	Badda
Booking Time	Date
9:00-10:00 PM	24-08-2025
Players	Amount
5 v 5	Tk. 2800

Transaction ID
21487987329578

[Home](#)

[Download Ticket](#)

YOUR LOCATION
Nikunjo-2

WELCOME BACK!
SLRM



Favourites

Hotfut SPR City

Stephanos road, Parambar

4.0 (4)



Badminton, Box Cricket, Cricket, Football, Tennis

Hotfut Vivira Mall

DMP Road, Navalur

4.0 (4)



Badminton, Box Cricket, Cricket, Football, Tennis



Home



Turf



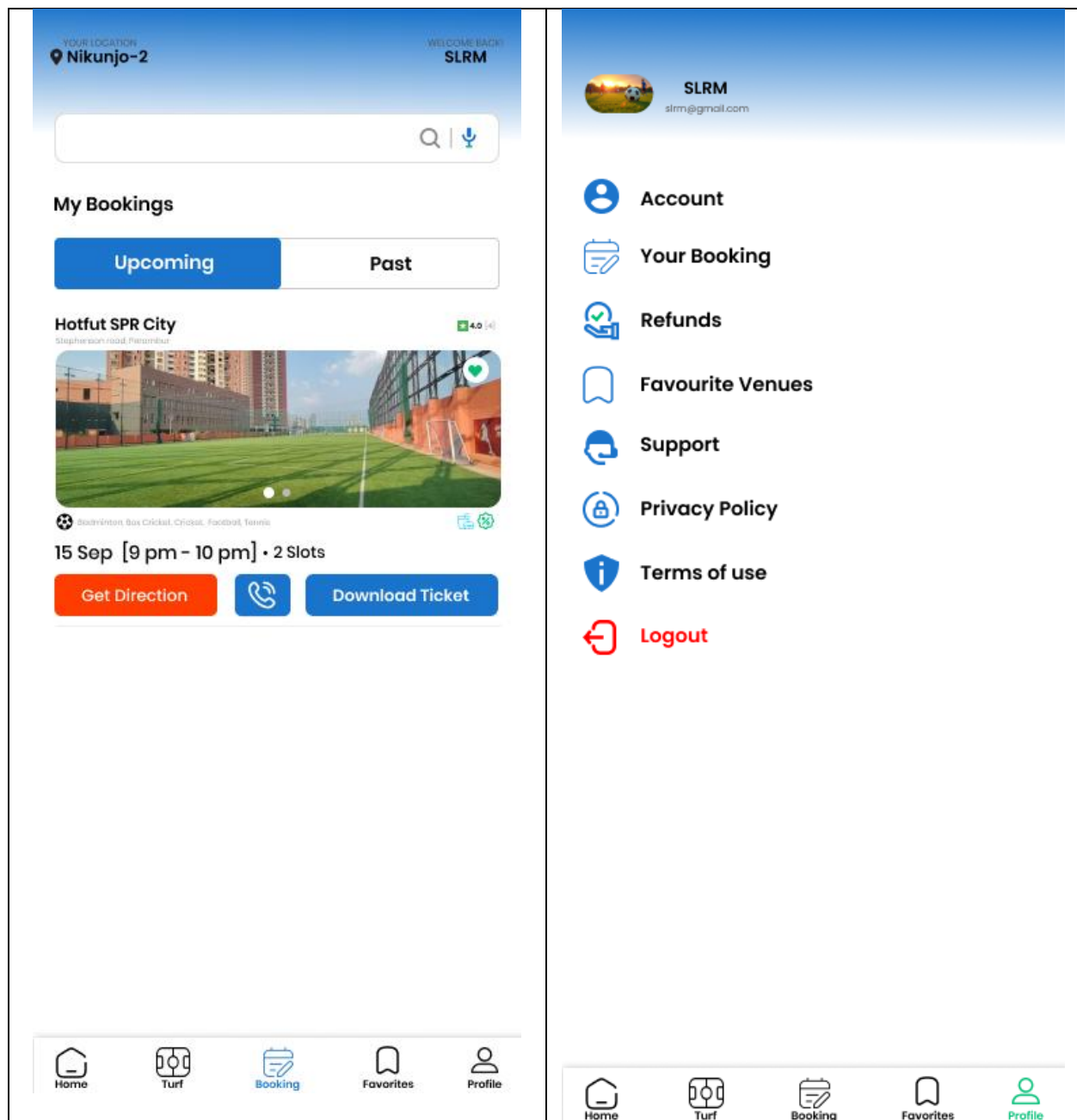
Booking



Favorites



Profile



4. SCENARIO DESCRIPTION:

In an Outdoor Sports Facility Management System, one Player can belong to only one Team. Each Player has a unique identification number (Player ID), Name, Date of Birth, and Phone Number. A Name is composed of First Name and Last Name.

Each Player has one Profile. A Profile has a unique identification number (Profile ID), Skill Level, and a record of Achievements.

One Team can make many Bookings, and one Booking is made by one Team. A Team has a unique identification number (Team ID), Team Name, and Sport Type.

One Player can also make many Bookings, and one Booking is made by one Player. A Booking has a unique identification number (Booking ID), Date, Start Time, End Time, and Status.

One Turf can be booked for many Bookings (at different times). A Turf has a unique identification number (Turf ID), Name, Location, Size, and Type (e.g., Grass, AstroTurf).

One Event requires one Turf, but one Turf can host many Events. An Event has a unique identification number (Event ID), Name, Date, and Rules.

Many Teams can participate in one Event, but for this system, we will model it as many Teams participating in one Event. An Event's details include its participating Teams.

One Staff Member manages many Events and also manages many Bookings. Each Staff Member has a unique identification number (Staff ID), Name, and Department. A Name is composed of First Name and Last Name.

One Booking can be paid for by many Payments (e.g., installment plans, multiple payment methods). A Payment has a unique identification number (Payment ID), Amount, and Payment Date.

Many Payments can be processed by one Staff Member. Each Payment is associated with one Payment Type (e.g., Credit Card, Debit Card, Cash). Each Payment Type has a unique code (Payment Type Code), a Payment Method, and a Description.

5. ER DIAGRAM:

Outdoor Sports Management System

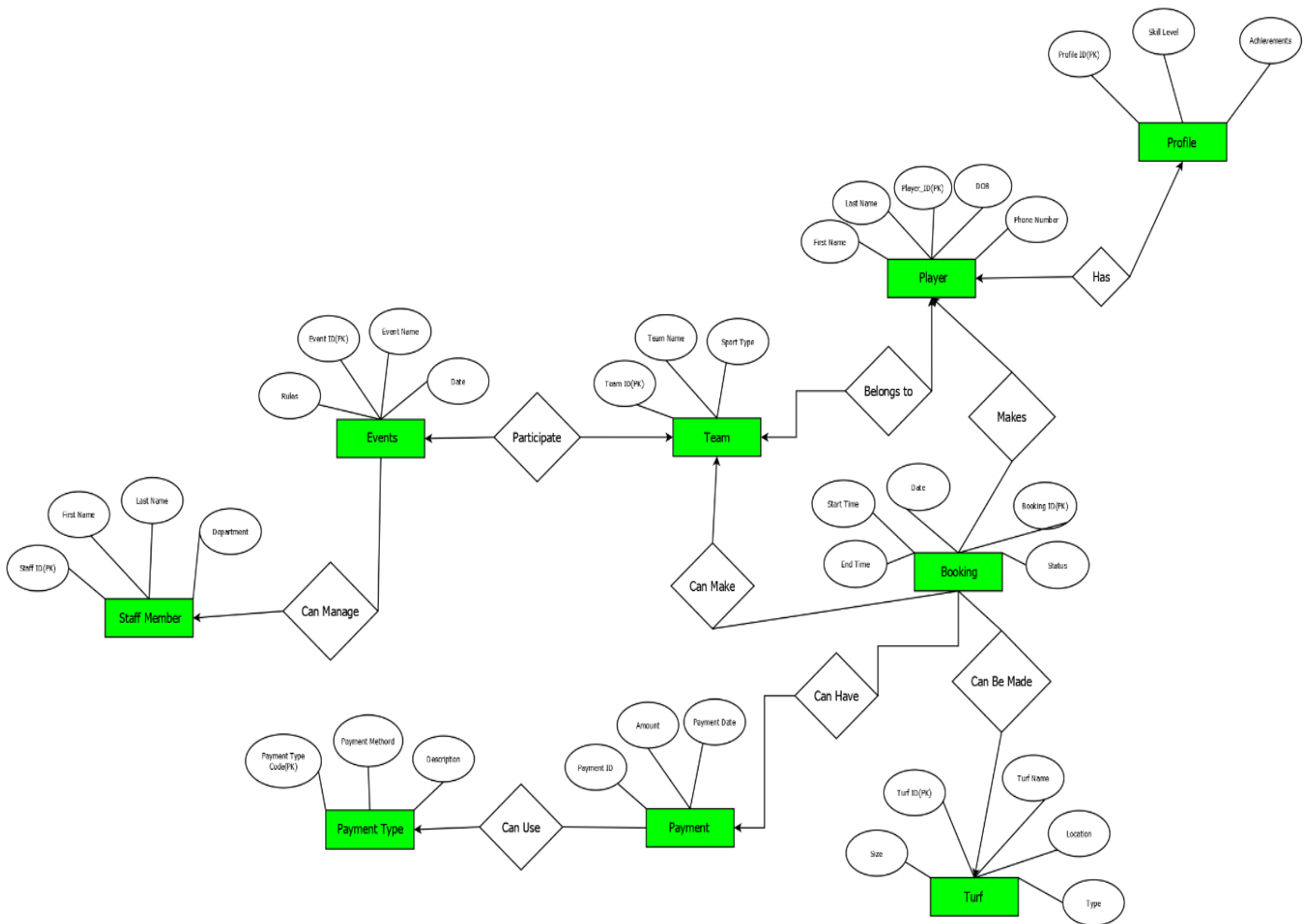


Fig-1: ER diagram

6. NORMALIZATION:

Relation 1: Can Manage (Staff Member Can Manage Event)

UNF:

Can_Manage(Staff_ID, First_Name, Last_Name, Department, Event_ID, Event_Name, Date, Rules)

1NF:

No multivalued attributes, so it remains

1. Staff_ID, First_Name, Last_Name, Department, Event_ID, Event_Name, Date, Rules

2NF:

Staff details depend on Staff_ID

Event details depend on Event_ID.

1. Staff_ID, First_Name, Last_Name, Department

2. Event_ID, Event_Name, Date, Rules

3. Staff_ID, Event_ID

3NF:

No Transitive dependencies.

1. Staff_ID, First_Name, Last_Name, Department

2. Event_ID, Event_Name, Date, Rules

3. Staff_ID, Event_ID

Table Creation:

Staff_Member(Staff_ID, First_Name, Last_Name, Department)

Event(Event_ID, Event_Name, Date, Rules)

Can_Manage(Staff_ID, Event_ID)

Relation 2: Participates (Event Participates in Team)

UNF:

Participates(Event_ID, Event_Name, Date, Rules, Team_ID, Team_Name, Sport_Type)

1NF:

No multivalued attributes, so it remains

1. Event_ID, Event_Name, Date, Rules, Team_ID, Team_Name, Sport_Type

2NF:

Event details depend on Event_ID.

Team details depend on Team_ID.

1. Event_ID, Event_Name, Date, Rules

2. Team_ID, Team_Name, Sport_Type

3. Event_ID, Team_ID

3NF:

No Transitive dependencies.

1. Event_ID, Event_Name, Date, Rules

2. Team_ID, Team_Name, Sport_Type

3. Event_ID, Team_ID

Table Creation:

Event(Event_ID, Event_Name, Date, Rules)

Team(Team_ID, Team_Name, Sport_Type)

Participates(Event_ID, Team_ID)

Relation 3: Belongs To (Player Belongs to Team)

UNF:

Belongs_To(Player_ID, First_Name, Last_Name, DOB, Phone_Number, Team_ID, Team_Name, Sport_Type)

1NF:

No multivalued attributes, so it remains

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number, Team_ID, Team_Name, Sport_Type

2NF:

Player details depend on Player_ID.

Team details depend on Team_ID.

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number

2. Team_ID, Team_Name, Sport_Type

3. Player_ID, Team_ID

3NF:

No Transitive dependencies.

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number

2. Team_ID, Team_Name, Sport_Type

3. Player_ID, Team_ID

Table Creation:

Player(Player_ID, First_Name, Last_Name, DOB, Phone_Number)

Team(Team_ID, Team_Name, Sport_Type)

Belongs_To(Player_ID, Team_ID)

Relation 4: Has (Player Has Profile)

UNF:

Has(Player_ID, First_Name, Last_Name, DOB, Phone_Number, Profile_ID, Skill_Level, Achievements)

1NF:

No multivalued attributes, so it remains

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number, Profile_ID, Skill_Level, Achievements

2NF:

Player details depend on Player_ID.

Profile details depend on Profile_ID.

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number

2. Profile_ID, Skill_Level, Achievements

3. Player_ID, Profile_ID

3NF:

No Transitive dependencies.

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number

2. Profile_ID, Skill_Level, Achievements

3. Player_ID, Profile_ID

Table Creation:

Player(Player_ID, First_Name, Last_Name, DOB, Phone_Number)

Profile(Profile_ID, Skill_Level, Achievements)

Has(Player_ID, Profile_ID)

Relation 5: Can Make (Team Can Make Booking)

UNF:

Can_Make(Team_ID, Team_Name, Sport_Type, Booking_ID, Start_Time, End_Time, Date, Status)

1NF:

No multivalued attributes, so it remains

1. Team_ID, Team_Name, Sport_Type, Booking_ID, Start_Time, End_Time, Date, Status

2NF:

Team details depend on Team_ID

Booking details depend on Booking_ID

1. Team_ID, Team_Name, Sport_Type

2. Booking_ID, Start_Time, End_Time, Date, Status

3. Team_ID, Booking_ID

3NF:

No Transitive dependencies.

1. Team_ID, Team_Name, Sport_Type

2. Booking_ID, Start_Time, End_Time, Date, Status

3. Team_ID, Booking_ID

Table Creation:

Team(Team_ID, Team_Name, Sport_Type)

Booking(Booking_ID, Start_Time, End_Time, Date, Status)

Can_Make(Team_ID, Booking_ID)

Relation 6: Makes (Player Makes Booking)

UNF:

Makes(Player_ID, First_Name, Last_Name, DOB, Phone_Number, Booking_ID, Start_Time, End_Time, Date, Status)

1NF:

No multivalued attributes, so it remains

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number, Booking_ID, Start_Time, End_Time, Date, Status

2NF:

Player details depend on Player_ID

Booking details depend on Booking_ID

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number

2. Booking_ID, Start_Time, End_Time, Date, Status

3. Player_ID, Booking_ID

3NF:

No Transitive dependencies.

1. Player_ID, First_Name, Last_Name, DOB, Phone_Number

2. Booking_ID, Start_Time, End_Time, Date, Status

3. Player_ID, Booking_ID

Table Creation:

Player(Player_ID, First_Name, Last_Name, DOB, Phone_Number)

Booking(Booking_ID, Start_Time, End_Time, Date, Status)

Makes(Player_ID, Booking_ID)

Relation 7: Can Be Made (Booking Can Be Made On Turf)

UNF:

Can_Be_Made(Booking_ID, Start_Time, End_Time, Date, Status, Turf_ID, Turf_Name, Location, Size, Type)

1NF:

No multivalued attributes, so it remains

1. Booking_ID, Start_Time, End_Time, Date, Status, Turf_ID, Turf_Name, Location, Size, Type

2NF:

Booking details depend on Booking_ID

Turf details depend on Turf_ID

1. Booking_ID, Start_Time, End_Time, Date, Status

2. Turf_ID, Turf_Name, Location, Size, Type

3. Booking_ID, Turf_ID

3NF:

No Transitive dependencies.

1. Booking_ID, Start_Time, End_Time, Date, Status

2. Turf_ID, Turf_Name, Location, Size, Type

3. Booking_ID, Turf_ID

Table Creation:

Booking(Booking_ID, Start_Time, End_Time, Date, Status)

Turf(Turf_ID, Turf_Name, Location, Size, Type)

Can_Be_Made(Booking_ID, Turf_ID)

Relation 8: Can Have (Booking Can Have Payment)

UNF:

Can_Have(Booking_ID, Start_Time, End_Time, Date, Status, Payment_ID, Amount, Payment_Date)

1NF:

No multivalued attributes, so it remains

1. Booking_ID, Start_Time, End_Time, Date, Status, Payment_ID, Amount, Payment_Date

2NF:

Booking details depend on Booking_ID

Payment details depend on Payment_ID

1. Booking(Booking_ID, Start_Time, End_Time, Date, Status)

2. Payment(Payment_ID, Amount, Payment_Date)

3. Can_Have(Booking_ID, Payment_ID)

3NF:

No Transitive dependencies.

1. Booking(Booking_ID, Start_Time, End_Time, Date, Status)

2. Payment(Payment_ID, Amount, Payment_Date)

3. Can_Have(Booking_ID, Payment_ID)

Table Creation:

Booking(Booking_ID, Start_Time, End_Time, Date, Status)

Payment(Payment_ID, Amount, Payment_Date)

Can_Have(Booking_ID, Payment_ID)

Relation 9: Can Use (Payment Can use Payment Type)

UNF:

Can_Use(Payment_ID, Amount, Payment_Date, Payment_Type_Code, Payment_Method, Description)

1NF:

No multivalued attributes, so it remains

1. Payment_ID, Amount, Payment_Date, Payment_Type_Code, Payment_Method, Description

2NF:

Payment details depend on Payment_ID

Payment type details depend on Payment_Type_Code

1. Payment(Payment_ID, Amount, Payment_Date, Payment_Type_Code)

2. Payment_Type(Payment_Type_Code, Payment_Method, Description)

3NF:

No Transitive dependencies.

1. Payment(Payment_ID, Amount, Payment_Date, Payment_Type_Code)

2. Payment_Type(Payment_Type_Code, Payment_Method, Description)

Table Creation:

Payment(Payment_ID, Amount, Payment_Date, Payment_Type_Code)

Payment_Type(Payment_Type_Code, Payment_Method, Description)

Final Table:

Entity Tables

1. **Staff_Member**(Staff_ID, First_Name, Last_Name, Department)
2. **Event**(Event_ID, Event_Name, Date, Rules)
3. **Team**(Team_ID, Team_Name, Sport_Type)
4. **Player**(Player_ID, First_Name, Last_Name, DOB, Phone_Number)
5. **Profile**(Profile_ID, Skill_Level, Achievements)
6. **Booking**(Booking_ID, Start_Time, End_Time, Date, Status)
7. **Turf**(Turf_ID, Turf_Name, Location, Size, Type)
8. **Payment**(Payment_ID, Amount, Payment_Date, Payment_Type_Code)
9. **Payment_Type**(Payment_Type_Code, Payment_Method, Description)

Relationship Tables

10. **Can_Manage**(Staff_ID, Event_ID)
11. **Participates**(Event_ID, Team_ID)
12. **Belongs_To**(Player_ID, Team_ID)
13. **Has_Profile**(Player_ID, Profile_ID)
14. **Can_Make**(Team_ID, Booking_ID)
15. **Makes**(Player_ID, Booking_ID)
16. **Can_Be_Made**(Booking_ID, Turf_ID)
17. **Can_Have**(Booking_ID, Payment_ID)
18. **Can_Use**(Payment_ID, Payment_Type_Code)

7. SCHEMA DIAGRAM

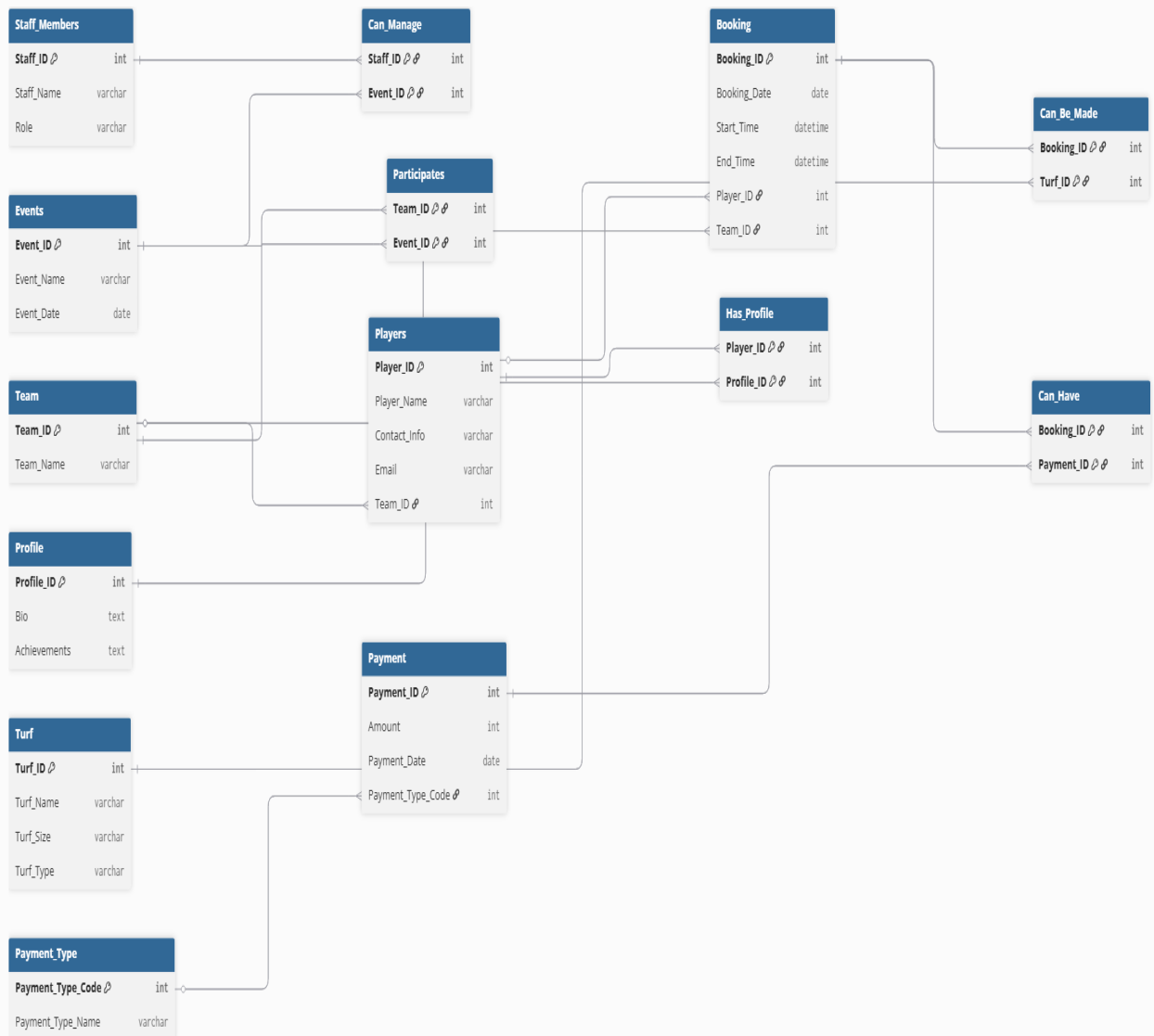


Fig-2: SCHEMA DIAGRAM

8. TABLE CREATION:

Table Name: Staff_Members

CREATE SEQUENCE staff_member_seq START WITH 1 INCREMENT BY 1;

```
CREATE TABLE Staff_Members (  
    Staff_ID NUMBER PRIMARY KEY,  
    First_Name VARCHAR2(50),  
    Last_Name VARCHAR2(50),  
    Department VARCHAR2(50)  
);
```

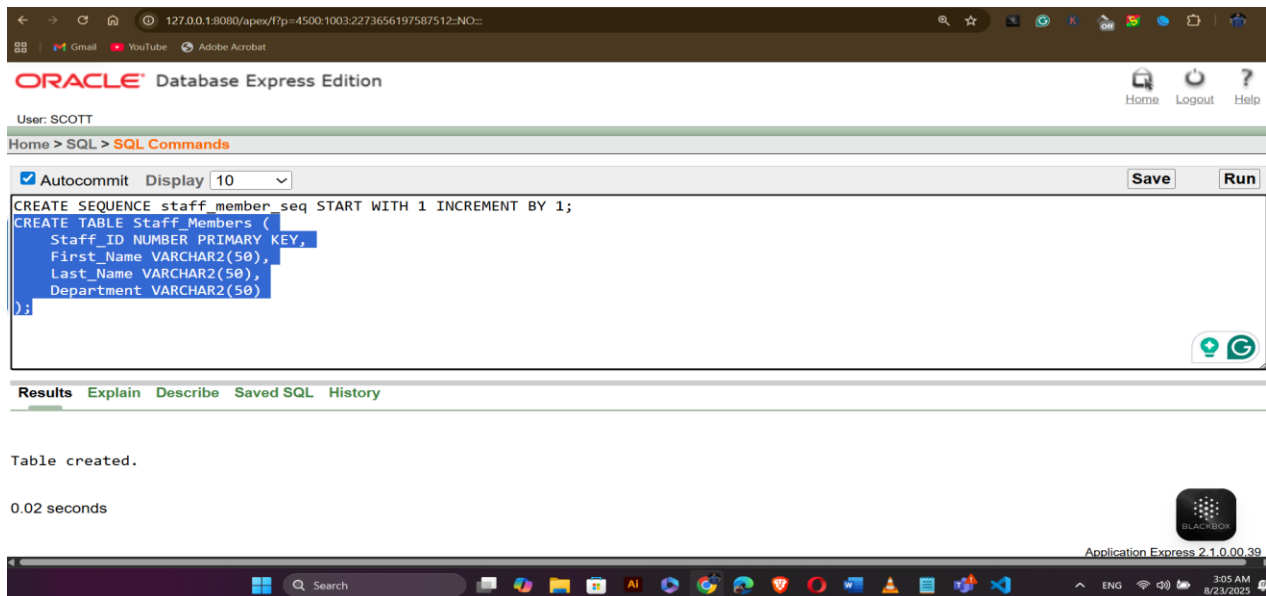


Table Name: Events

CREATE SEQUENCE event_seq START WITH 1 INCREMENT BY 1;

```
CREATE TABLE Events (  
    Event_ID NUMBER PRIMARY KEY,  
    Event_Name VARCHAR2(100),
```



```
Event_Date DATE,  
Rules VARCHAR2(500)  
);
```

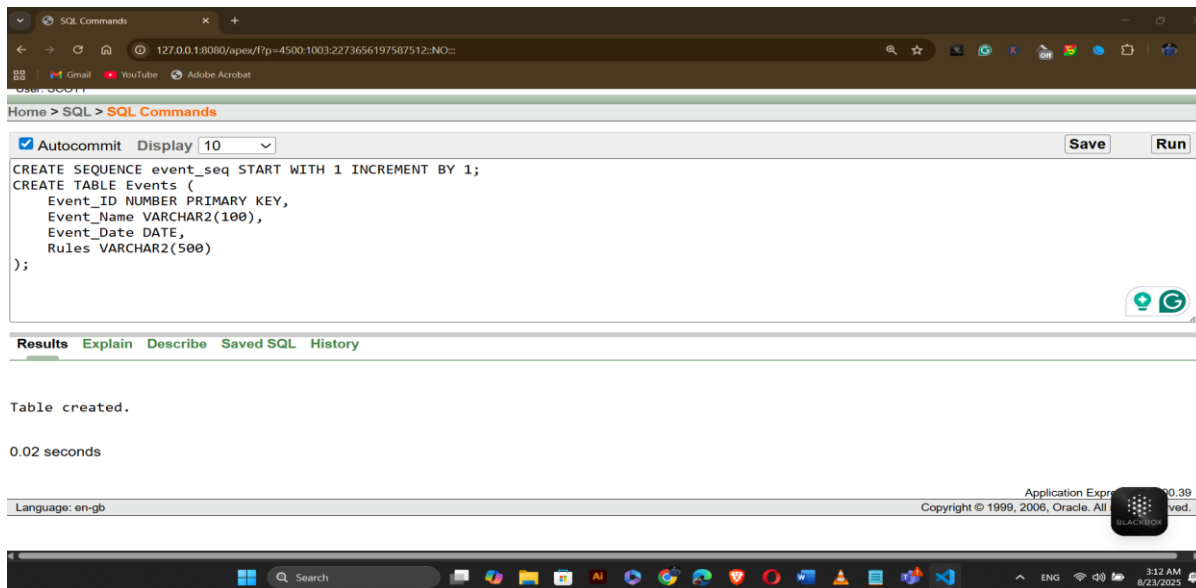


Table Name: Team

```
CREATE SEQUENCE team_seq START WITH 1 INCREMENT BY 1;  
CREATE TABLE Team (  
  Team_ID NUMBER PRIMARY KEY,  
  Team_Name VARCHAR2(100),  
  Sport_Type VARCHAR2(50)  
);
```

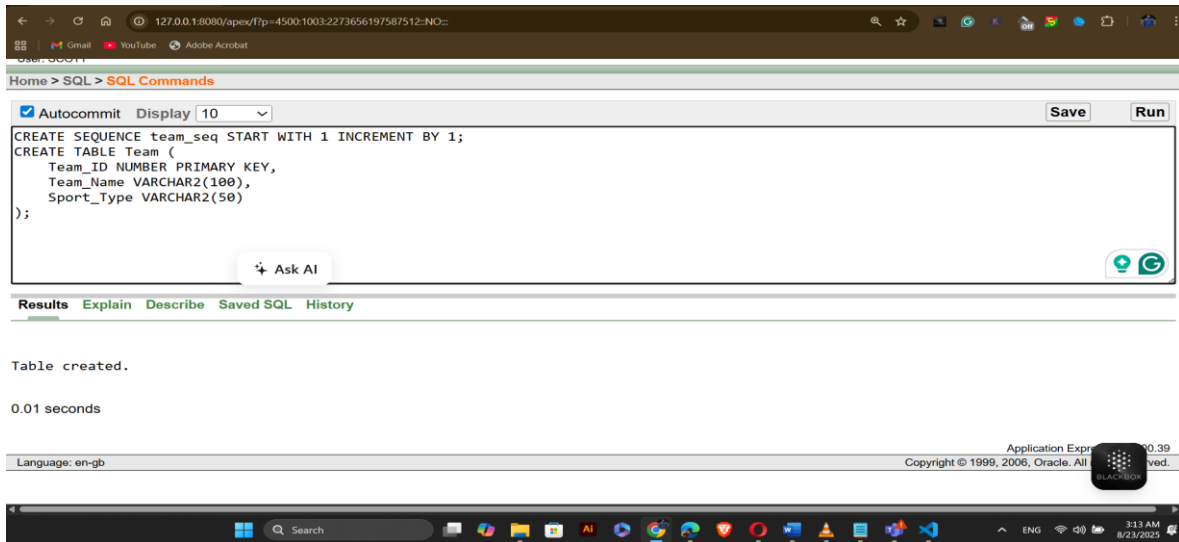


Table Name: Players

CREATE SEQUENCE players_seq START WITH 1 INCREMENT BY 1;

CREATE TABLE Players (

Player_ID NUMBER PRIMARY KEY,

First_Name VARCHAR2(50),

Last_Name VARCHAR2(50),

DOB DATE,

Phone_Number VARCHAR2(15)

);

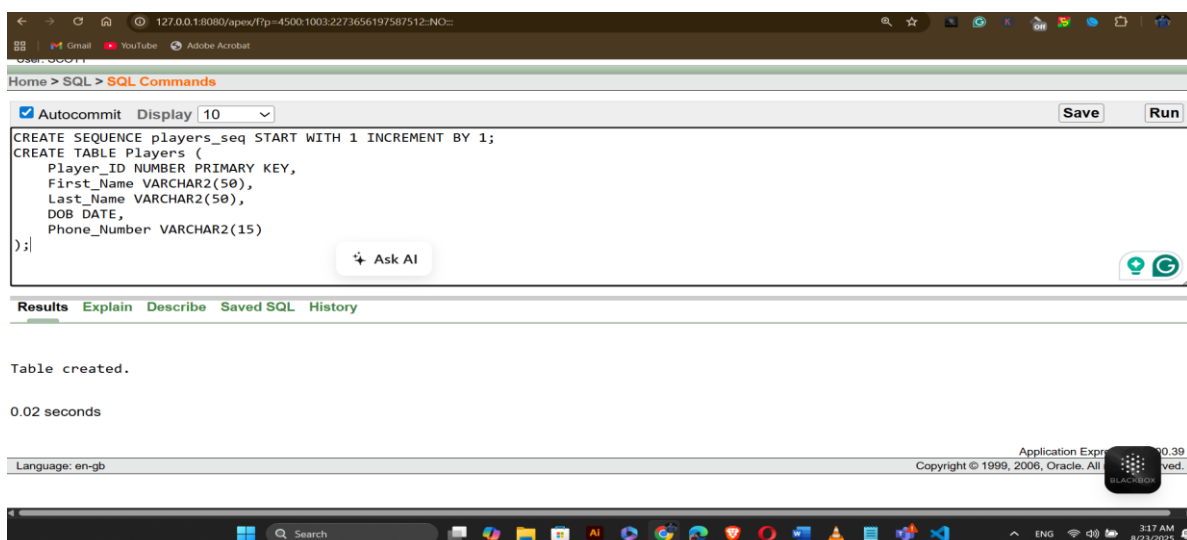


Table Name: Profile

```
CREATE SEQUENCE profile_seq START WITH 1 INCREMENT BY 1;
```

```
CREATE TABLE Profile (  
    Profile_ID NUMBER PRIMARY KEY,  
    Skill_Level VARCHAR2(50),  
    Achievements VARCHAR2(500)  
);
```

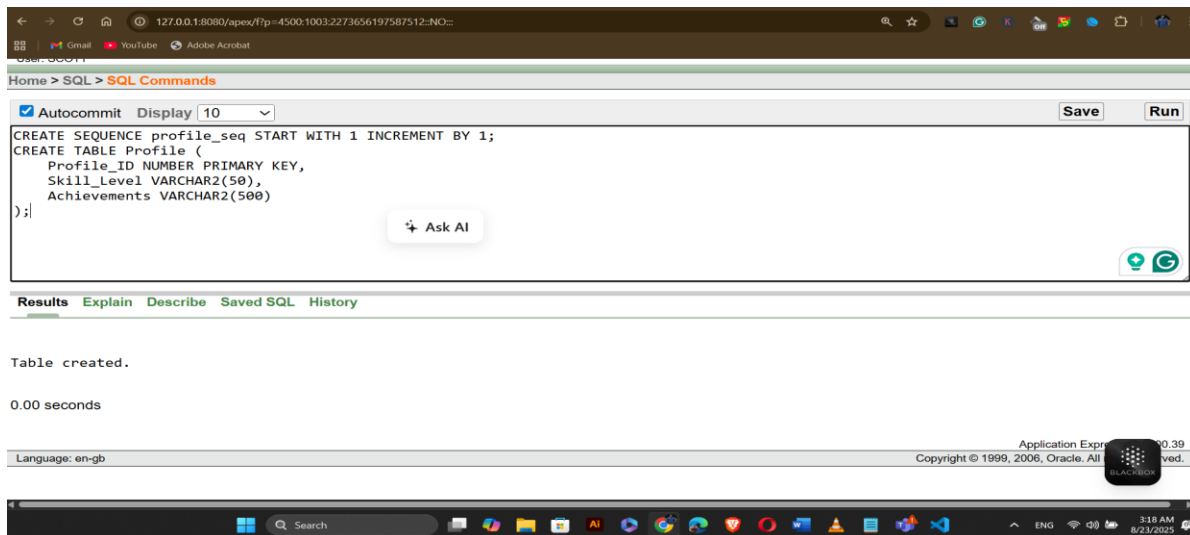


Table Name: Booking

```
CREATE SEQUENCE booking_seq START WITH 1 INCREMENT BY 1;
```

```
CREATE TABLE Booking (  
    Booking_ID NUMBER PRIMARY KEY,  
    Start_Time TIMESTAMP,  
    End_Time TIMESTAMP,  
    Booking_Date DATE,  
    Status VARCHAR2(50)  
);
```

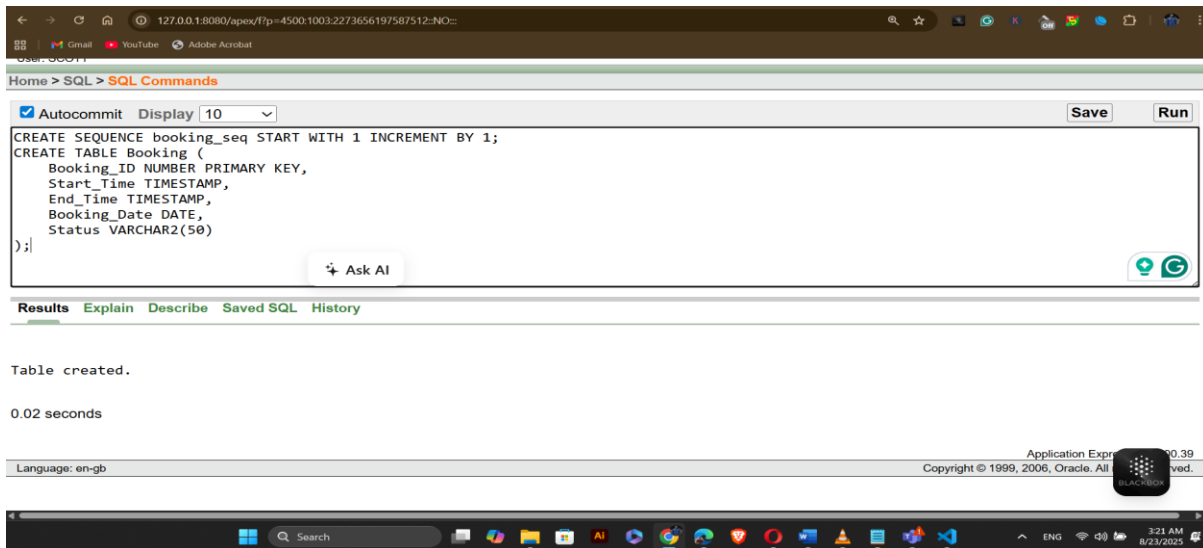


Table Name: Turf

CREATE SEQUENCE turfs_seq START WITH 1 INCREMENT BY 1;

CREATE TABLE Turf (

Turf_ID NUMBER PRIMARY KEY,

Turf_Name VARCHAR2(100),

Turf_location VARCHAR2(100),

Turf_size VARCHAR2(50),

Turf_type VARCHAR2(50)

);

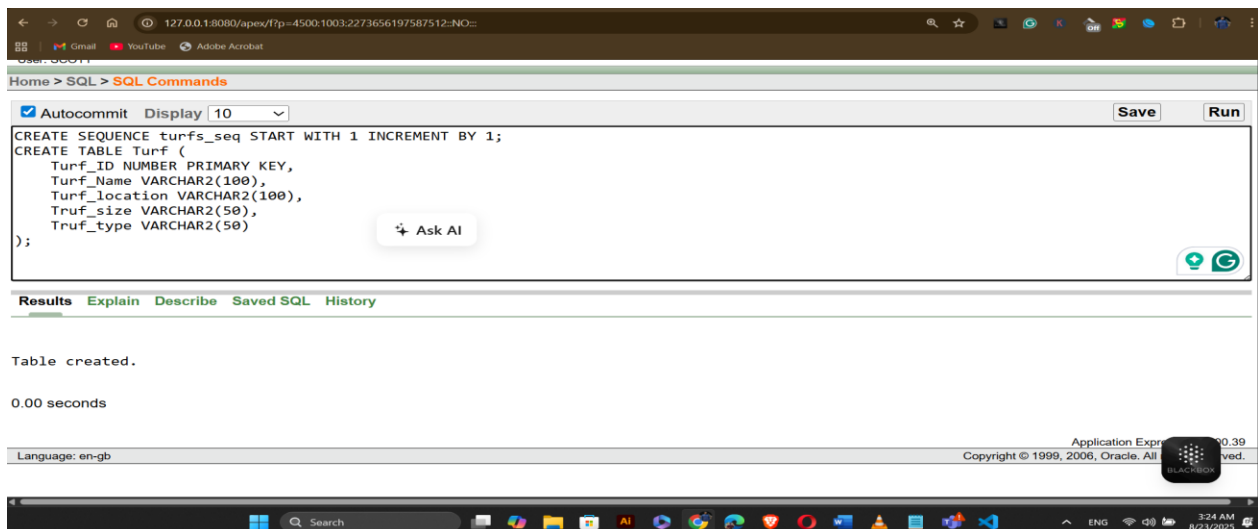


Table Name: Payment

CREATE SEQUENCE payment_seq START WITH 1 INCREMENT BY 1;

CREATE TABLE Payment (

 Payment_ID NUMBER PRIMARY KEY,

 Amount NUMBER(10,2),

 Payment_Date DATE,

 Payment_Type_Code NUMBER

);

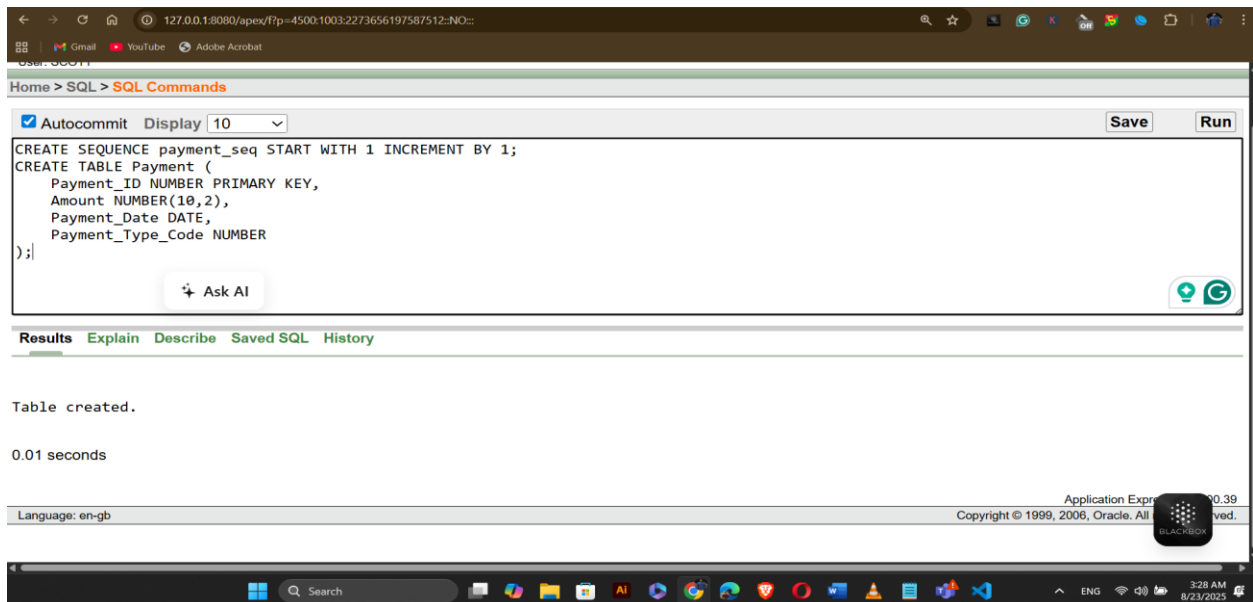


Table Name: Payment_Type

CREATE SEQUENCE payment_type_seq START WITH 1 INCREMENT BY 1;

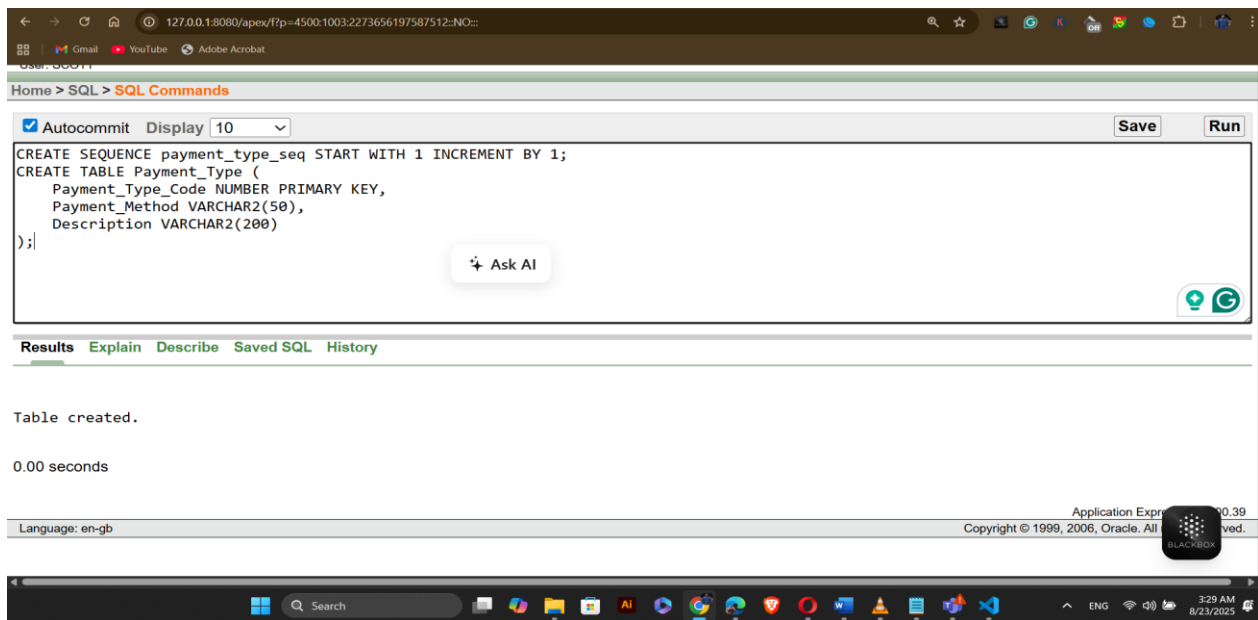
CREATE TABLE Payment_Type (

 Payment_Type_Code NUMBER PRIMARY KEY,

 Payment_Method VARCHAR2(50),

 Description VARCHAR2(200)

);



Relation Tables:

Table Name: Can_Manage

```
CREATE TABLE Can_Manage (  
    Staff_ID NUMBER,  
    Event_ID NUMBER,  
    PRIMARY KEY (Staff_ID, Event_ID),  
    FOREIGN KEY (Staff_ID) REFERENCES Staff_Members(Staff_ID),  
    FOREIGN KEY (Event_ID) REFERENCES Events(Event_ID)  
);
```

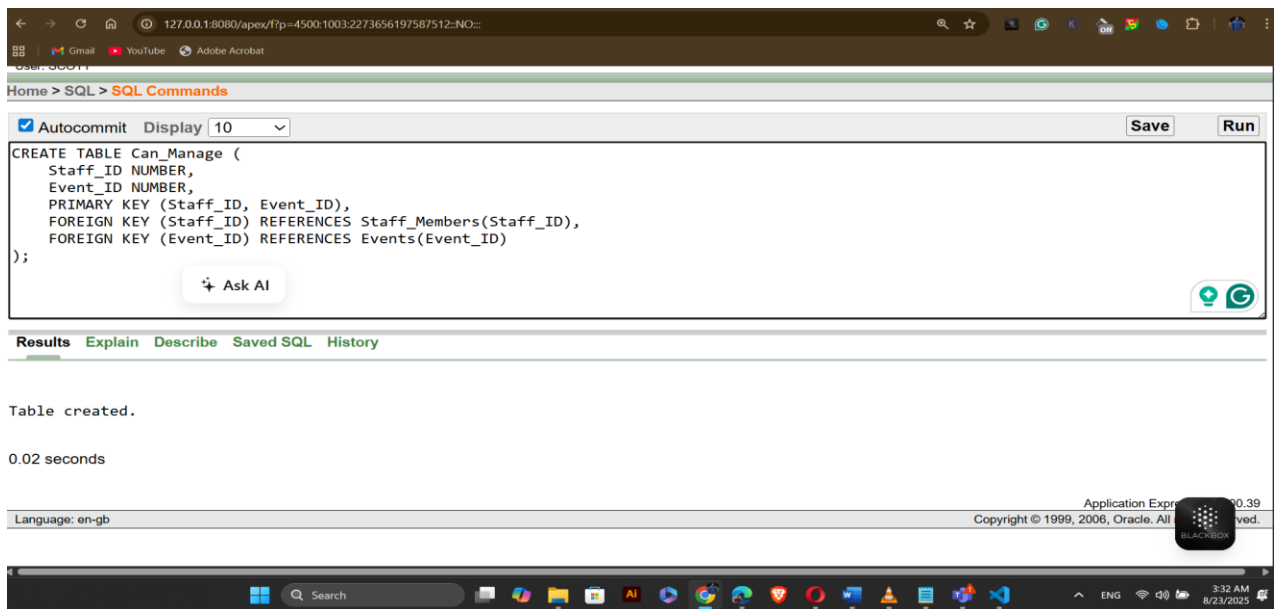


Table Name: Participates

```
CREATE TABLE Participates (
  Event_ID NUMBER,
  Team_ID NUMBER,
  PRIMARY KEY (Event_ID, Team_ID),
  FOREIGN KEY (Event_ID) REFERENCES Events(Event_ID),
  FOREIGN KEY (Team_ID) REFERENCES Team(Team_ID)
);
```

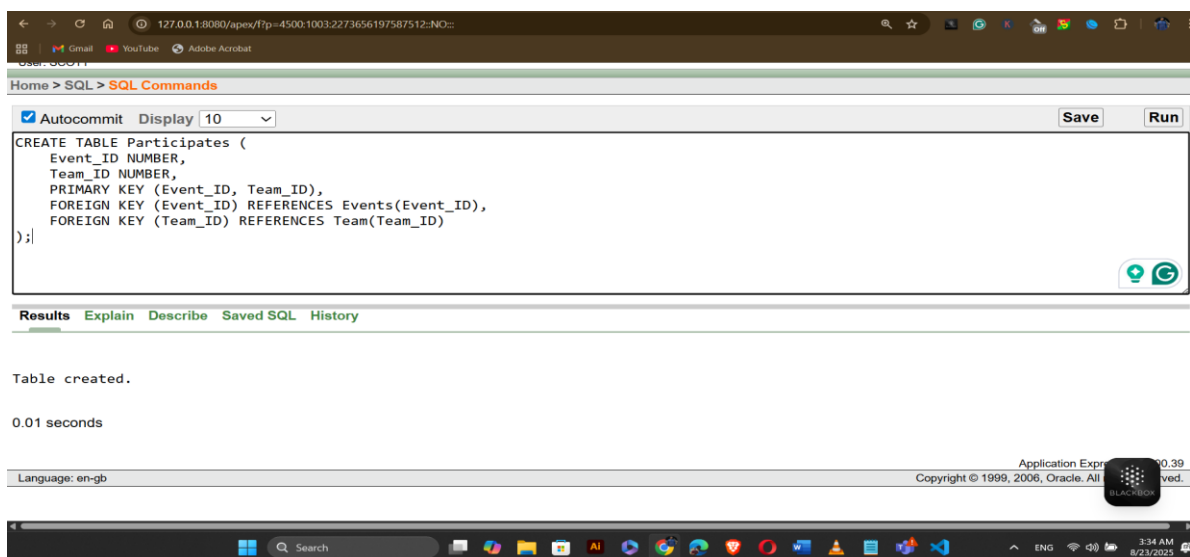


Table Name: Belongs_To

```
CREATE TABLE Belongs_To (  
    Player_ID NUMBER,  
    Team_ID NUMBER,  
    PRIMARY KEY (Player_ID, Team_ID),  
    FOREIGN KEY (Player_ID) REFERENCES Players(Player_ID),  
    FOREIGN KEY (Team_ID) REFERENCES Team(Team_ID)  
);
```

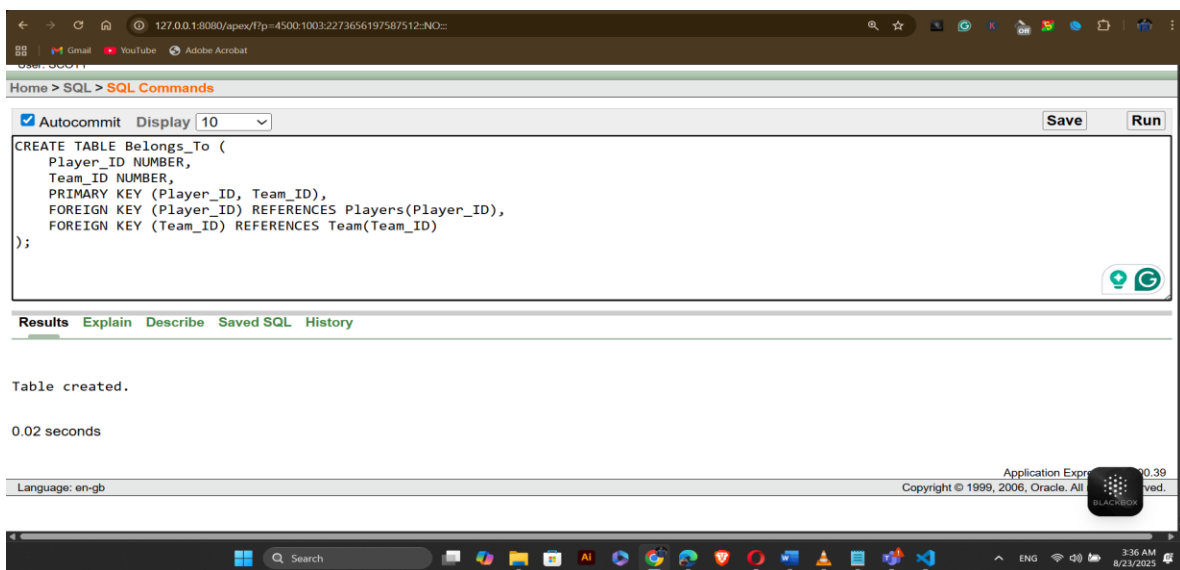


Table Name: Has_Profile

```
CREATE TABLE Has_Profile (  
    Player_ID NUMBER,  
    Profile_ID NUMBER,  
    PRIMARY KEY (Player_ID, Profile_ID),  
    FOREIGN KEY (Player_ID) REFERENCES Players(Player_ID),  
    FOREIGN KEY (Profile_ID) REFERENCES Profile(Profile_ID)  
);
```

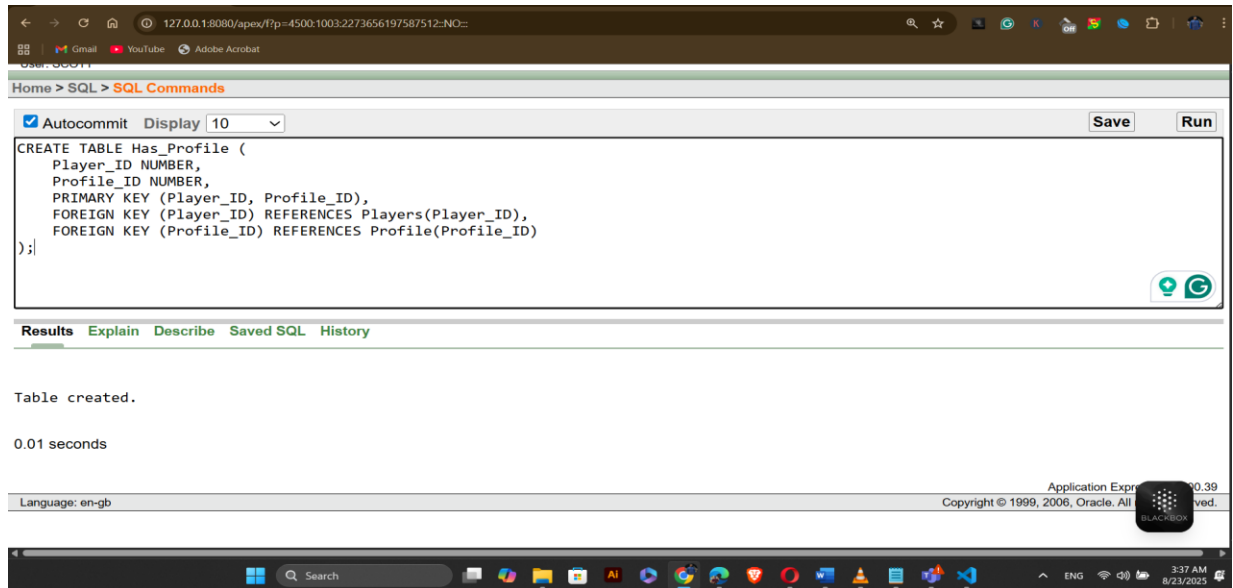



Table Name: Can_Make

```
CREATE TABLE Can_Make (  
  Team_ID NUMBER,  
  Booking_ID NUMBER,  
  PRIMARY KEY (Team_ID, Booking_ID),  
  FOREIGN KEY (Team_ID) REFERENCES Team(Team_ID),  
  FOREIGN KEY (Booking_ID) REFERENCES Booking(Booking_ID)  
);
```

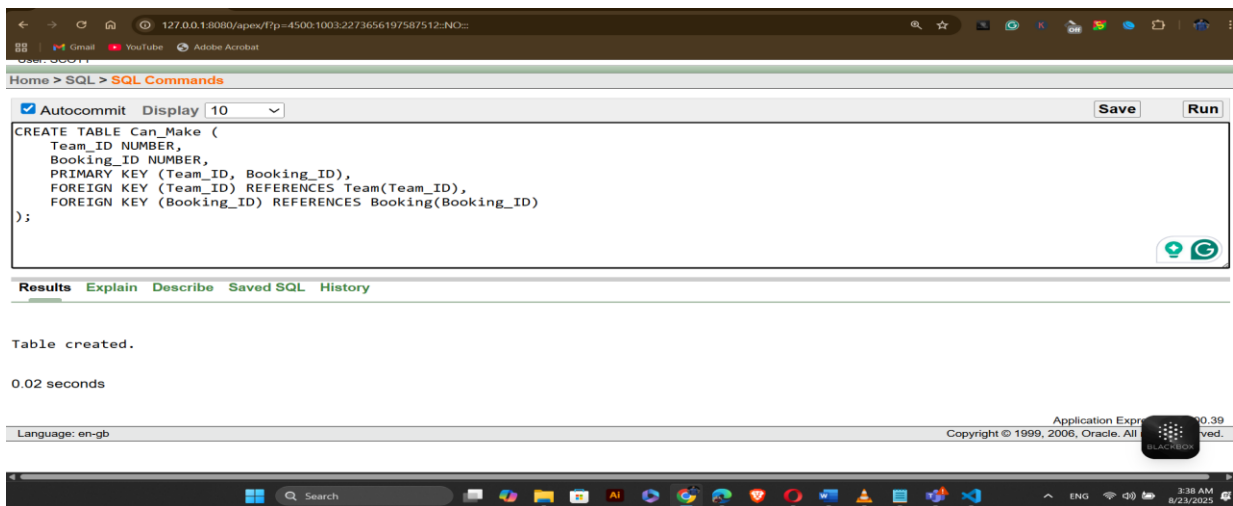


Table Name: Makes

```
CREATE TABLE Makes (  
    Player_ID NUMBER,  
    Booking_ID NUMBER,  
    PRIMARY KEY (Player_ID, Booking_ID),  
    FOREIGN KEY (Player_ID) REFERENCES Players(Player_ID),  
    FOREIGN KEY (Booking_ID) REFERENCES Booking(Booking_ID)  
);
```

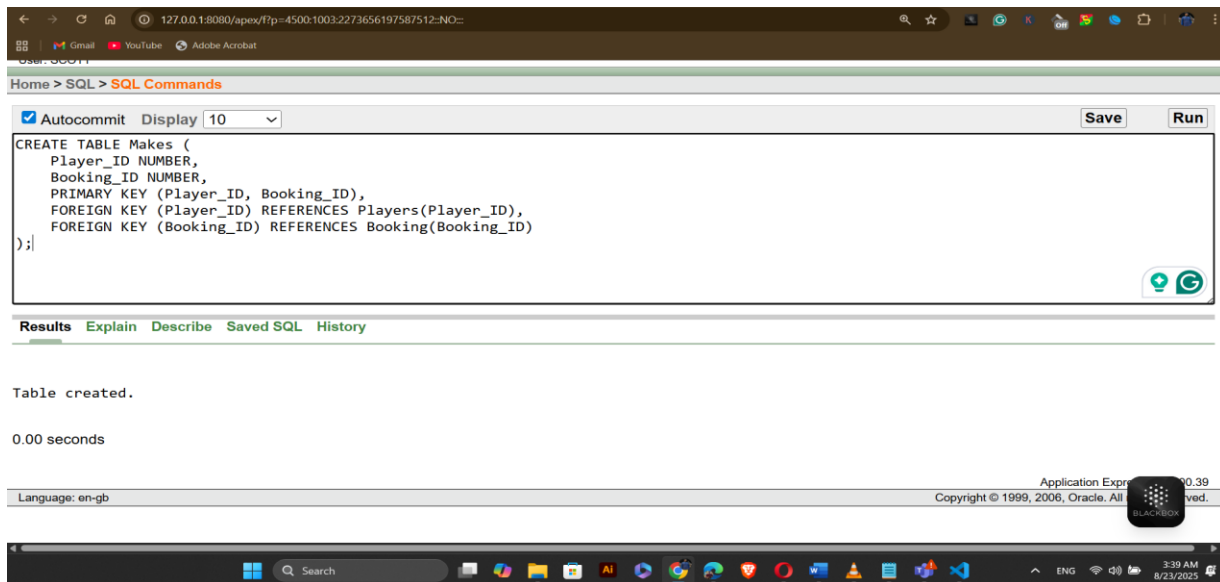


Table Name: Can_Be_Made

```
CREATE TABLE Can_Be_Made (  
    Booking_ID NUMBER,  
    Turf_ID NUMBER,  
    PRIMARY KEY (Booking_ID, Turf_ID),  
    FOREIGN KEY (Booking_ID) REFERENCES Booking(Booking_ID),  
    FOREIGN KEY (Turf_ID) REFERENCES Turf(Turf_ID)  
);
```

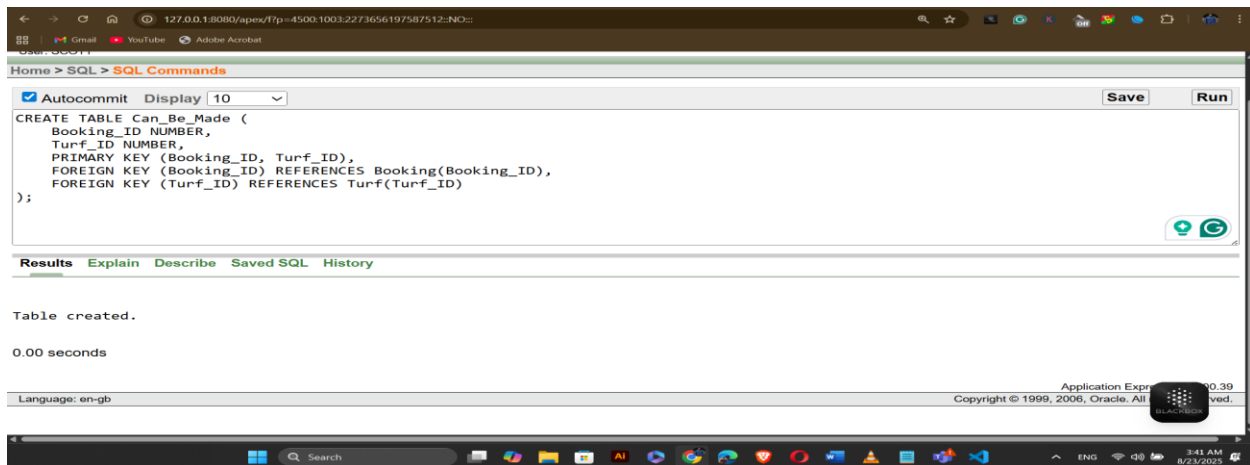


Table Name: Can_Have

```
CREATE TABLE Can_Have (  
  
  Booking_ID NUMBER,  
  
  Payment_ID NUMBER,  
  
  PRIMARY KEY (Booking_ID, Payment_ID),  
  
  FOREIGN KEY (Booking_ID) REFERENCES Booking(Booking_ID),  
  
  FOREIGN KEY (Payment_ID) REFERENCES Payment(Payment_ID)  
  
);
```

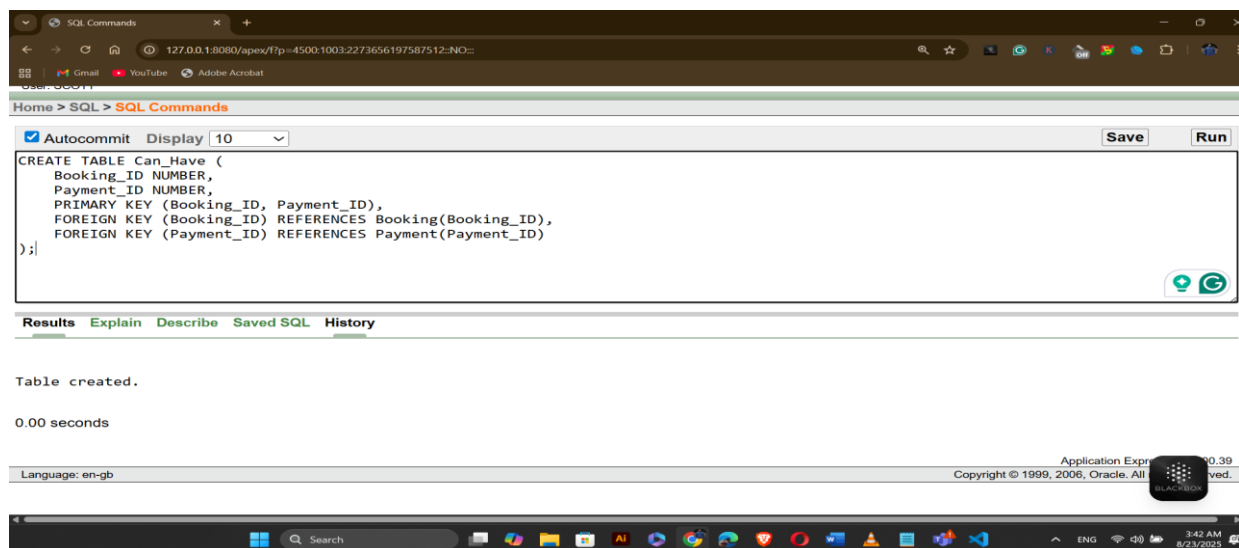
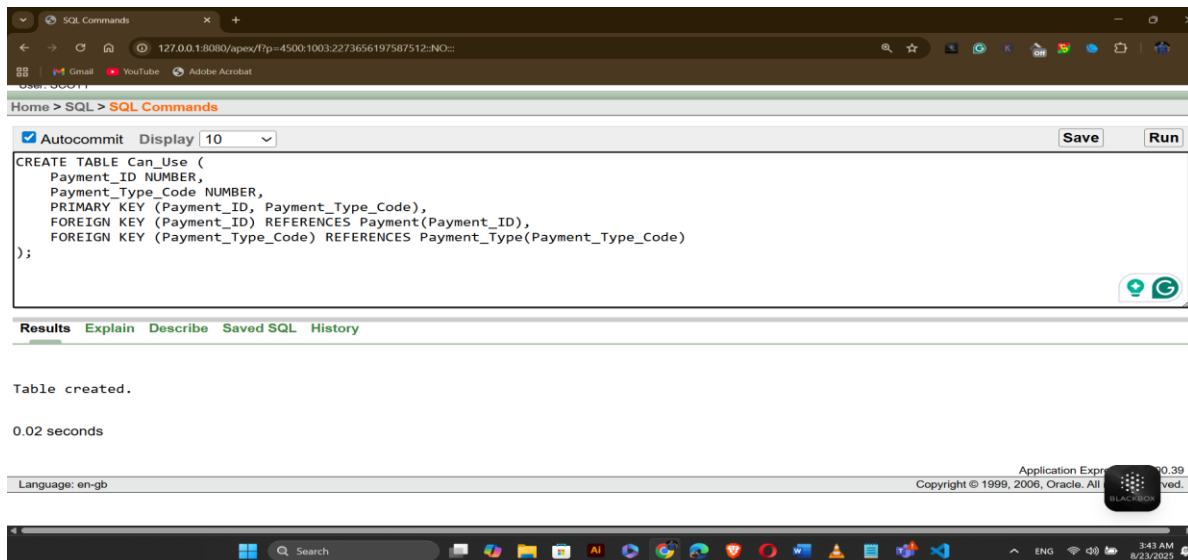


Table Name: Can_Use

```
CREATE TABLE Can_Use (  
    Payment_ID NUMBER,  
    Payment_Type_Code NUMBER,  
    PRIMARY KEY (Payment_ID, Payment_Type_Code),  
    FOREIGN KEY (Payment_ID) REFERENCES Payment(Payment_ID),  
    FOREIGN KEY (Payment_Type_Code) REFERENCES  
Payment_Type(Payment_Type_Code)  
);
```



9. DATA INSERTING:

Table Name: Staff_Members

```
INSERT INTO Staff_Members (Staff_ID, First_Name, Last_Name, Department) VALUES  
(staff_member_seq.NEXTVAL, 'alam', 'shakib', 'Operations');
```

```
INSERT INTO Staff_Members (Staff_ID, First_Name, Last_Name, Department) VALUES  
(staff_member_seq.NEXTVAL, 'rimon', 'promanik', 'Marketing');
```

```
INSERT INTO Staff_Members (Staff_ID, First_Name, Last_Name, Department) VALUES  
(staff_member_seq.NEXTVAL, 'maruf', 'maruf', 'Finance');
```

```
INSERT INTO Staff_Members (Staff_ID, First_Name, Last_Name, Department) VALUES  
(staff_member_seq.NEXTVAL, 'lamia', 'thahsin', 'HR');
```

```
INSERT INTO Staff_Members (Staff_ID, First_Name, Last_Name, Department) VALUES
(staff_member_seq.NEXTVAL, 'shadman', 'shakib', 'IT');
```

```
SELECT * FROM Staff_Members
```

The screenshot shows the Oracle SQL Developer interface. The top pane contains the following SQL commands:

```
INSERT INTO Staff_Members (Staff_ID, First_Name, Last_Name, Department) VALUES (staff_member_seq.NEXTVAL, 'alam', 'shakib', 'Operations');
INSERT INTO Staff_Members (Staff_ID, First_Name, Last_Name, Department) VALUES (staff_member_seq.NEXTVAL, 'rimon', 'promanik', 'Marketing');
INSERT INTO Staff_Members (Staff_ID, First_Name, Last_Name, Department) VALUES (staff_member_seq.NEXTVAL, 'maruf', 'maruf', 'Finance');
INSERT INTO Staff_Members (Staff_ID, First_Name, Last_Name, Department) VALUES (staff_member_seq.NEXTVAL, 'lamia', 'thahsin', 'HR');
INSERT INTO Staff_Members (Staff_ID, First_Name, Last_Name, Department) VALUES (staff_member_seq.NEXTVAL, 'shadman', 'shakib', 'IT');
SELECT * FROM Staff_Members
```

The bottom pane shows the results of the SQL commands. The results are displayed in a table with the following columns: STAFF_ID, FIRST_NAME, LAST_NAME, and DEPARTMENT. The table contains 5 rows of data.

STAFF_ID	FIRST_NAME	LAST_NAME	DEPARTMENT
1	alam	shakib	Operations
2	rimon	promanik	Marketing
3	maruf	maruf	Finance
4	lamia	thahsin	HR
5	shadman	shakib	IT

5 rows returned in 0.03 seconds

Table Name: Events

```
INSERT INTO Events (Event_ID, Event_Name, Event_Date, Rules) VALUES
(event_seq.NEXTVAL, 'Football Championship', TO_DATE('2025-09-01', 'YYYY-MM-DD'),
'Standard FIFA rules');
```

```
INSERT INTO Events (Event_ID, Event_Name, Event_Date, Rules) VALUES
(event_seq.NEXTVAL, 'Basketball Tournament', TO_DATE('2025-09-15', 'YYYY-MM-DD'),
'NBA rules');
```

```
INSERT INTO Events (Event_ID, Event_Name, Event_Date, Rules) VALUES
(event_seq.NEXTVAL, 'Cricket League', TO_DATE('2025-10-10', 'YYYY-MM-DD'), 'ICC
rules');
```

```
INSERT INTO Events (Event_ID, Event_Name, Event_Date, Rules) VALUES
(event_seq.NEXTVAL, 'Tennis Open', TO_DATE('2025-11-05', 'YYYY-MM-DD'), 'ATP rules');
```

```
INSERT INTO Events (Event_ID, Event_Name, Event_Date, Rules) VALUES
(event_seq.NEXTVAL, 'Hockey Cup', TO_DATE('2025-12-20', 'YYYY-MM-DD'), 'FIH rules');
```

```
SELECT * FROM Events
```

The screenshot shows a web-based SQL interface. The command window contains the following SQL commands:

```

'YYYY-MM-DD'), 'NBA rules');
INSERT INTO Events (Event_ID, Event_Name, Event_Date, Rules) VALUES (event_seq.NEXTVAL, 'Cricket League', TO_DATE('2025-10-10', 'YYYY-MM-DD'), 'ICC rules');
INSERT INTO Events (Event_ID, Event_Name, Event_Date, Rules) VALUES (event_seq.NEXTVAL, 'Tennis Open', TO_DATE('2025-11-05', 'YYYY-MM-DD'), 'ATP rules');
INSERT INTO Events (Event_ID, Event_Name, Event_Date, Rules) VALUES (event_seq.NEXTVAL, 'Hockey Cup', TO_DATE('2025-12-20', 'YYYY-MM-DD'), 'FIH rules');

SELECT * FROM Events
  
```

The results section displays a table with 5 rows:

EVENT_ID	EVENT_NAME	EVENT DATE	RULES
1	Football Championship	01-SEP-25	Standard FIFA rules
2	Basketball Tournament	15-SEP-25	NBA rules
3	Cricket League	10-OCT-25	ICC rules
4	Tennis Open	05-NOV-25	ATP rules
5	Hockey Cup	20-DEC-25	FIH rules

5 rows returned in 0.03 seconds

Table Name: Team

INSERT INTO Team (Team_ID, Team_Name, Sport_Type) VALUES (team_seq.NEXTVAL, 'Warriors', 'Football');

INSERT INTO Team (Team_ID, Team_Name, Sport_Type) VALUES (team_seq.NEXTVAL, 'Titans', 'Basketball');

INSERT INTO Team (Team_ID, Team_Name, Sport_Type) VALUES (team_seq.NEXTVAL, 'Strikers', 'Cricket');

INSERT INTO Team (Team_ID, Team_Name, Sport_Type) VALUES (team_seq.NEXTVAL, 'Aces', 'Tennis');

INSERT INTO Team (Team_ID, Team_Name, Sport_Type) VALUES (team_seq.NEXTVAL, 'Blazers', 'Hockey');

SELECT * FROM Team

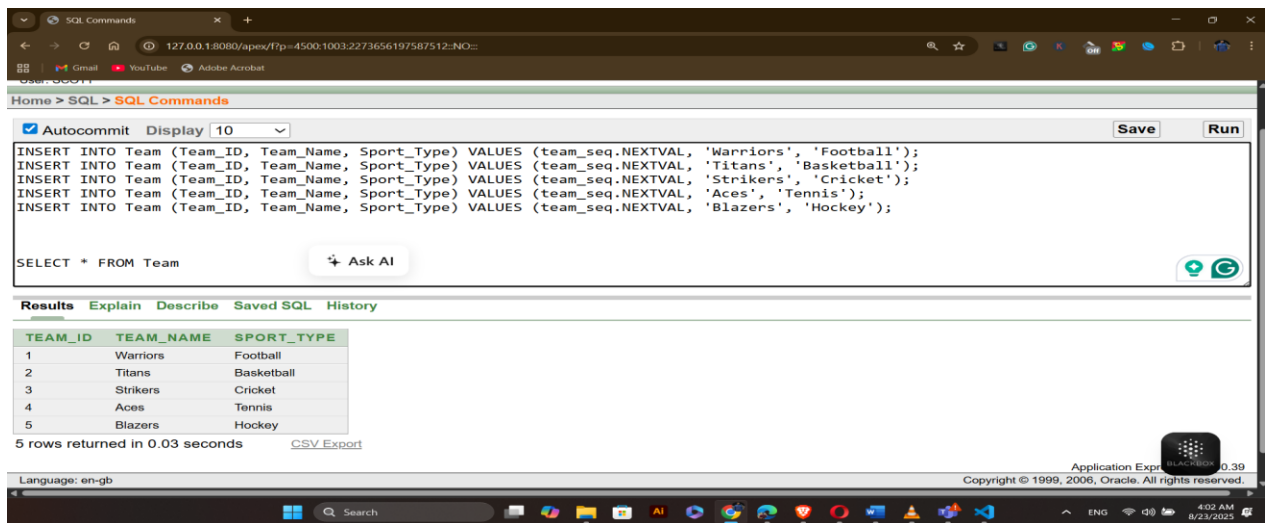


Table Name: Players

INSERT INTO Players (Player_ID, First_Name, Last_Name, DOB, Phone_Number) VALUES (players_seq.NEXTVAL, 'shadman', 'alam', TO_DATE('1990-01-01', 'YYYY-MM-DD'), '01888488409');

INSERT INTO Players (Player_ID, First_Name, Last_Name, DOB, Phone_Number) VALUES (players_seq.NEXTVAL, 'amin', 'ali', TO_DATE('1992-04-15', 'YYYY-MM-DD'), '01677747747');

INSERT INTO Players (Player_ID, First_Name, Last_Name, DOB, Phone_Number) VALUES (players_seq.NEXTVAL, 'arif', 'aslma', TO_DATE('1985-06-13', 'YYYY-MM-DD'), '0198787766');

INSERT INTO Players (Player_ID, First_Name, Last_Name, DOB, Phone_Number) VALUES (players_seq.NEXTVAL, 'tanvir', 'evan', TO_DATE('1984-11-22', 'YYYY-MM-DD'), '01715748774');

INSERT INTO Players (Player_ID, First_Name, Last_Name, DOB, Phone_Number) VALUES (players_seq.NEXTVAL, 'mainul', 'islam', TO_DATE('1965-04-04', 'YYYY-MM-DD'), '0181660948');

SELECT * FROM Players

The screenshot shows the Oracle SQL Developer interface. The top pane contains the following SQL commands:

```

04-15', 'YYYY-MM-DD'), '01677747747');
INSERT INTO Players (Player_ID, First_Name, Last_Name, DOB, Phone_Number) VALUES (players_seq.NEXTVAL, 'arif', 'asima', TO_DATE('1985-06-13', 'YYYY-MM-DD'), '0198787766');
INSERT INTO Players (Player_ID, First_Name, Last_Name, DOB, Phone_Number) VALUES (players_seq.NEXTVAL, 'tanvir', 'evan', TO_DATE('1984-11-22', 'YYYY-MM-DD'), '01715748774');
INSERT INTO Players (Player_ID, First_Name, Last_Name, DOB, Phone_Number) VALUES (players_seq.NEXTVAL, 'mainul', 'islam', TO_DATE('1965-04-04', 'YYYY-MM-DD'), '0181660948');

SELECT * FROM Players

```

The bottom pane shows the results of the query, displaying 5 rows of data from the 'Players' table:

PLAYER_ID	FIRST_NAME	LAST_NAME	DOB	PHONE_NUMBER
1	shadman	alam	01-JAN-90	01888488409
2	amin	ali	15-APR-92	01677747747
3	arif	asima	13-JUN-85	0198787766
4	tanvir	evan	22-NOV-84	01715748774
5	mainul	islam	04-APR-65	0181660948

5 rows returned in 0.03 seconds

Table Name: Profile

INSERT INTO Profile (Profile_ID, Skill_Level, Achievements) VALUES (profile_seq.NEXTVAL, 'Advanced', 'MVP 2022');

INSERT INTO Profile (Profile_ID, Skill_Level, Achievements) VALUES (profile_seq.NEXTVAL, 'Intermediate', 'Best Player 2021');

INSERT INTO Profile (Profile_ID, Skill_Level, Achievements) VALUES (profile_seq.NEXTVAL, 'Beginner', 'Rookie of the Year');

INSERT INTO Profile (Profile_ID, Skill_Level, Achievements) VALUES (profile_seq.NEXTVAL, 'Advanced', 'Champion 2020');

INSERT INTO Profile (Profile_ID, Skill_Level, Achievements) VALUES (profile_seq.NEXTVAL, 'Intermediate', 'Top Scorer 2019');

SELECT * FROM Profile

The screenshot shows an SQL interface with a query window and a results window. The query window contains the following SQL statements:

```

INSERT INTO Profile (Profile_ID, Skill_Level, Achievements) VALUES (profile_seq.NEXTVAL, 'Advanced', 'MVP 2022');
INSERT INTO Profile (Profile_ID, Skill_Level, Achievements) VALUES (profile_seq.NEXTVAL, 'Intermediate', 'Best Player 2021');
INSERT INTO Profile (Profile_ID, Skill_Level, Achievements) VALUES (profile_seq.NEXTVAL, 'Beginner', 'Rookie of the Year');
INSERT INTO Profile (Profile_ID, Skill_Level, Achievements) VALUES (profile_seq.NEXTVAL, 'Advanced', 'Champion 2020');
INSERT INTO Profile (Profile_ID, Skill_Level, Achievements) VALUES (profile_seq.NEXTVAL, 'Intermediate', 'Top Scorer 2019');

SELECT * FROM Profile

```

The results window shows the following data:

PROFILE_ID	SKILL_LEVEL	ACHIEVEMENTS
1	Advanced	MVP 2022
2	Intermediate	Best Player 2021
3	Beginner	Rookie of the Year
4	Advanced	Champion 2020
5	Intermediate	Top Scorer 2019

5 rows returned in 0.01 seconds

Table Name: Booking

```

INSERT INTO Booking (Booking_ID, Start_Time, End_Time, Booking_Date, Status) VALUES
(booking_seq.NEXTVAL, TO_TIMESTAMP('2025-08-01 10:00:00', 'YYYY-MM-DD
HH24:MI:SS'), TO_TIMESTAMP('2025-08-01 12:00:00', 'YYYY-MM-DD HH24:MI:SS'),
TO_DATE('2025-08-01', 'YYYY-MM-DD'), 'Confirmed');

```

```

INSERT INTO Booking (Booking_ID, Start_Time, End_Time, Booking_Date, Status) VALUES
(booking_seq.NEXTVAL, TO_TIMESTAMP('2025-08-02 14:00:00', 'YYYY-MM-DD
HH24:MI:SS'), TO_TIMESTAMP('2025-08-02 16:00:00', 'YYYY-MM-DD HH24:MI:SS'),
TO_DATE('2025-08-02', 'YYYY-MM-DD'), 'Pending');

```

```

INSERT INTO Booking (Booking_ID, Start_Time, End_Time, Booking_Date, Status) VALUES
(booking_seq.NEXTVAL, TO_TIMESTAMP('2025-08-03 09:00:00', 'YYYY-MM-DD
HH24:MI:SS'), TO_TIMESTAMP('2025-08-03 11:00:00', 'YYYY-MM-DD HH24:MI:SS'),
TO_DATE('2025-08-03', 'YYYY-MM-DD'), 'Cancelled');

```

```

INSERT INTO Booking (Booking_ID, Start_Time, End_Time, Booking_Date, Status) VALUES
(booking_seq.NEXTVAL, TO_TIMESTAMP('2025-08-04 13:00:00', 'YYYY-MM-DD
HH24:MI:SS'), TO_TIMESTAMP('2025-08-04 15:00:00', 'YYYY-MM-DD HH24:MI:SS'),
TO_DATE('2025-08-04', 'YYYY-MM-DD'), 'Confirmed');

```

```

INSERT INTO Booking (Booking_ID, Start_Time, End_Time, Booking_Date, Status) VALUES
(booking_seq.NEXTVAL, TO_TIMESTAMP('2025-08-05 17:00:00', 'YYYY-MM-DD
HH24:MI:SS'), TO_TIMESTAMP('2025-08-05 19:00:00', 'YYYY-MM-DD HH24:MI:SS'),
TO_DATE('2025-08-05', 'YYYY-MM-DD'), 'Pending');

```

```

SELECT * FROM Booking

```

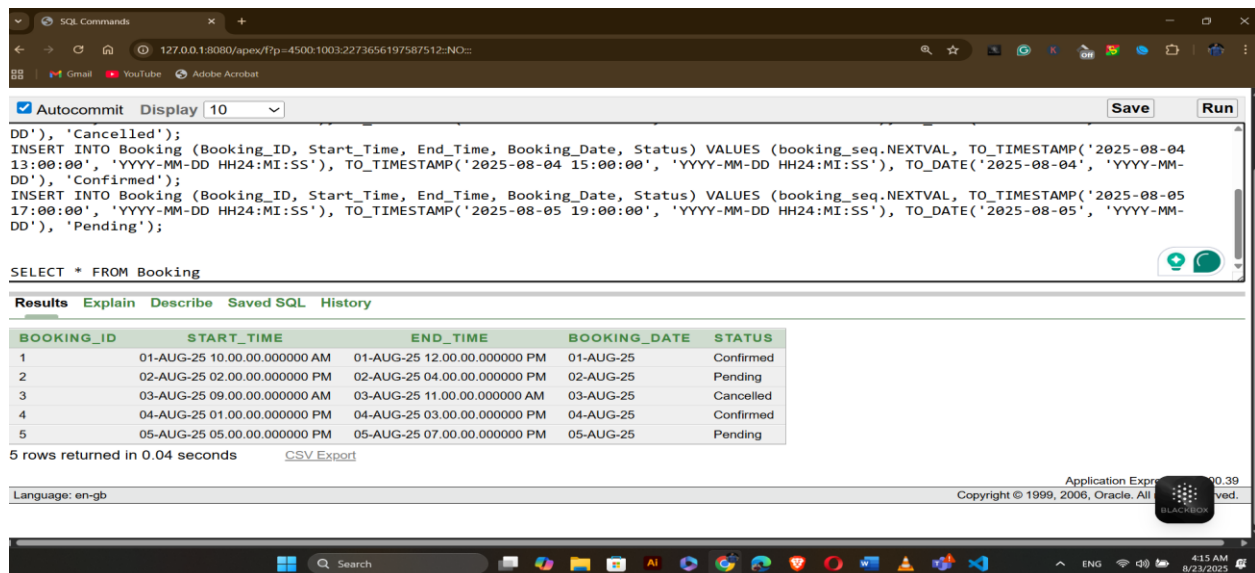


Table Name: Turf

INSERT INTO Turf (TURF_ID, TURF_NAME, TURF_LOCATION, TRUF_SIZE, TRUF_TYPE) VALUES (turfs_seq.NEXTVAL, 'Green Field', 'Mirpur-2', '100x50', 'Grass');

INSERT INTO Turf (Turf_ID, Turf_Name, Turf_location, Truf_size, Truf_type) VALUES (turfs_seq.NEXTVAL, 'Blue Arena', 'Mirpur-10', '80x40', 'Synthetic');

INSERT INTO Turf (Turf_ID, Turf_Name, Turf_location, Truf_size, Truf_type) VALUES (turfs_seq.NEXTVAL, 'Red Court', 'Badda', '60x30', 'Clay');

INSERT INTO Turf (Turf_ID, Turf_Name, Turf_location, Truf_size, Truf_type) VALUES (turfs_seq.NEXTVAL, 'Yellow Pitch', 'Uttora', '120x60', 'Artificial');

INSERT INTO Turf (Turf_ID, Turf_Name, Turf_location, Truf_size, Truf_type) VALUES (turfs_seq.NEXTVAL, 'White Ground', 'Nikunjo-2', '90x45', 'Hybrid');

SELECT * FROM Turf

The screenshot shows an SQL Command window with the following content:

```

INSERT INTO Turf (Turf_ID, Turf_Name, Turf_location, Truf_size, Truf_type) VALUES (turfs_seq.NEXTVAL, 'Red Court', 'Badda', '60x30', 'Clay');
INSERT INTO Turf (Turf_ID, Turf_Name, Turf_location, Truf_size, Truf_type) VALUES (turfs_seq.NEXTVAL, 'Yellow Pitch', 'Uttora', '120x60', 'Artificial');
INSERT INTO Turf (Turf_ID, Turf_Name, Turf_location, Truf_size, Truf_type) VALUES (turfs_seq.NEXTVAL, 'White Ground', 'Nikunjo-2', '90x45', 'Hybrid');

SELECT * FROM Turf
  
```

The results section shows a table with 5 rows:

TURF_ID	TURF_NAME	TURF_LOCATION	TRUF_SIZE	TRUF_TYPE
1	Green Field	Mirpur-2	100x50	Grass
2	Blue Arena	Mirpur-10	80x40	Synthetic
3	Red Court	Badda	60x30	Clay
4	Yellow Pitch	Uttora	120x60	Artificial
5	White Ground	Nikunjo-2	90x45	Hybrid

5 rows returned in 0.02 seconds

Table Name: Payment_Type

INSERT INTO Payment_Type (Payment_Type_Code, Payment_Method, Description) VALUES (payment_type_seq.NEXTVAL, 'Credit Card', 'Visa or MasterCard');

INSERT INTO Payment_Type (Payment_Type_Code, Payment_Method, Description) VALUES (payment_type_seq.NEXTVAL, 'Debit Card', 'Bank issued debit cards');

INSERT INTO Payment_Type (Payment_Type_Code, Payment_Method, Description) VALUES (payment_type_seq.NEXTVAL, 'Cash', 'Physical currency');

INSERT INTO Payment_Type (Payment_Type_Code, Payment_Method, Description) VALUES (payment_type_seq.NEXTVAL, 'Online Transfer', 'Bank to bank transfer');

INSERT INTO Payment_Type (Payment_Type_Code, Payment_Method, Description) VALUES (payment_type_seq.NEXTVAL, 'Mobile Payment', 'Payment via mobile apps');

SELECT * FROM Payment_Type

The screenshot shows the Oracle SQL Developer interface. The top pane displays the following SQL commands:

```

debit cards');
INSERT INTO Payment_Type (Payment_Type_Code, Payment_Method, Description) VALUES (payment_type_seq.NEXTVAL, 'Cash', 'Physical
currency');
INSERT INTO Payment_Type (Payment_Type_Code, Payment_Method, Description) VALUES (payment_type_seq.NEXTVAL, 'Online Transfer', 'Bank
to bank transfer');
INSERT INTO Payment_Type (Payment_Type_Code, Payment_Method, Description) VALUES (payment_type_seq.NEXTVAL, 'Mobile Payment', 'Payment
via mobile apps');
SELECT * FROM Payment_Type
  
```

The bottom pane shows the results of the last query, which is a table with 5 rows:

PAYMENT_TYPE_CODE	PAYMENT_METHOD	DESCRIPTION
1	Credit Card	Visa or MasterCard
2	Debit Card	Bank issued debit cards
3	Cash	Physical currency
4	Online Transfer	Bank to bank transfer
5	Mobile Payment	Payment via mobile apps

Below the table, it states "5 rows returned in 0.01 seconds" and provides a "CSV Export" link. The bottom status bar shows "Language: en-gb" and "Copyright © 1999, 2008, Oracle. All rights reserved."

Table Name: Payment

INSERT INTO Payment (Payment_ID, Amount, Payment_Date, Payment_Type_Code)
VALUES (payment_seq.NEXTVAL, 1000.00, TO_DATE('2025-08-01', 'YYYY-MM-DD'), 1);

INSERT INTO Payment (Payment_ID, Amount, Payment_Date, Payment_Type_Code)
VALUES (payment_seq.NEXTVAL, 1500.00, TO_DATE('2025-08-02', 'YYYY-MM-DD'), 2);

INSERT INTO Payment (Payment_ID, Amount, Payment_Date, Payment_Type_Code)
VALUES (payment_seq.NEXTVAL, 2000.00, TO_DATE('2025-08-03', 'YYYY-MM-DD'), 3);

INSERT INTO Payment (Payment_ID, Amount, Payment_Date, Payment_Type_Code)
VALUES (payment_seq.NEXTVAL, 2500.00, TO_DATE('2025-08-04', 'YYYY-MM-DD'), 4);

INSERT INTO Payment (Payment_ID, Amount, Payment_Date, Payment_Type_Code)
VALUES (payment_seq.NEXTVAL, 3000.00, TO_DATE('2025-08-05', 'YYYY-MM-DD'), 5);

SELECT * FROM Payment;

The screenshot shows a web browser window with the URL `127.0.0.1:8080/apex/f?p=4500:1003:2424709462361380:NO::`. The page title is "SQL Commands". The "Autocommit" checkbox is checked, and the "Display" dropdown is set to 10. The SQL command entered is:

```
INSERT INTO Payment (Payment_ID, Amount, Payment_Date, Payment_Type_Code) VALUES (payment_seq.NEXTVAL, 1500.00, TO_DATE('2025-08-02', 'YYYY-MM-DD'), 2);
INSERT INTO Payment (Payment_ID, Amount, Payment_Date, Payment_Type_Code) VALUES (payment_seq.NEXTVAL, 2000.00, TO_DATE('2025-08-03', 'YYYY-MM-DD'), 3);
INSERT INTO Payment (Payment_ID, Amount, Payment_Date, Payment_Type_Code) VALUES (payment_seq.NEXTVAL, 2500.00, TO_DATE('2025-08-04', 'YYYY-MM-DD'), 4);
INSERT INTO Payment (Payment_ID, Amount, Payment_Date, Payment_Type_Code) VALUES (payment_seq.NEXTVAL, 3000.00, TO_DATE('2025-08-05', 'YYYY-MM-DD'), 5);
SELECT * FROM Payment;
```

The "Results" tab is selected, showing a table with 5 rows:

PAYMENT_ID	AMOUNT	PAYMENT_DATE	PAYMENT_TYPE_CODE
1	1000	01-AUG-25	1
2	1500	02-AUG-25	2
21	2000	03-AUG-25	3
22	2500	04-AUG-25	4
23	3000	05-AUG-25	5

5 rows returned in 0.01 seconds. A "CSV Export" link is available. The bottom status bar shows "Language: en-gb" and "Copyright © 1999, 2006, Oracle. All rights reserved."

Table Name: Can_Manage

INSERT INTO Can_Manage (Staff_ID, Event_ID) VALUES (1, 1);

INSERT INTO Can_Manage (Staff_ID, Event_ID) VALUES (2, 2);

INSERT INTO Can_Manage (Staff_ID, Event_ID) VALUES (3, 3);

INSERT INTO Can_Manage (Staff_ID, Event_ID) VALUES (4, 4);

INSERT INTO Can_Manage (Staff_ID, Event_ID) VALUES (5, 5);

SELECT * FROM Can_Manage;

The screenshot shows the same web browser window with the URL `127.0.0.1:8080/apex/f?p=4500:1003:2424709462361380:NO::`. The "Autocommit" checkbox is checked, and the "Display" dropdown is set to 10. The SQL command entered is:

```
INSERT INTO Can_Manage (Staff_ID, Event_ID) VALUES (1, 1);
INSERT INTO Can_Manage (Staff_ID, Event_ID) VALUES (2, 2);
INSERT INTO Can_Manage (Staff_ID, Event_ID) VALUES (3, 3);
INSERT INTO Can_Manage (Staff_ID, Event_ID) VALUES (4, 4);
INSERT INTO Can_Manage (Staff_ID, Event_ID) VALUES (5, 5);
SELECT * FROM Can_Manage;
```

The "Results" tab is selected, showing a table with 5 rows:

STAFF_ID	EVENT_ID
1	1
2	2
3	3
4	4
5	5

5 rows returned in 0.00 seconds. A "CSV Export" link is available. The bottom status bar shows "Language: en-gb" and "Copyright © 1999, 2006, Oracle. All rights reserved."

Table Name: Participates

INSERT INTO Participates (Event_ID, Team_ID) VALUES (1, 1);

INSERT INTO Participates (Event_ID, Team_ID) VALUES (2, 2);

INSERT INTO Participates (Event_ID, Team_ID) VALUES (3, 3);

INSERT INTO Participates (Event_ID, Team_ID) VALUES (4, 4);

INSERT INTO Participates (Event_ID, Team_ID) VALUES (5, 5);

SELECT * FROM Participates;

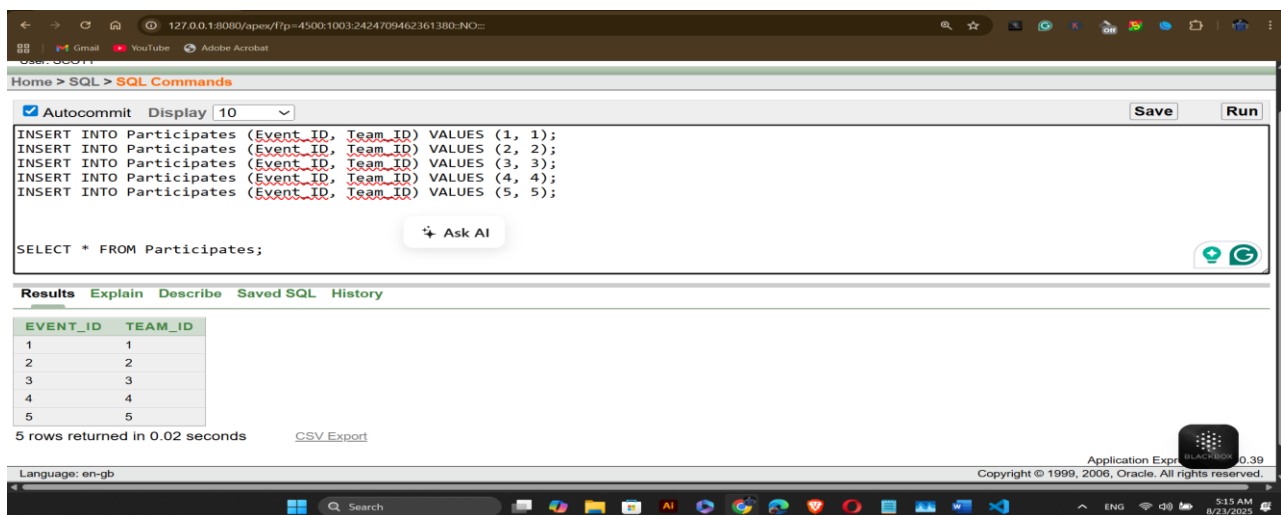


Table Name: Belongs_To

INSERT INTO Belongs_To (Player_ID, Team_ID) VALUES (1, 1);

INSERT INTO Belongs_To (Player_ID, Team_ID) VALUES (2, 2);

INSERT INTO Belongs_To (Player_ID, Team_ID) VALUES (3, 3);

INSERT INTO Belongs_To (Player_ID, Team_ID) VALUES (4, 4);

INSERT INTO Belongs_To (Player_ID, Team_ID) VALUES (5, 5);

SELECT * FROM Belongs_To ;

SQL Commands

Autocommit Display 10 Save Run

```

INSERT INTO Belongs_To (Player_ID, Team_ID) VALUES (1, 1);
INSERT INTO Belongs_To (Player_ID, Team_ID) VALUES (2, 2);
INSERT INTO Belongs_To (Player_ID, Team_ID) VALUES (3, 3);
INSERT INTO Belongs_To (Player_ID, Team_ID) VALUES (4, 4);
INSERT INTO Belongs_To (Player_ID, Team_ID) VALUES (5, 5);

SELECT * FROM Belongs_To ;

```

Results Explain Describe Saved SQL History

PLAYER_ID	TEAM_ID
1	1
2	2
3	3
4	4
5	5

5 rows returned in 0.00 seconds CSV Export

Language: en-gb Copyright © 1999, 2006, Oracle. All rights reserved. 5:17 AM 8/23/2025

Table Name: Has_Profile

```

INSERT INTO Has_Profile (Player_ID, Profile_ID) VALUES (1, 1);
INSERT INTO Has_Profile (Player_ID, Profile_ID) VALUES (2, 2);
INSERT INTO Has_Profile (Player_ID, Profile_ID) VALUES (3, 3);
INSERT INTO Has_Profile (Player_ID, Profile_ID) VALUES (4, 4);
INSERT INTO Has_Profile (Player_ID, Profile_ID) VALUES (5, 5);

SELECT * FROM Has_Profile ;

```

SQL Commands

Autocommit Display 10 Save Run

```

INSERT INTO Has_Profile (Player_ID, Profile_ID) VALUES (1, 1);
INSERT INTO Has_Profile (Player_ID, Profile_ID) VALUES (2, 2);
INSERT INTO Has_Profile (Player_ID, Profile_ID) VALUES (3, 3);
INSERT INTO Has_Profile (Player_ID, Profile_ID) VALUES (4, 4);
INSERT INTO Has_Profile (Player_ID, Profile_ID) VALUES (5, 5);

SELECT * FROM Has_Profile ;

```

Results Explain Describe Saved SQL History

PLAYER_ID	PROFILE_ID
1	1
2	2
3	3
4	4
5	5

5 rows returned in 0.00 seconds CSV Export

Language: en-gb Copyright © 1999, 2006, Oracle. All rights reserved. 5:19 AM 8/23/2025

Table Name: Can_Make

INSERT INTO Can_Make (Team_ID, Booking_ID) VALUES (1, 1);

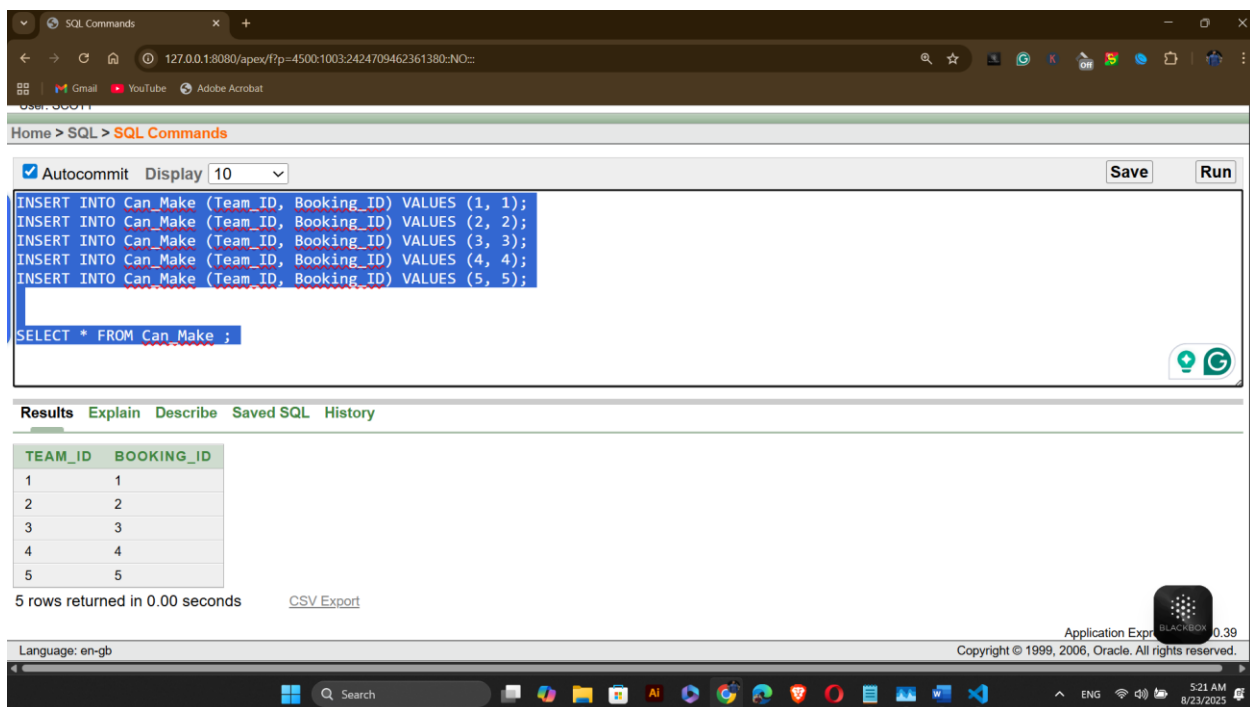
INSERT INTO Can_Make (Team_ID, Booking_ID) VALUES (2, 2);

INSERT INTO Can_Make (Team_ID, Booking_ID) VALUES (3, 3);

INSERT INTO Can_Make (Team_ID, Booking_ID) VALUES (4, 4);

INSERT INTO Can_Make (Team_ID, Booking_ID) VALUES (5, 5);

SELECT * FROM Can_Make ;



The screenshot shows the Oracle SQL Developer interface. The top pane displays the following SQL commands:

```
INSERT INTO Can_Make (Team_ID, Booking_ID) VALUES (1, 1);
INSERT INTO Can_Make (Team_ID, Booking_ID) VALUES (2, 2);
INSERT INTO Can_Make (Team_ID, Booking_ID) VALUES (3, 3);
INSERT INTO Can_Make (Team_ID, Booking_ID) VALUES (4, 4);
INSERT INTO Can_Make (Team_ID, Booking_ID) VALUES (5, 5);

SELECT * FROM Can_Make ;
```

The bottom pane shows the results of the SELECT statement:

TEAM_ID	BOOKING_ID
1	1
2	2
3	3
4	4
5	5

5 rows returned in 0.00 seconds

Language: en-gb

Copyright © 1999, 2006, Oracle. All rights reserved.

Application Express 4.2.3.9

Table Name: Makes

INSERT INTO Makes (Player_ID, Booking_ID) VALUES (1, 1);

INSERT INTO Makes (Player_ID, Booking_ID) VALUES (2, 2);

INSERT INTO Makes (Player_ID, Booking_ID) VALUES (3, 3);

INSERT INTO Makes (Player_ID, Booking_ID) VALUES (4, 4);

INSERT INTO Makes (Player_ID, Booking_ID) VALUES (5, 5);

SELECT * FROM Makes ;

The screenshot shows the SQL Developer interface with the following SQL commands entered in the 'SQL Commands' window:

```

INSERT INTO Makes (Player_ID, Booking_ID) VALUES (1, 1);
INSERT INTO Makes (Player_ID, Booking_ID) VALUES (2, 2);
INSERT INTO Makes (Player_ID, Booking_ID) VALUES (3, 3);
INSERT INTO Makes (Player_ID, Booking_ID) VALUES (4, 4);
INSERT INTO Makes (Player_ID, Booking_ID) VALUES (5, 5);

SELECT * FROM Makes ;

```

The 'Results' window displays the following data:

PLAYER_ID	BOOKING_ID
1	1
2	2
3	3
4	4
5	5

5 rows returned in 0.00 seconds

Table Name: Can_Be_Made

INSERT INTO Can_Be_Made (Booking_ID, Turf_ID) VALUES (1, 1);

INSERT INTO Can_Be_Made (Booking_ID, Turf_ID) VALUES (2, 2);

INSERT INTO Can_Be_Made (Booking_ID, Turf_ID) VALUES (3, 3);

INSERT INTO Can_Be_Made (Booking_ID, Turf_ID) VALUES (4, 4);

INSERT INTO Can_Be_Made (Booking_ID, Turf_ID) VALUES (5, 5);

SELECT * FROM Can_Be_Made ;

The screenshot shows the SQL Developer interface with the following SQL commands entered in the 'SQL Commands' window:

```

INSERT INTO Can_Be_Made (Booking_ID, Turf_ID) VALUES (1, 1);
INSERT INTO Can_Be_Made (Booking_ID, Turf_ID) VALUES (2, 2);
INSERT INTO Can_Be_Made (Booking_ID, Turf_ID) VALUES (3, 3);
INSERT INTO Can_Be_Made (Booking_ID, Turf_ID) VALUES (4, 4);
INSERT INTO Can_Be_Made (Booking_ID, Turf_ID) VALUES (5, 5);

SELECT * FROM Can_Be_Made ;

```

The 'Results' window displays the following data:

BOOKING_ID	TURF_ID
1	1
2	2
3	3
4	4
5	5

5 rows returned in 0.00 seconds

Table Name: Can_Have

```
INSERT INTO Can_Have (Booking_ID, Payment_ID) VALUES (1, 1);
INSERT INTO Can_Have (Booking_ID, Payment_ID) VALUES (2, 2);
INSERT INTO Can_Have (Booking_ID, Payment_ID) VALUES (3, 21);
INSERT INTO Can_Have (Booking_ID, Payment_ID) VALUES (4, 22);
INSERT INTO Can_Have (Booking_ID, Payment_ID) VALUES (5, 23);

SELECT * FROM Can_Have ;
```

The screenshot shows the Oracle SQL Developer interface. The top pane displays the following SQL commands:

```
INSERT INTO Can_Have (Booking_ID, Payment_ID) VALUES (1, 1);
INSERT INTO Can_Have (Booking_ID, Payment_ID) VALUES (2, 2);
INSERT INTO Can_Have (Booking_ID, Payment_ID) VALUES (3, 21);
INSERT INTO Can_Have (Booking_ID, Payment_ID) VALUES (4, 22);
INSERT INTO Can_Have (Booking_ID, Payment_ID) VALUES (5, 23);

SELECT * FROM Can_Have ;
```

The bottom pane shows the results of the execution:

BOOKING_ID	PAYMENT_ID
1	1
2	2
3	21
4	22
5	23

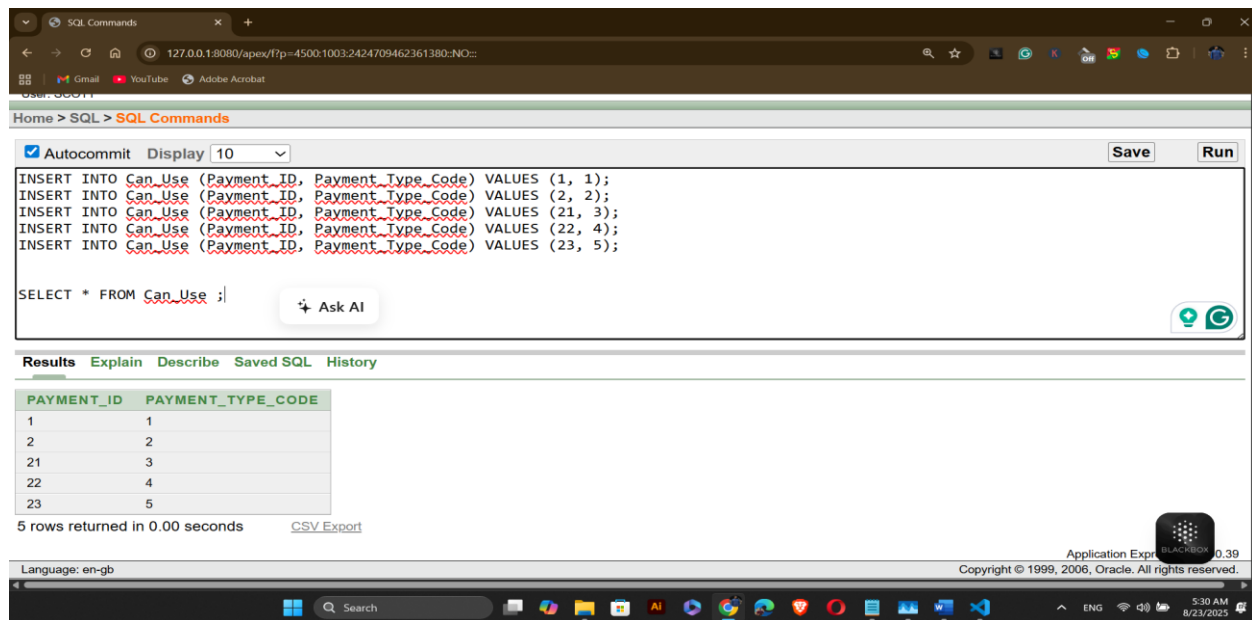
5 rows returned in 0.00 seconds

The interface also includes tabs for Results, Explain, Describe, Saved SQL, and History. The bottom status bar shows the language as en-gb and the copyright notice: Copyright © 1999, 2006, Oracle. All rights reserved.

Table Name: Can_Use

```
INSERT INTO Can_Use (Payment_ID, Payment_Type_Code) VALUES (1, 1);
INSERT INTO Can_Use (Payment_ID, Payment_Type_Code) VALUES (2, 2);
INSERT INTO Can_Use (Payment_ID, Payment_Type_Code) VALUES (21, 3);
INSERT INTO Can_Use (Payment_ID, Payment_Type_Code) VALUES (22, 4);
INSERT INTO Can_Use (Payment_ID, Payment_Type_Code) VALUES (23, 5);

SELECT * FROM Can_Use ;
```



10. BASIC PL/SQL:

2- variables type questions:

Question-1: Calculate Total Payment Amount for a Specific Player

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

DECLARE

 v_total_amount NUMBER(10, 2);

BEGIN

 SELECT SUM(p.Amount) INTO v_total_amount

 FROM Payment p

 JOIN Can_Have ch ON p.Payment_ID = ch.Payment_ID

 JOIN Makes m ON ch.Booking_ID = m.Booking_ID

 WHERE m.Player_ID = 1;

```

IF v_total_amount IS NULL THEN

    v_total_amount := 0;

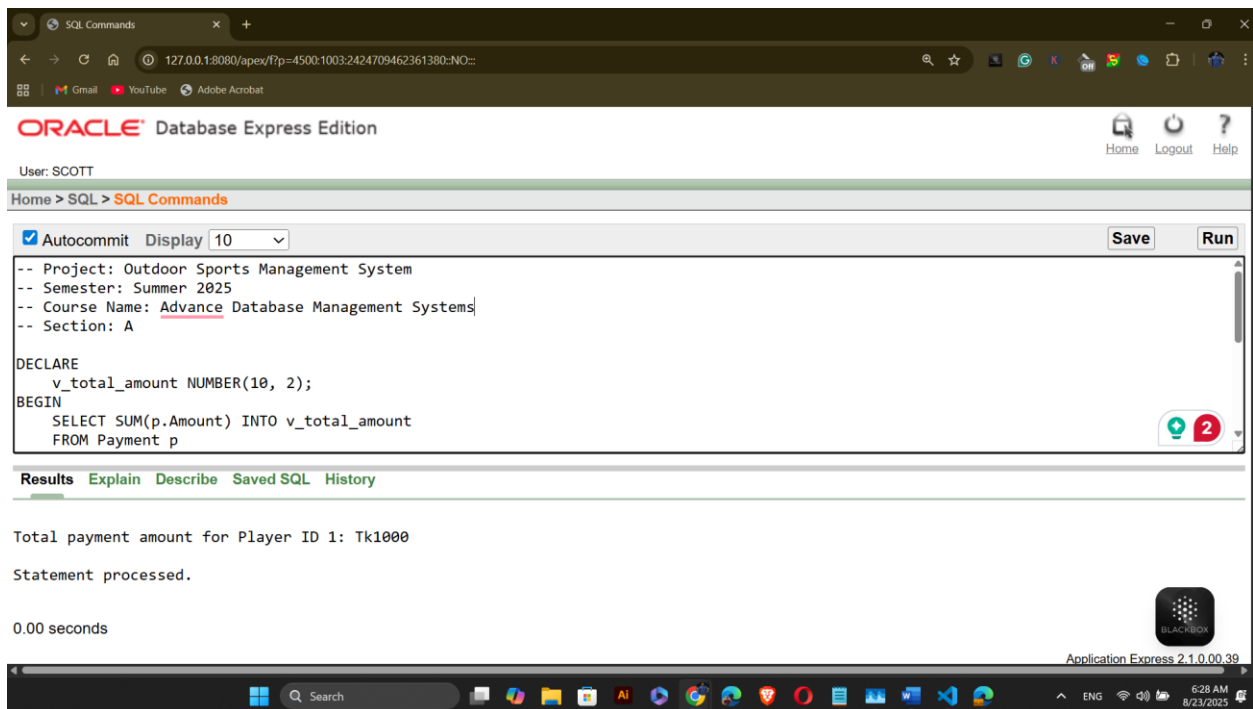
END IF;

DBMS_OUTPUT.PUT_LINE('Total payment amount for Player ID 1: Tk' || v_total_amount);

END;

/

```



Question-2: Count Number of Bookings for a Specific Turf

Answer:

```

-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A

```

DECLARE

```

    v_turf_name VARCHAR2(100);
    v_booking_count NUMBER;

```

BEGIN

SELECT t.Turf_Name, COUNT(cbm.Booking_ID)

INTO v_turf_name, v_booking_count

FROM Turf t

LEFT JOIN Can_Be_Made cbm ON t.Turf_ID = cbm.Turf_ID

WHERE t.Turf_ID = 1

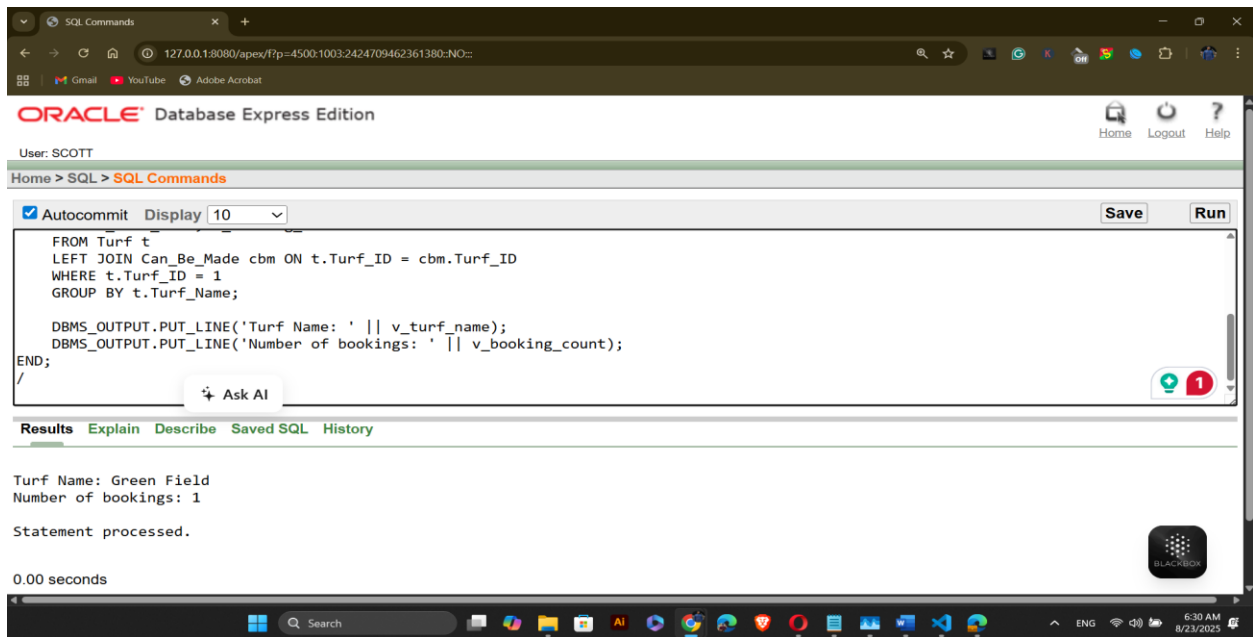
GROUP BY t.Turf_Name;

DBMS_OUTPUT.PUT_LINE('Turf Name: ' || v_turf_name);

DBMS_OUTPUT.PUT_LINE('Number of bookings: ' || v_booking_count);

END;

/



2- operators type questions:

Question-1: Calculate Booking Duration and Apply Late Fees

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

DECLARE

v_booking_id NUMBER := 1;

v_start_time TIMESTAMP;

v_end_time TIMESTAMP;

v_scheduled_duration NUMBER;

v_actual_duration NUMBER;

v_overtime_minutes NUMBER;

v_late_fee NUMBER := 0;

BEGIN

SELECT Start_Time, End_Time

INTO v_start_time, v_end_time

FROM Booking

WHERE Booking_ID = v_booking_id;

v_scheduled_duration := (EXTRACT(HOUR FROM (v_end_time - v_start_time)) * 60) +
EXTRACT(MINUTE FROM (v_end_time - v_start_time));

v_actual_duration := (EXTRACT(HOUR FROM (SYSTIMESTAMP - v_start_time)) * 60) +
EXTRACT(MINUTE FROM (SYSTIMESTAMP - v_start_time));

-- Calculate overtime and late fee

IF v_actual_duration > v_scheduled_duration THEN

v_overtime_minutes := v_actual_duration - v_scheduled_duration;

-- Tk120 per hour or part thereof

v_late_fee := CEIL(v_overtime_minutes / 60) * 100;

END IF;

-- Output results

DBMS_OUTPUT.PUT_LINE('Booking ID: ' || v_booking_id);

DBMS_OUTPUT.PUT_LINE('Scheduled Duration: ' || v_scheduled_duration || ' minutes');

DBMS_OUTPUT.PUT_LINE('Actual Duration: ' || v_actual_duration || ' minutes');

DBMS_OUTPUT.PUT_LINE('Overtime Minutes: ' || NVL(v_overtime_minutes, 0));

DBMS_OUTPUT.PUT_LINE('Late Fee: Tk' || v_late_fee);

END;

/

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the following code:

```
v_late_fee := CEIL(v_overtime_minutes / 60) * 100;
END IF;

-- Output results
DBMS_OUTPUT.PUT_LINE('Booking ID: ' || v_booking_id);
DBMS_OUTPUT.PUT_LINE('Scheduled Duration: ' || v_scheduled_duration || ' minutes');
DBMS_OUTPUT.PUT_LINE('Actual Duration: ' || v_actual_duration || ' minutes');
DBMS_OUTPUT.PUT_LINE('Overtime Minutes: ' || NVL(v_overtime_minutes, 0));
DBMS_OUTPUT.PUT_LINE('Late Fee: Tk' || v_late_fee);
END;
```

The Results tab shows the output of the query:

```
Booking ID: 1
Scheduled Duration: 120 minutes
Actual Duration: 879 minutes
Overtime Minutes: 759
Late Fee: Tk1300
```

The status bar at the bottom indicates "Statement processed." and the time is 6:41 AM on 8/23/2025.

Question-2: Calculate Event Revenue and Apply Seasonal Discount

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

DECLARE

v_event_id NUMBER := 1;

v_event_name VARCHAR2(100);

v_event_date DATE;

v_event_month NUMBER;

v_total_revenue NUMBER(10,2) := 0;

v_discount_percent NUMBER := 0;

v_discounted_revenue NUMBER(10,2);

BEGIN

SELECT Event_Name, Event_Date

INTO v_event_name, v_event_date

FROM Events

WHERE Event_ID = v_event_id;

v_event_month := EXTRACT(MONTH FROM v_event_date);

SELECT SUM(p.Amount)

INTO v_total_revenue

FROM Payment p

JOIN Can_Have ch ON p.Payment_ID = ch.Payment_ID

JOIN Can_Be_Made cbm ON ch.Booking_ID = cbm.Booking_ID

JOIN Participates part ON cbm.Booking_ID = part.Event_ID

WHERE part.Event_ID = v_event_id;

IF v_event_month = 9 THEN

 v_discount_percent := 15;

ELSIF v_event_month = 10 THEN

 v_discount_percent := 10;

ELSE

 v_discount_percent := 0;

END IF;

v_discounted_revenue := v_total_revenue - (v_total_revenue * v_discount_percent / 100);

IF v_total_revenue IS NULL THEN

 v_total_revenue := 0;

 v_discounted_revenue := 0;

END IF;

DBMS_OUTPUT.PUT_LINE('Event: ' || v_event_name);

DBMS_OUTPUT.PUT_LINE('Event Date: ' || TO_CHAR(v_event_date, 'DD-MON-YYYY'));

DBMS_OUTPUT.PUT_LINE('Event Month: ' || v_event_month);

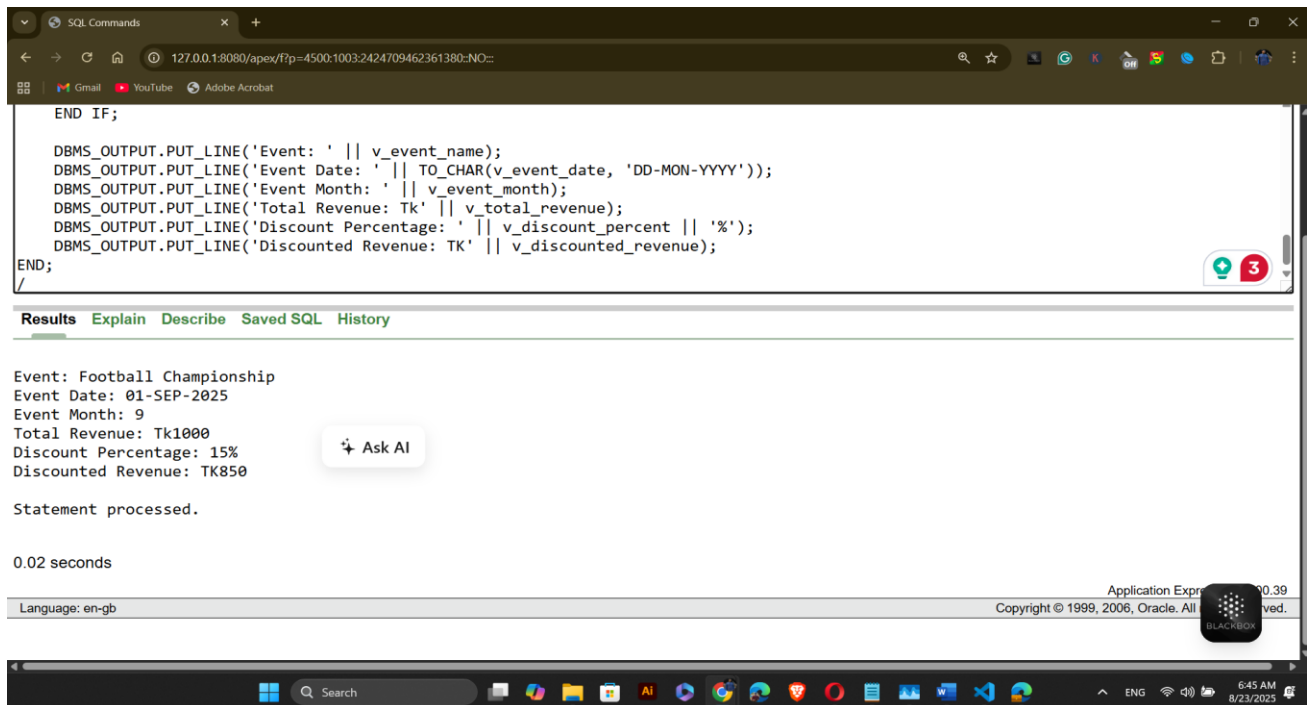
DBMS_OUTPUT.PUT_LINE('Total Revenue: Tk' || v_total_revenue);

DBMS_OUTPUT.PUT_LINE('Discount Percentage: ' || v_discount_percent || '%');

DBMS_OUTPUT.PUT_LINE('Discounted Revenue: TK' || v_discounted_revenue);

END;

/



2- single-row function type questions:

Question-1: Retrieve the player's full name (concatenated first and last names) in uppercase.

Answer:

- Project: Outdoor Sports Management System
- Semester: Summer 2025
- Course Name: Advance Database Management Systems
- Section: A

DECLARE

 v_player_name VARCHAR2(100);

BEGIN

 SELECT UPPER(First_Name || ' ' || Last_Name)

 INTO v_player_name

 FROM Players

 WHERE Player_ID = 1;

 DBMS_OUTPUT.PUT_LINE('Uppercase Player Name: ' || v_player_name);

END;

/

The screenshot shows the Oracle Database Express Edition interface. The browser address bar displays the URL 127.0.0.1:8080/apex/f?p=4500:1003:2395720576162872::NO-. The page title is "ORACLE Database Express Edition". The user is logged in as "SCOTT". The breadcrumb navigation shows "Home > SQL > SQL Commands". The "Autocommit" checkbox is checked, and the "Display" dropdown is set to "10". The "Save" and "Run" buttons are visible. The SQL command window contains the following text:

```
-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A

DECLARE
    v_player_name VARCHAR2(100);
BEGIN
    SELECT UPPER(First_Name || ' ' || Last_Name)
    INTO v_player_name
    FROM Players
    WHERE Player_ID = 1;

    DBMS_OUTPUT.PUT_LINE('Uppercase Player Name: ' || v_player_name);
END;
/
```

Below the command window, the "Results" tab is selected, showing the output: "Uppercase Player Name: SHADMAN ALAM". The status "Statement processed." and "0.00 seconds" are displayed. The bottom of the window shows the "Application Express 2.1.0.00.39" logo and the system tray with the date and time "11:12 AM 8/23/2025".

Question-2: Retrieve the player's age (calculated from their date of birth) and round it to the nearest whole number.

Answer:

-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A

DECLARE

 v_player_age NUMBER;

BEGIN

 SELECT ROUND(MONTHS_BETWEEN(SYSDATE, DOB) / 12)

 INTO v_player_age

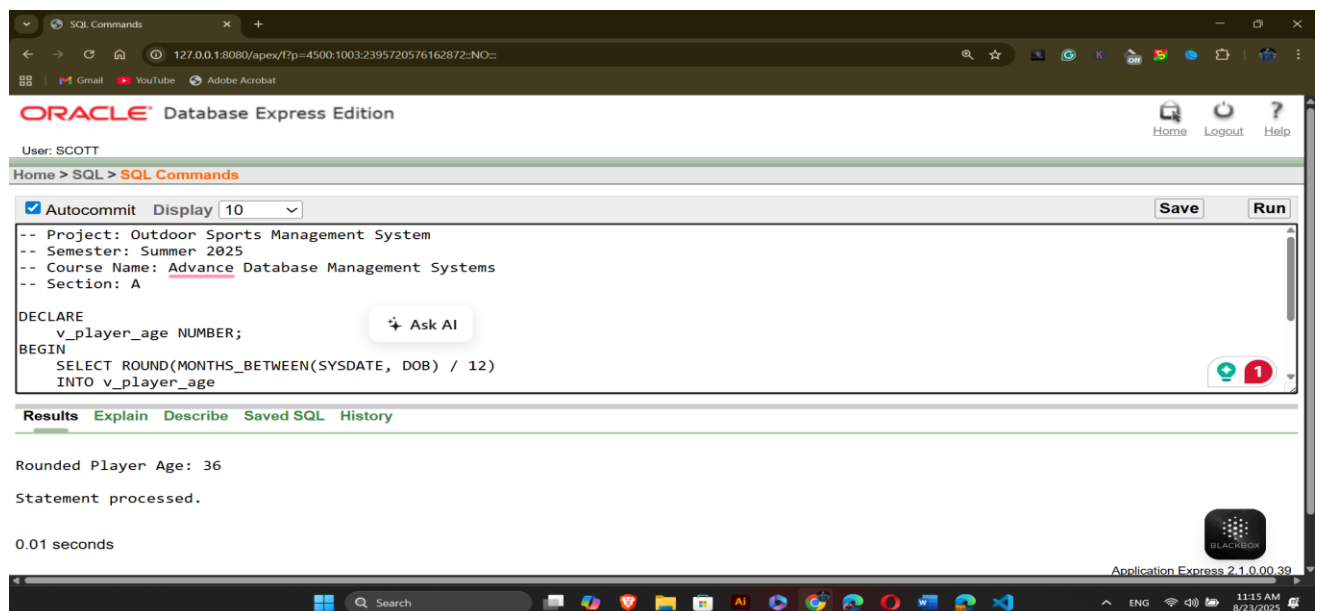
 FROM Players

 WHERE Player_ID = 1;

 DBMS_OUTPUT.PUT_LINE('Rounded Player Age: ' || v_player_age);

END;

/



2- Group function type questions:

Question-1: Find the average number of players in all teams.

Answer:

-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A

DECLARE

 v_avg_players NUMBER;

BEGIN

 SELECT AVG(player_count)

 INTO v_avg_players

 FROM (SELECT Team_ID, COUNT(*) AS player_count

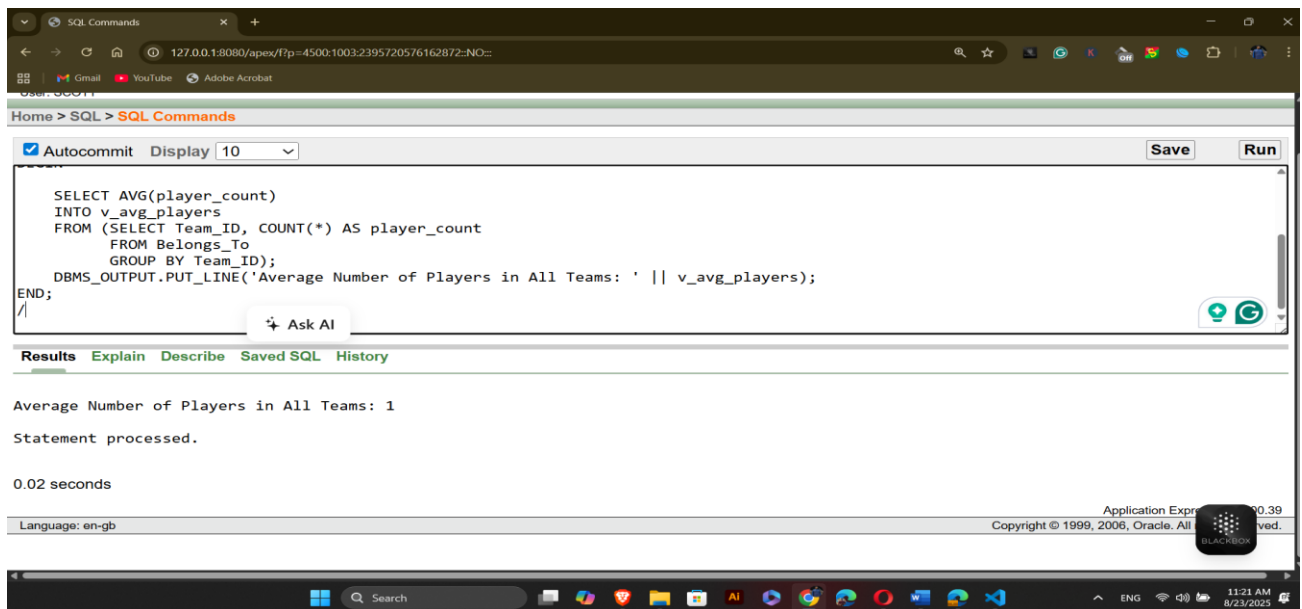
 FROM Belongs_To

 GROUP BY Team_ID);

 DBMS_OUTPUT.PUT_LINE('Average Number of Players in All Teams: ' || v_avg_players);

END;

/



Question-2: Find the total number of bookings that have a status of 'Confirmed'.

Answer:

- Project: Outdoor Sports Management System
- Semester: Summer 2025
- Course Name: Advance Database Management Systems
- Section: A

DECLARE

 v_total_confirmed NUMBER;

BEGIN

 SELECT COUNT(*)

 INTO v_total_confirmed

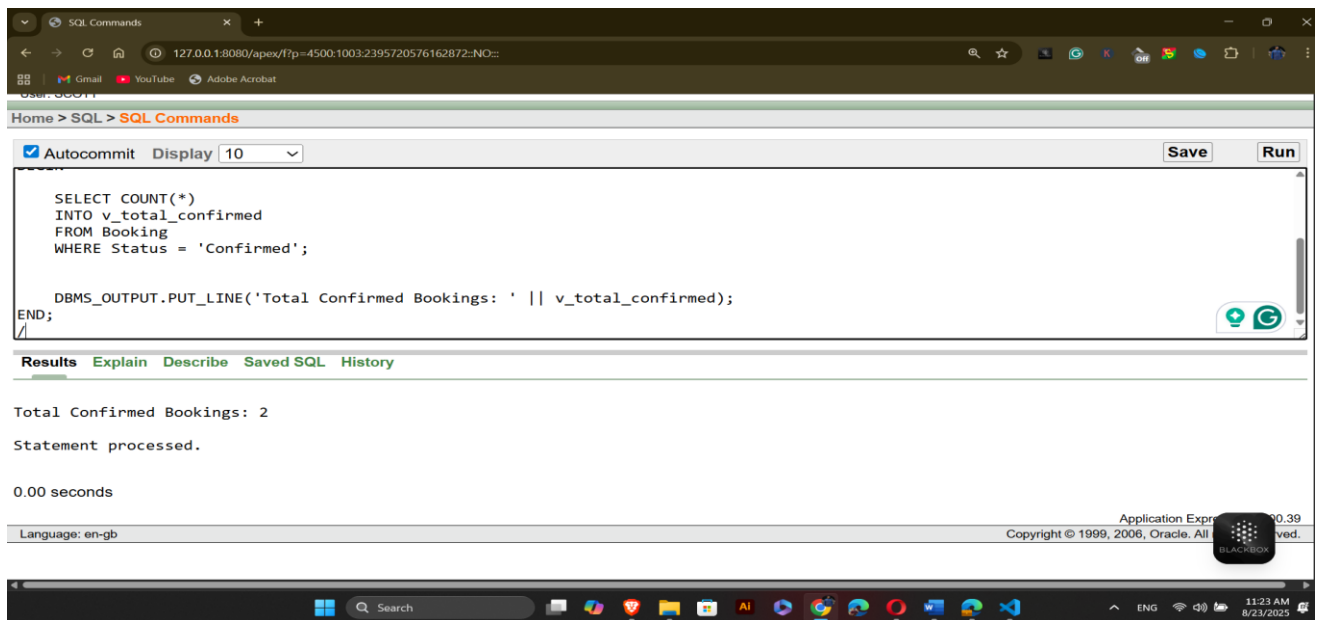
 FROM Booking

 WHERE Status = 'Confirmed';

 DBMS_OUTPUT.PUT_LINE('Total Confirmed Bookings: ' || v_total_confirmed);

END;

/



2- Loop type questions:

Question-1: Write a PL/SQL block that uses a FOR loop to display the names of all players in the 'Warriors' team.

Answer:

```
-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A
```

DECLARE

CURSOR c_players IS

SELECT p.First_Name || ' ' || p.Last_Name AS player_name

FROM Players p

JOIN Belongs_To bt ON p.Player_ID = bt.Player_ID

JOIN Team t ON bt.Team_ID = t.Team_ID

WHERE t.Team_Name = 'Warriors';

BEGIN

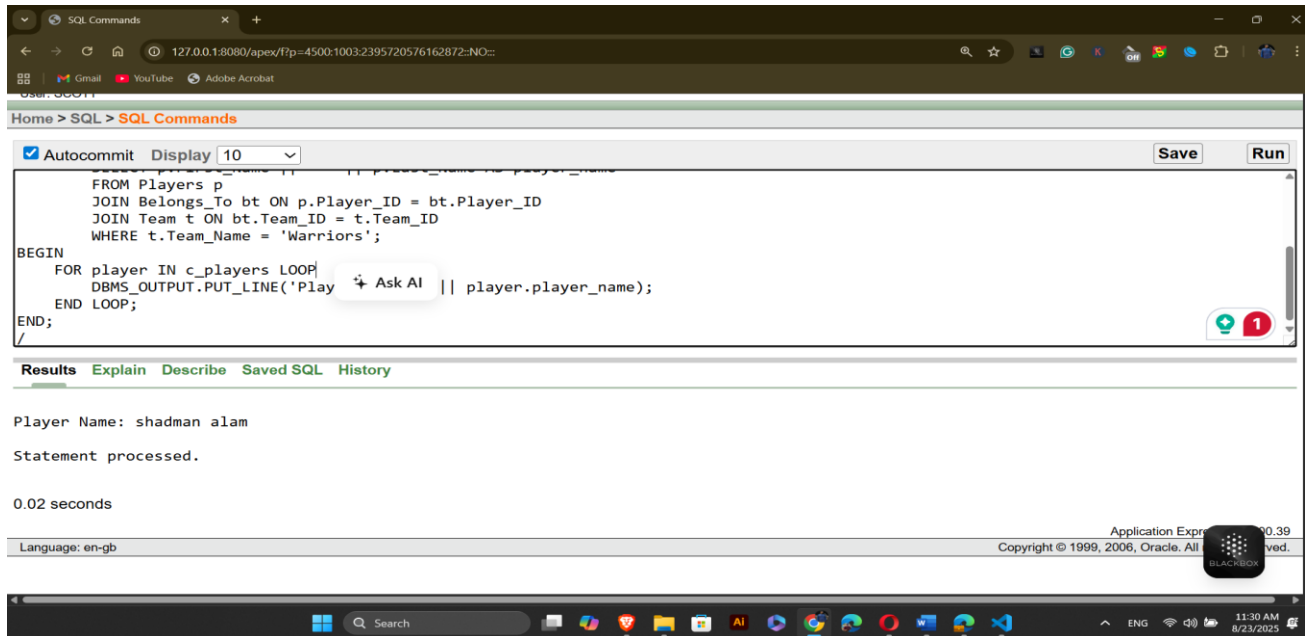
FOR player IN c_players LOOP

DBMS_OUTPUT.PUT_LINE('Player Name: ' || player.player_name);

END LOOP;

END;

/



Question-2: Write a PL/SQL block that uses a WHILE loop to display all booking IDs and their corresponding statuses for bookings made on '2025-08-01'.

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

DECLARE

v_booking_id NUMBER;

v_booking_status VARCHAR2(50);


```

CURSOR c_bookings IS

    SELECT Booking_ID, Status

    FROM Booking

    WHERE Booking_Date = TO_DATE('2025-08-01', 'YYYY-MM-DD');

BEGIN

    OPEN c_bookings;

    FETCH c_bookings INTO v_booking_id, v_booking_status;

    WHILE c_bookings%FOUND LOOP

        DBMS_OUTPUT.PUT_LINE('Booking ID: ' || v_booking_id || ', Status: ' ||
v_booking_status);

        FETCH c_bookings INTO v_booking_id, v_booking_status;

    END LOOP;

    CLOSE c_bookings;

END;

/

```

The screenshot shows the Oracle SQL Developer interface. The 'SQL Commands' window contains the following code:

```

FETCH c_bookings INTO v_booking_id, v_booking_status;

WHILE c_bookings%FOUND LOOP
    DBMS_OUTPUT.PUT_LINE('Booking ID: ' || v_booking_id || ', Status: ' || v_booking_status);
    FETCH c_bookings INTO v_booking_id, v_booking_status;
END LOOP;
CLOSE c_bookings;

END;
/

```

The 'Results' window shows the output of the execution:

```

Booking ID: 1, Status: Confirmed
Statement processed.
0.00 seconds

```

The status bar at the bottom indicates 'Language: en-gb' and 'Copyright © 1999, 2006, Oracle. All rights reserved.'.

2- Conditional type questions:

Question-1: Check if the player is eligible for a specific event based on their skill level.

Answer:

-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A

DECLARE

 v_skill_level VARCHAR2(50); -- Variable to store the player's skill level

BEGIN

 -- Retrieve the player's skill level

 SELECT Skill_Level

 INTO v_skill_level

 FROM Profile

 JOIN Has_Profile hp ON Profile.Profile_ID = hp.Profile_ID

 WHERE hp.Player_ID = 1;

 -- Check if the player's skill level qualifies for the event

 IF v_skill_level = 'Advanced' THEN

 DBMS_OUTPUT.PUT_LINE('Player is eligible for the event.');

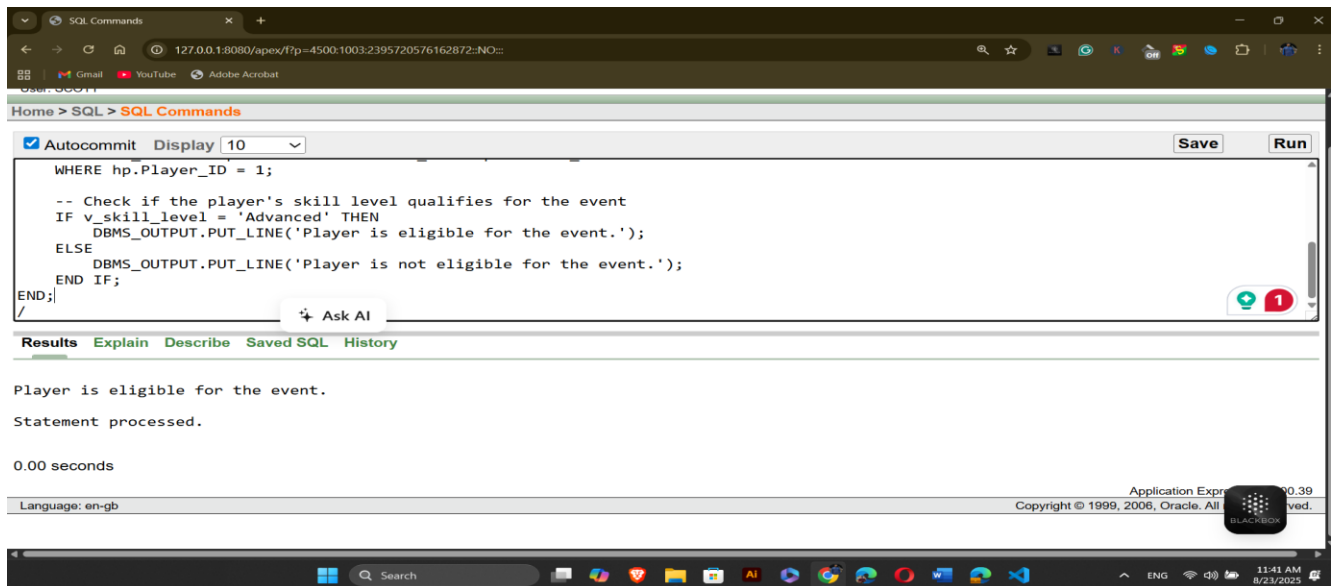
 ELSE

 DBMS_OUTPUT.PUT_LINE('Player is not eligible for the event.');

 END IF;

END;

/



Question-2: Check if a booking is confirmed for a specific date. If the status is 'Confirmed', display the booking details. Otherwise, display a message saying the booking is not confirmed.

Answer:

```
-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A
```

DECLARE

```
v_booking_status VARCHAR2(50);
```

```
v_booking_date DATE := TO_DATE('2025-08-01', 'YYYY-MM-DD');
```

BEGIN

```
SELECT Status
```

```
INTO v_booking_status
```

```
FROM Booking
```

```
WHERE Booking_Date = v_booking_date;
```

```

IF v_booking_status = 'Confirmed' THEN

    DBMS_OUTPUT.PUT_LINE('Booking for ' || TO_CHAR(v_booking_date, 'YYYY-MM-DD') || ' is confirmed.');
```

ELSE

```

    DBMS_OUTPUT.PUT_LINE('Booking for ' || TO_CHAR(v_booking_date, 'YYYY-MM-DD') || ' is not confirmed.');
```

END IF;

```

END;
```

/

The screenshot shows the Oracle SQL Developer 'SQL Commands' window. The query is as follows:

```

FROM Booking
WHERE Booking_Date = v_booking_date;

IF v_booking_status = 'Confirmed' THEN
    DBMS_OUTPUT.PUT_LINE('Booking for ' || TO_CHAR(v_booking_date, 'YYYY-MM-DD') || ' is confirmed.');
```

```

ELSE
    DBMS_OUTPUT.PUT_LINE('Booking for ' || TO_CHAR(v_booking_date, 'YYYY-MM-DD') || ' is not confirmed.');
```

```

END IF;

END;
```

The execution results show:

```

Booking for 2025-08-01 is confirmed.
Statement processed.
0.00 seconds
```

The interface also shows tabs for Results, Explain, Describe, Saved SQL, and History. The bottom status bar indicates the application is running on Oracle 12.2.0.1.0.39.

2- subquery questions:

Question-1: Retrieve the Player ID of the player who has participated in the most number of events.

Answer:

- Project: Outdoor Sports Management System
- Semester: Summer 2025
- Course Name: Advance Database Management Systems
- Section: A

```
DECLARE
```

```
    v_top_player_id NUMBER;
```

```
BEGIN
```

```
    SELECT Player_ID
```

```
    INTO v_top_player_id
```

```
    FROM (
```

```
        SELECT Player_ID
```

```
        FROM Belongs_To
```

```
        GROUP BY Player_ID
```

```
        HAVING COUNT(*) = (
```

```
            SELECT MAX(event_count)
```

```
            FROM (
```

```
                SELECT Player_ID, COUNT(*) AS event_count
```

```
                FROM Belongs_To
```

```
                GROUP BY Player_ID
```

```
            )
```

```
        )
```

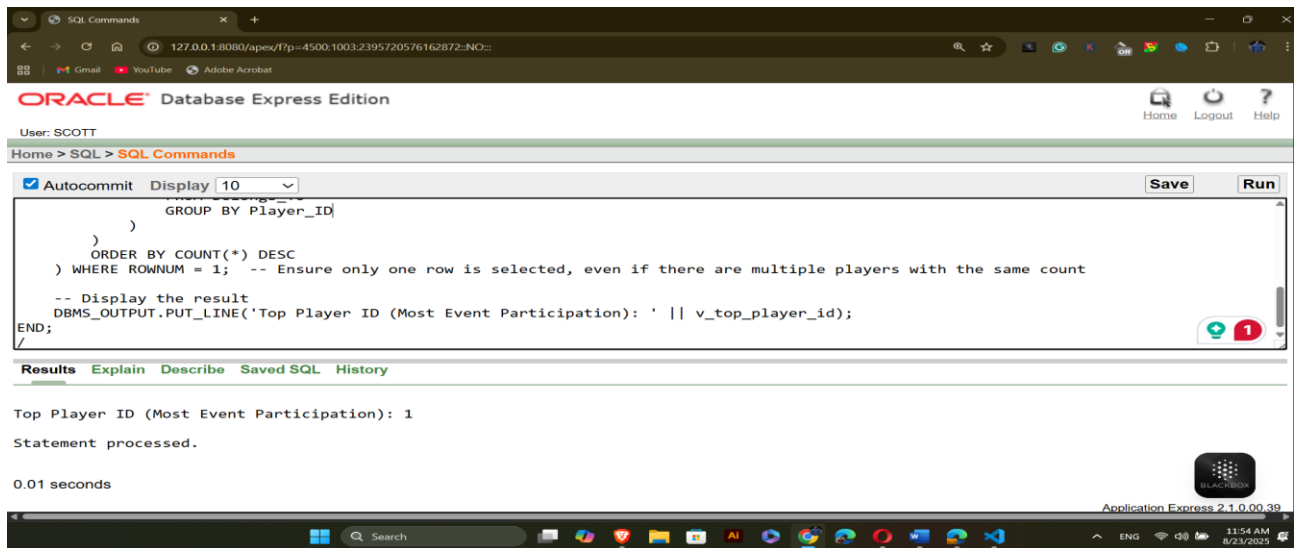
```
        ORDER BY COUNT(*) DESC
```

```
    ) WHERE ROWNUM = 1; --only one row is selected
```

```
    DBMS_OUTPUT.PUT_LINE('Top Player ID (Most Event Participation): ' || v_top_player_id);
```

```
END;
```

```
/
```



Question-2: Retrieve the Booking ID of the booking with the highest payment amount.

Answer:

- Project: Outdoor Sports Management System
- Semester: Summer 2025
- Course Name: Advance Database Management Systems
- Section: A

DECLARE

 v_top_booking_id NUMBER;

BEGIN

 SELECT Booking_ID

 INTO v_top_booking_id

 FROM (

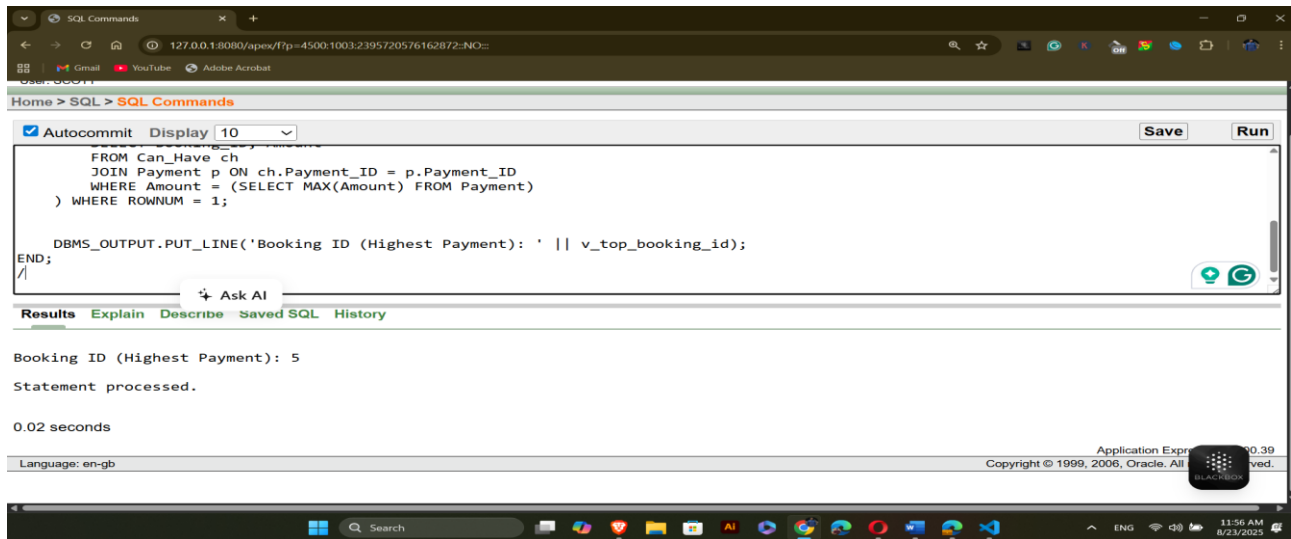
 SELECT Booking_ID, Amount

 FROM Can_Have ch

 JOIN Payment p ON ch.Payment_ID = p.Payment_ID

```
WHERE Amount = (SELECT MAX(Amount) FROM Payment)
) WHERE ROWNUM = 1;
```

```
DBMS_OUTPUT.PUT_LINE('Booking ID (Highest Payment): ' || v_top_booking_id);
END;
/
```



2- Joining questions:

Question-1: Join the Players and Profile tables to display the player's name and their skill level.

Answer:

- Project: Outdoor Sports Management System
- Semester: Summer 2025
- Course Name: Advance Database Management Systems
- Section: A

BEGIN

FOR r IN (

SELECT p.First_Name || ' ' || p.Last_Name AS player_name, pr.Skill_Level

FROM Players p

JOIN Has_Profile hp ON p.Player_ID = hp.Player_ID

JOIN Profile pr ON hp.Profile_ID = pr.Profile_ID

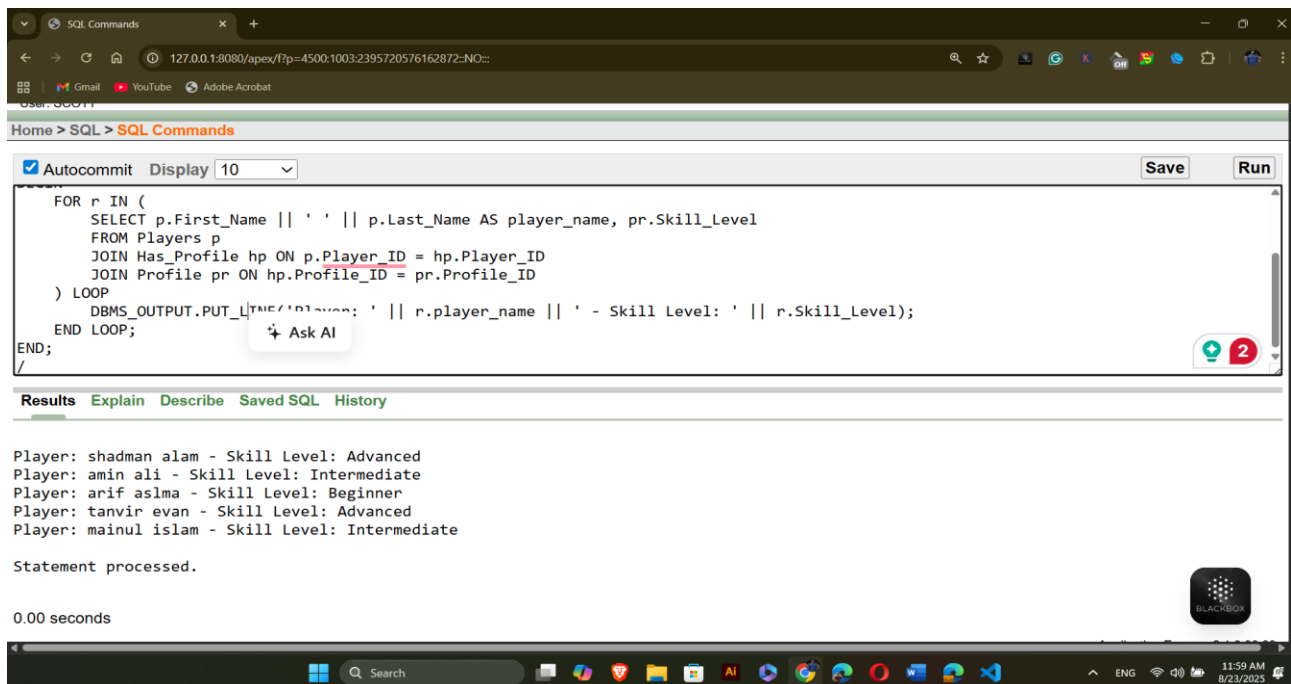
) LOOP

DBMS_OUTPUT.PUT_LINE('Player: ' || r.player_name || ' - Skill Level: ' || r.Skill_Level);

END LOOP;

END;

/



```
FOR r IN (
  SELECT p.First_Name || ' ' || p.Last_Name AS player_name, pr.Skill_Level
  FROM Players p
  JOIN Has_Profile hp ON p.Player_ID = hp.Player_ID
  JOIN Profile pr ON hp.Profile_ID = pr.Profile_ID
) LOOP
  DBMS_OUTPUT.PUT_LINE('Player: ' || r.player_name || ' - Skill Level: ' || r.Skill_Level);
END LOOP;
END;
```

Results Explain Describe Saved SQL History

Player: shadman alam - Skill Level: Advanced
Player: amin ali - Skill Level: Intermediate
Player: arif aslma - Skill Level: Beginner
Player: tanvir evan - Skill Level: Advanced
Player: mainul islam - Skill Level: Intermediate

Statement processed.

0.00 seconds

Question-2: Join the Booking and Turf tables to display the booking ID and turf location.

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

BEGIN

FOR r IN (

SELECT b.Booking_ID, t.Turf_location

FROM Booking b

JOIN Can_Be_Made cbm ON b.Booking_ID = cbm.Booking_ID

JOIN Turf t ON cbm.Turf_ID = t.Turf_ID

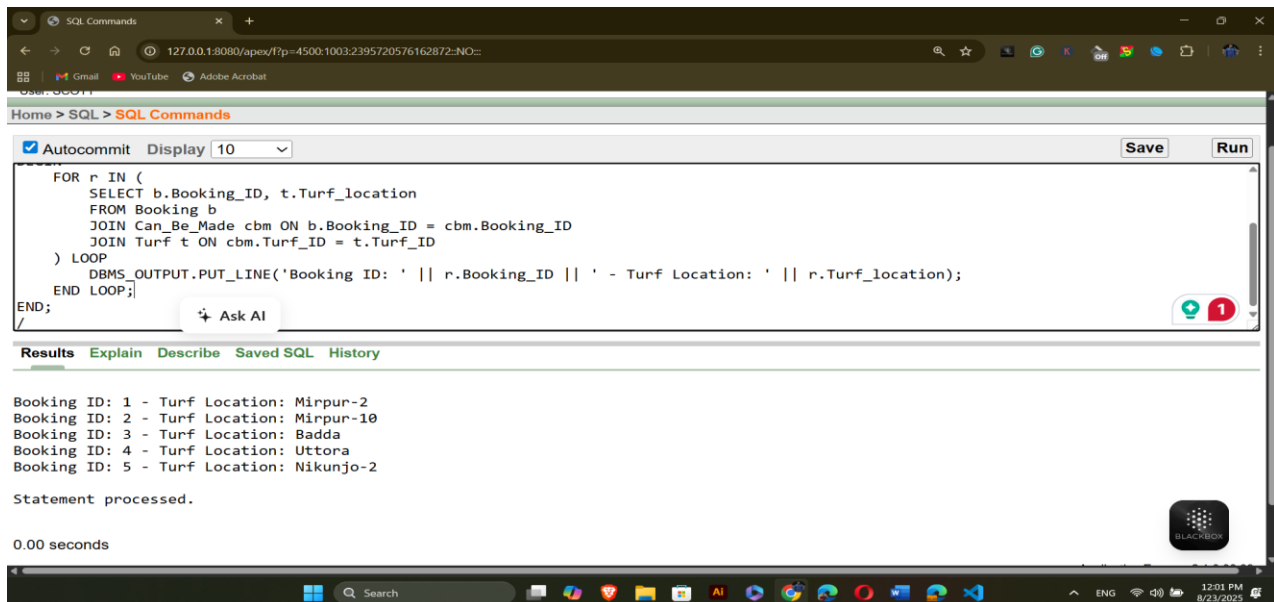
) LOOP

DBMS_OUTPUT.PUT_LINE('Booking ID: ' || r.Booking_ID || ' - Turf Location: ' ||
r.Turf_location);

END LOOP;

END;

/



```
SQL Commands
127.0.0.1:8080/apex/f?p=4500:1003:2395720576162872::NO::
Gmail YouTube Adobe Acrobat
Home > SQL > SQL Commands
Autocommit Display 10 Save Run
FOR r IN (
  SELECT b.Booking_ID, t.Turf_location
  FROM Booking b
  JOIN Can_Be_Made cbm ON b.Booking_ID = cbm.Booking_ID
  JOIN Turf t ON cbm.Turf_ID = t.Turf_ID
) LOOP
  DBMS_OUTPUT.PUT_LINE('Booking ID: ' || r.Booking_ID || ' - Turf Location: ' || r.Turf_location);
END LOOP;
END;
Ask AI
Results Explain Describe Saved SQL History
Booking ID: 1 - Turf Location: Mirpur-2
Booking ID: 2 - Turf Location: Mirpur-10
Booking ID: 3 - Turf Location: Badda
Booking ID: 4 - Turf Location: Uttora
Booking ID: 5 - Turf Location: Nikunjo-2
Statement processed.
0.00 seconds
BLACKBOX
12:01 PM 8/23/2025
```

11.Advance PL/SQL:

1. Stored function

Question-1: Create a function to retrieve the skill level of a player based on their Player_ID.

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

```
CREATE OR REPLACE FUNCTION get_skill_level (p_player_id IN NUMBER)
```

```
RETURN VARCHAR2 IS
```

```
    v_skill_level VARCHAR2(50);
```

```
BEGIN
```

```
    SELECT pr.Skill_Level
```

```
    INTO v_skill_level
```

```
    FROM Profile pr
```

```
    JOIN Has_Profile hp ON pr.Profile_ID = hp.Profile_ID
```

```
    WHERE hp.Player_ID = p_player_id;
```

```
    RETURN v_skill_level;
```

```
END get_skill_level;
```

```
/
```

To see the answer

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

DECLARE

v_player_id NUMBER := 1;

v_skill_level VARCHAR2(50);

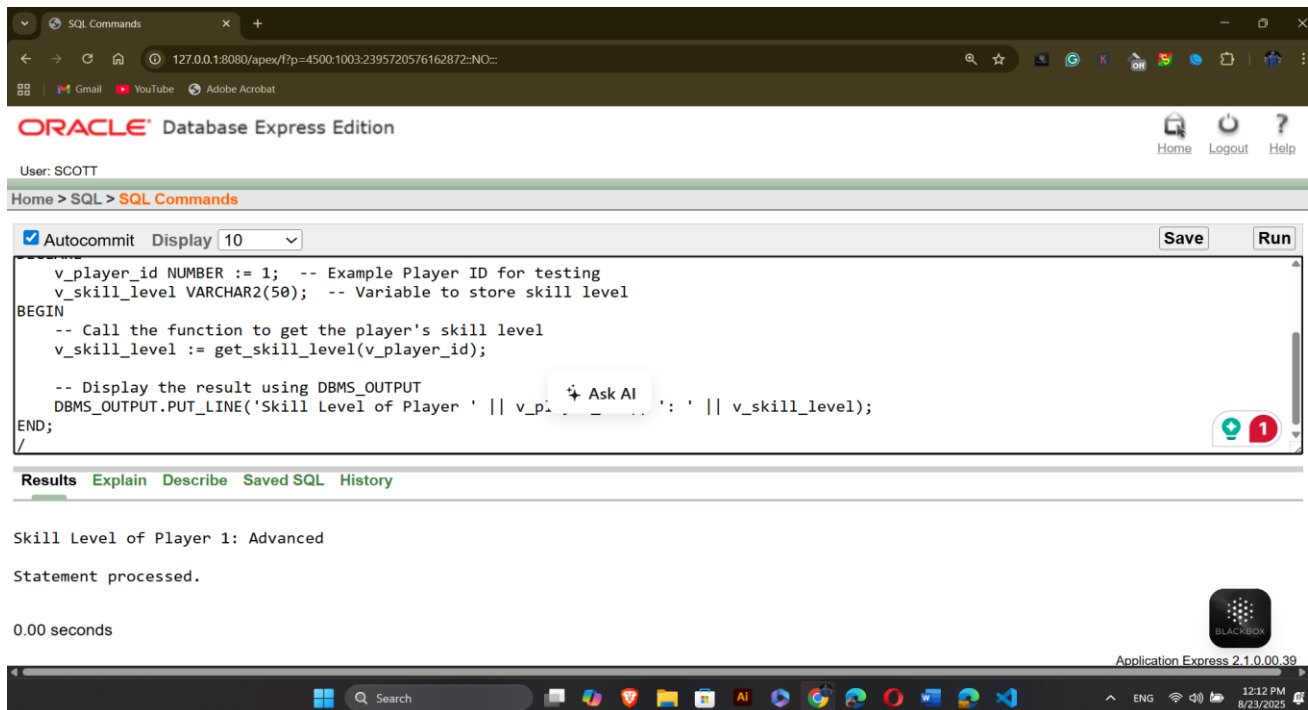
BEGIN

v_skill_level := get_skill_level(v_player_id);

DBMS_OUTPUT.PUT_LINE('Skill Level of Player ' || v_player_id || ': ' || v_skill_level);

END;

/



Question-2: Create a function to retrieve the turf location based on the Booking_ID.

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

CREATE OR REPLACE FUNCTION get_turf_location (p_booking_id IN NUMBER)

```

RETURN VARCHAR2 IS

    v_turf_location VARCHAR2(100);

BEGIN

    SELECT t.Turf_location
    INTO v_turf_location
    FROM Turf t
    JOIN Can_Be_Made cbm ON t.Turf_ID = cbm.Turf_ID
    WHERE cbm.Booking_ID = p_booking_id;

    RETURN v_turf_location;

END get_turf_location;

/

```

To see the answer:

```

-- to see the function
-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A

```

```

DECLARE

    v_booking_id NUMBER := 2;

    v_turf_location VARCHAR2(100);

BEGIN

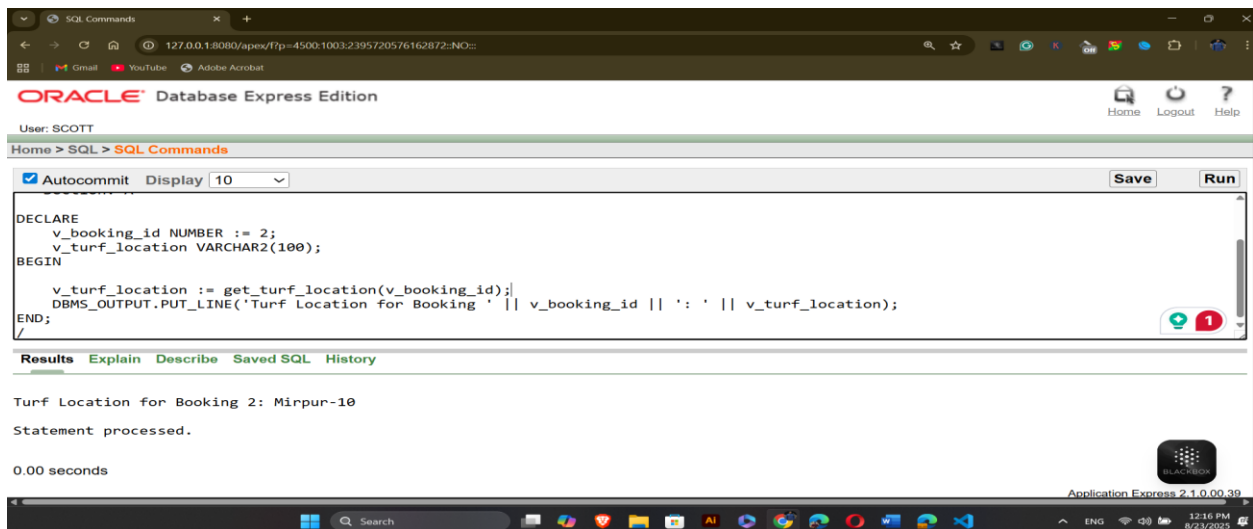
    v_turf_location := get_turf_location(v_booking_id);

    DBMS_OUTPUT.PUT_LINE('Turf Location for Booking ' || v_booking_id || ': ' ||
v_turf_location);

END;

/

```



2. Stored procedure

Question-1: Create a stored procedure to update the skill level of a player based on their Player_ID

Answer:

- to see the function
- Project: Outdoor Sports Management System
- Semester: Summer 2025
- Course Name: Advance Database Management Systems
- Section: A

CREATE OR REPLACE PROCEDURE update_skill_level (p_player_id IN NUMBER,
p_skill_level IN VARCHAR2)

IS

BEGIN

UPDATE Profile

SET Skill_Level = p_skill_level

```
WHERE Profile_ID = (SELECT Profile_ID FROM Has_Profile WHERE Player_ID =  
p_player_id);
```

```
COMMIT;
```

```
END update_skill_level;
```

```
/
```

To test the procedure:

```
-- to see the function
```

```
-- Project: Outdoor Sports Management System
```

```
-- Semester: Summer 2025
```

```
-- Course Name: Advance Database Management Systems
```

```
-- Section: A
```

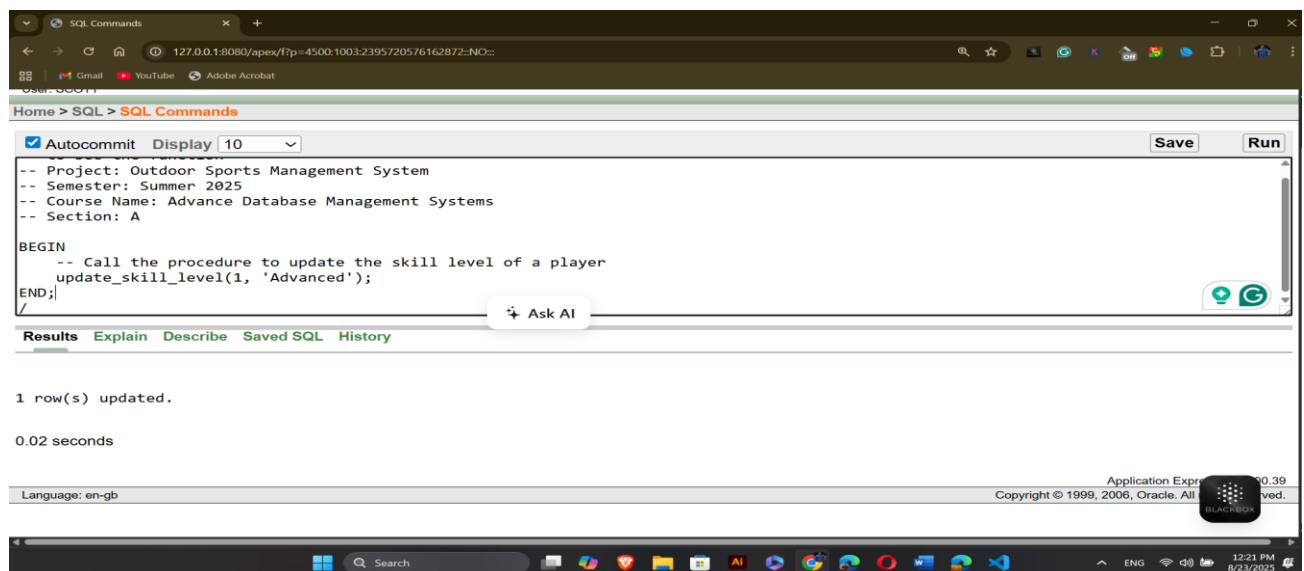
```
BEGIN
```

```
-- Call the procedure to update the skill level of a player
```

```
update_skill_level(1, 'Advanced');
```

```
END;
```

```
/
```



Question-2: Create a stored procedure to update the booking status of a specific booking based on the Booking_ID.

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

```
CREATE OR REPLACE PROCEDURE update_booking_status (p_booking_id IN NUMBER,  
p_status IN VARCHAR2)
```

```
IS
```

```
BEGIN
```

```
    UPDATE Booking
```

```
    SET Status = p_status
```

```
    WHERE Booking_ID = p_booking_id;
```

```
    COMMIT;
```

```
END update_booking_status;
```

```
/
```

To test the procedure:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

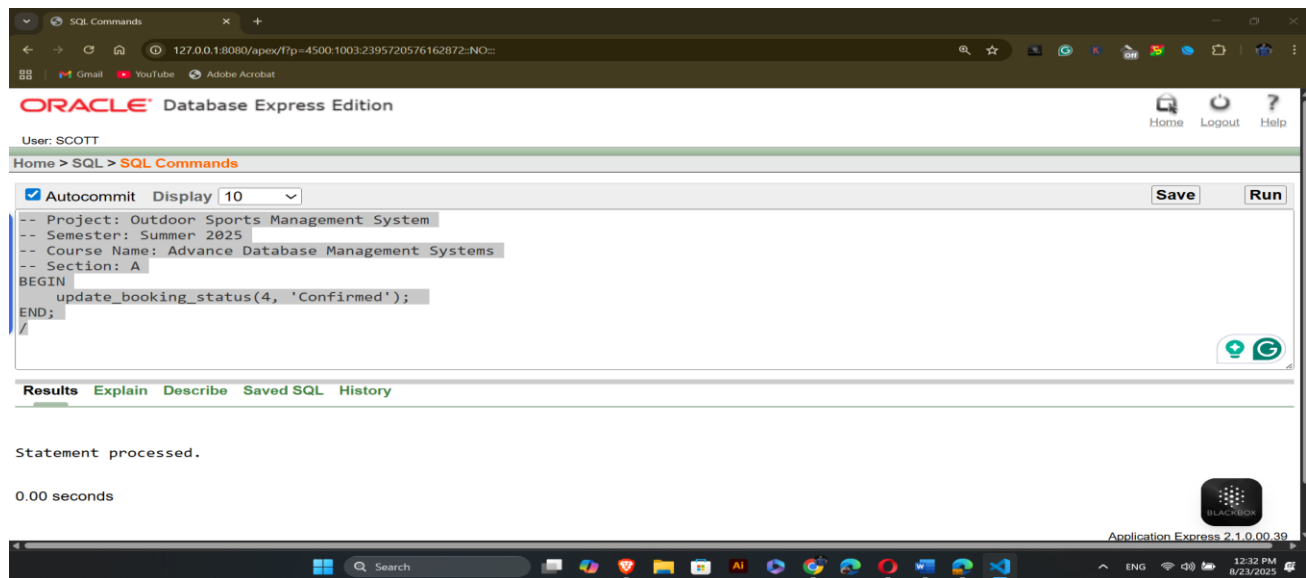
-- Section: A

```
BEGIN
```

```
    update_booking_status(4, 'Confirmed');
```

```
END;
```

```
/
```



3. Table record

Question-1: Create a table-based record type to store player data and loop through it.

Answer:

```
-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A
```

DECLARE

```
TYPE player_record_type IS TABLE OF Players%ROWTYPE;
```

```
v_players player_record_type;
```

BEGIN

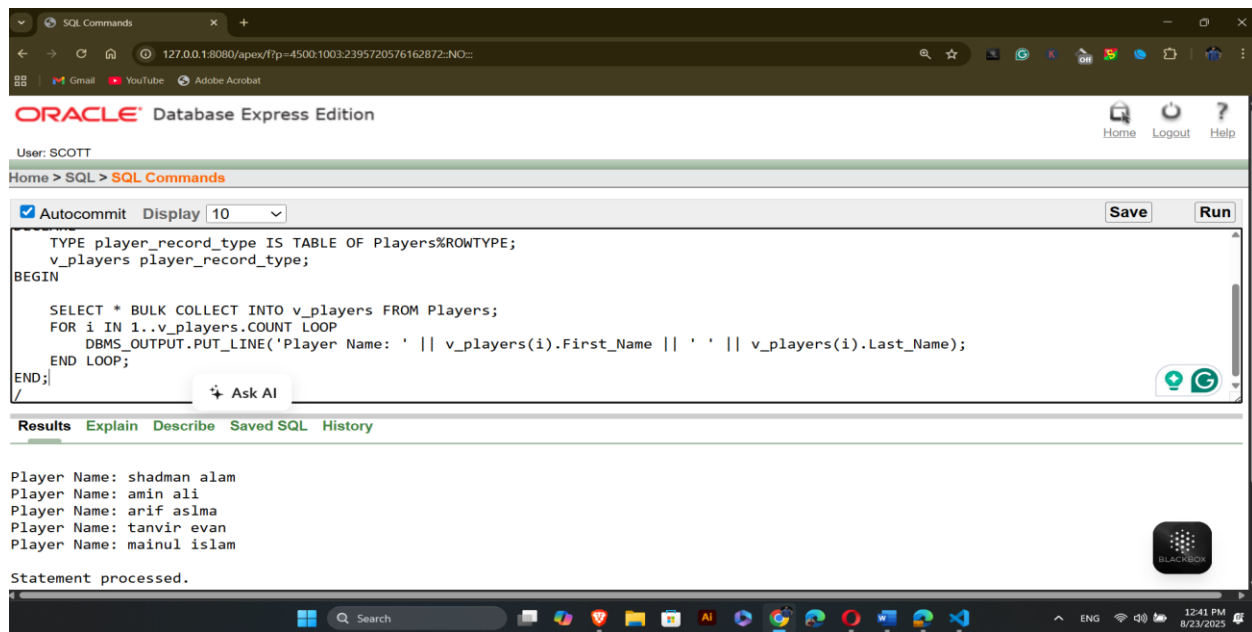
```
SELECT * BULK COLLECT INTO v_players FROM Players;
```

```
FOR i IN 1..v_players.COUNT LOOP
```

```
    DBMS_OUTPUT.PUT_LINE('Player Name: ' || v_players(i).First_Name || ' ' ||
v_players(i).Last_Name);
```

```
END LOOP;
```

```
END;
```

Question-2: Create a table-based record type to store booking data and loop through it.

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

DECLARE

 TYPE booking_record_type IS TABLE OF Booking%ROWTYPE;

 v_bookings booking_record_type;

BEGIN

 SELECT * BULK COLLECT INTO v_bookings FROM Booking;

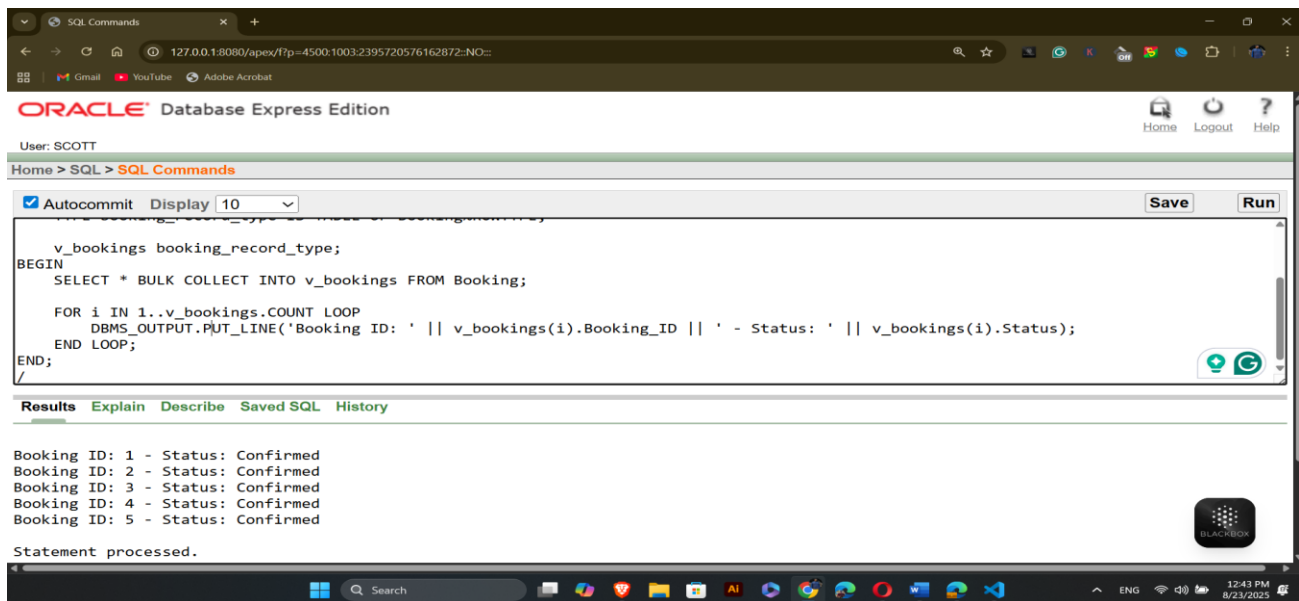
 FOR i IN 1..v_bookings.COUNT LOOP

 DBMS_OUTPUT.PUT_LINE('Booking ID: ' || v_bookings(i).Booking_ID || ' - Status: ' || v_bookings(i).Status);

 END LOOP;

END;

/



4. Explicit cursor

Question-1: Create an explicit cursor to fetch players with their booking details.

Answer:

```

-- Project: Outdoor Sports Management System
-- Semester: Summer 2025
-- Course Name: Advance Database Management Systems
-- Section: A

```

DECLARE

CURSOR c_player_bookings IS

SELECT p.Player_ID, b.Booking_ID, b.Status

FROM Players p

JOIN Belongs_To bt ON p.Player_ID = bt.Player_ID

JOIN Can_Make cm ON bt.Team_ID = cm.Team_ID

JOIN Booking b ON cm.Booking_ID = b.Booking_ID;

v_player_booking c_player_bookings%ROWTYPE;

BEGIN

```

OPEN c_player_bookings;

LOOP

    FETCH c_player_bookings INTO v_player_booking;

    EXIT WHEN c_player_bookings%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE('Player ID: ' || v_player_booking.Player_ID || ', Booking ID: '
|| v_player_booking.Booking_ID || ', Status: ' || v_player_booking.Status);

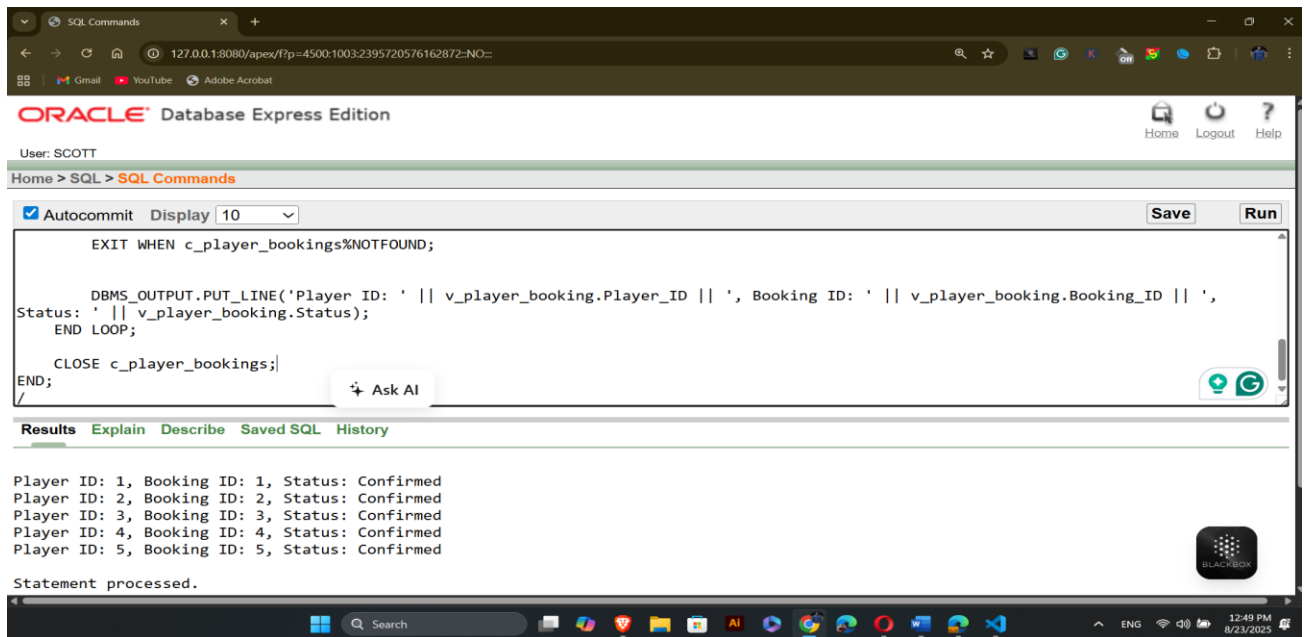
END LOOP;

CLOSE c_player_bookings;

END;

/

```



Question-2: Create an explicit cursor to fetch the booking details (Booking ID, Start Time, and End Time) for a specific team.

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

DECLARE

CURSOR c_team_bookings IS

SELECT b.Booking_ID, b.Start_Time, b.End_Time

FROM Booking b

JOIN Can_Make cm ON b.Booking_ID = cm.Booking_ID

WHERE cm.Team_ID = 1;

v_team_booking c_team_bookings%ROWTYPE;

BEGIN

OPEN c_team_bookings;

LOOP

FETCH c_team_bookings INTO v_team_booking;

EXIT WHEN c_team_bookings%NOTFOUND;

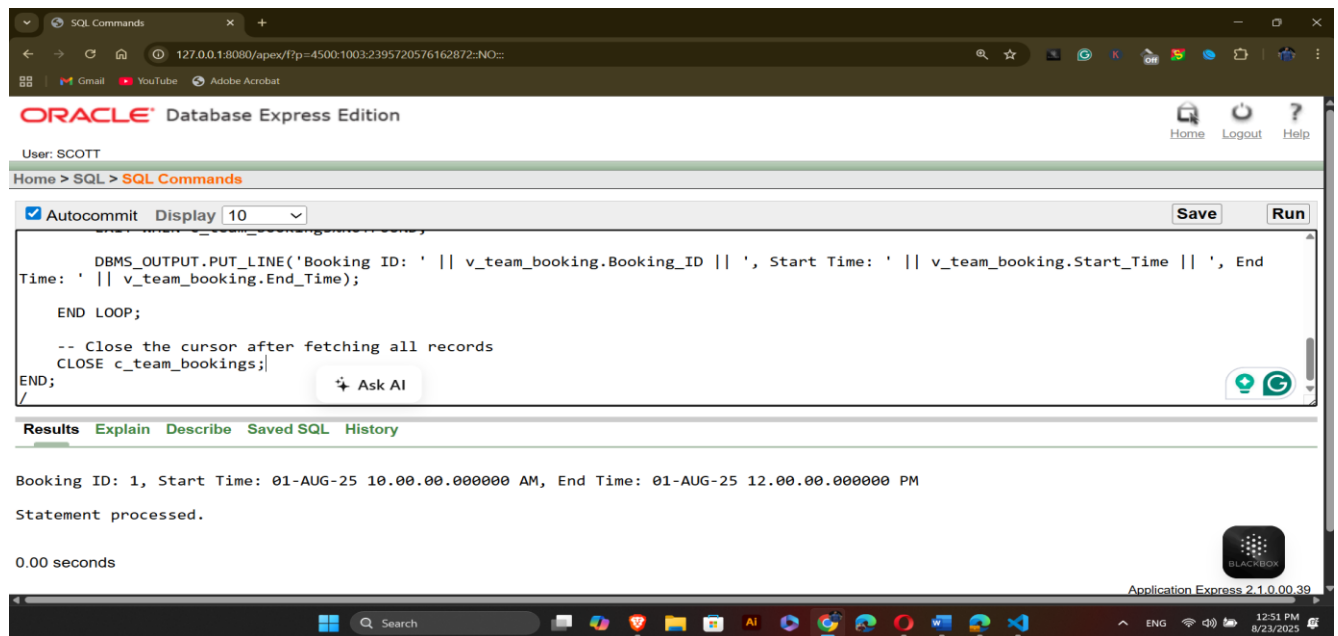
DBMS_OUTPUT.PUT_LINE('Booking ID: ' || v_team_booking.Booking_ID || ', Start
Time: ' || v_team_booking.Start_Time || ', End Time: ' || v_team_booking.End_Time);

END LOOP;

CLOSE c_team_bookings;

END;

/



5. Cursor-based-record

Question-1: Use a cursor-based record to print the names of players who have confirmed bookings.

Answer:

-- Project: Outdoor Sports Management System
 -- Semester: Summer 2025
 -- Course Name: Advance Database Management Systems
 -- Section: A

DECLARE

```

TYPE player_rec_type IS RECORD (
  player_id NUMBER,
  first_name VARCHAR2(100),
  last_name VARCHAR2(100)
);

```

```

TYPE player_tab_type IS TABLE OF player_rec_type;

v_players player_tab_type;

BEGIN

SELECT p.Player_ID, p.First_Name, p.Last_Name
BULK COLLECT INTO v_players

FROM Players p

JOIN Belongs_To bt ON p.Player_ID = bt.Player_ID

JOIN Can_Make cm ON bt.Team_ID = cm.Team_ID

JOIN Booking b ON cm.Booking_ID = b.Booking_ID

WHERE b.Status = 'Confirmed';

FOR i IN 1..v_players.COUNT LOOP

    DBMS_OUTPUT.PUT_LINE('Player Name: ' || v_players(i).first_name || ' ' ||
v_players(i).last_name);

END LOOP;

END;

/

```

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the following code:

```

FROM Players p
JOIN Belongs_To bt ON p.Player_ID = bt.Player_ID
JOIN Can_Make cm ON bt.Team_ID = cm.Team_ID
JOIN Booking b ON cm.Booking_ID = b.Booking_ID
WHERE b.Status = 'Confirmed';
FOR i IN 1..v_players.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE('Player Name: ' || v_players(i).first_name || ' ' || v_players(i).last_name);
END LOOP;
END;
/

```

The Results tab shows the output of the query:

```

Player Name: shadman alam
Player Name: amin ali
Player Name: arif aslma
Player Name: tanvir evan
Player Name: mainul islam

```

Below the results, it states "Statement processed."

Question-2: Use a cursor-based record to print the player names and their associated team names.

Answer:

-- Project: Outdoor Sports Management System

-- Semester: Summer 2025

-- Course Name: Advance Database Management Systems

-- Section: A

DECLARE

```
TYPE player_team_rec_type IS RECORD (  
    player_name VARCHAR2(100),  
    team_name VARCHAR2(100)  
);
```

```
TYPE player_team_tab_type IS TABLE OF player_team_rec_type;
```

```
v_player_teams player_team_tab_type;
```

BEGIN

```
SELECT p.First_Name || ' ' || p.Last_Name AS player_name, t.Team_Name  
BULK COLLECT INTO v_player_teams  
FROM Players p  
JOIN Belongs_To bt ON p.Player_ID = bt.Player_ID  
JOIN Team t ON bt.Team_ID = t.Team_ID;
```

```
FOR i IN 1..v_player_teams.COUNT LOOP
```

```

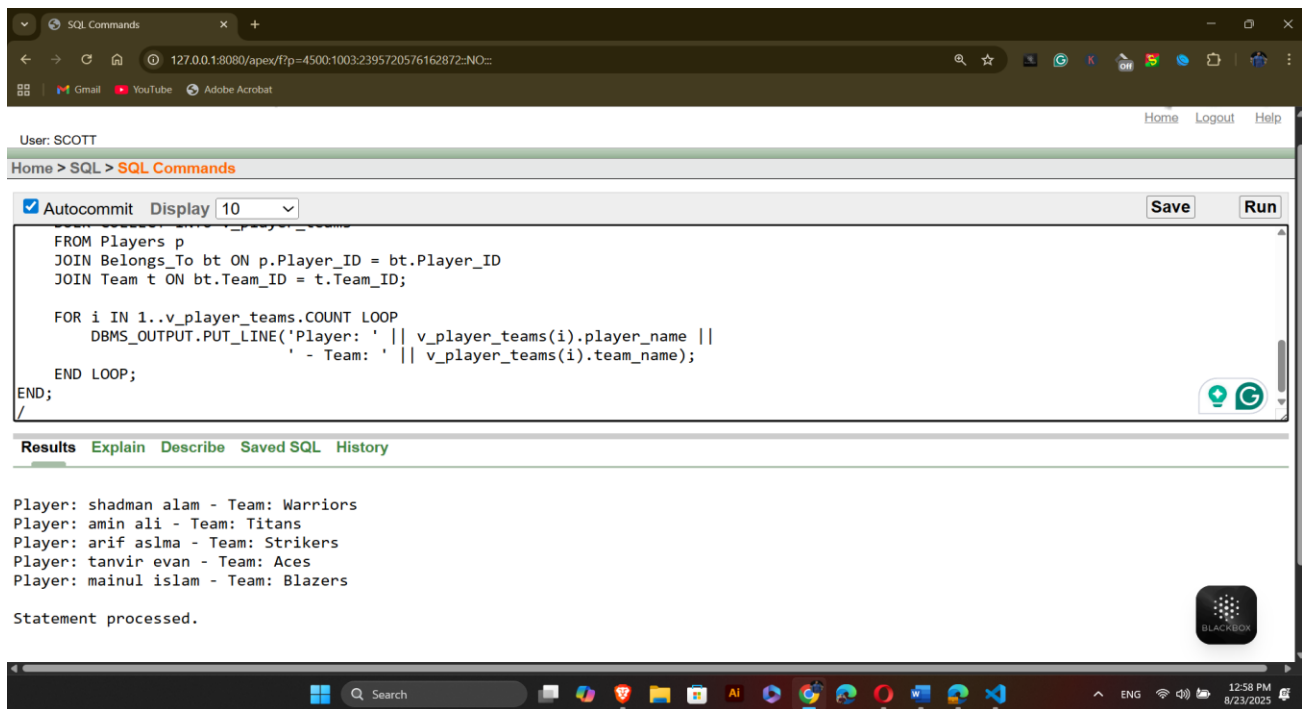
        DBMS_OUTPUT.PUT_LINE('Player: ' || v_player_teams(i).player_name ||
                               ' - Team: ' || v_player_teams(i).team_name);

    END LOOP;

END;

/

```



6. Row level trigger

Question-1: Create a row-level trigger that automatically sets the booking status to 'Pending' when a new booking record is inserted.

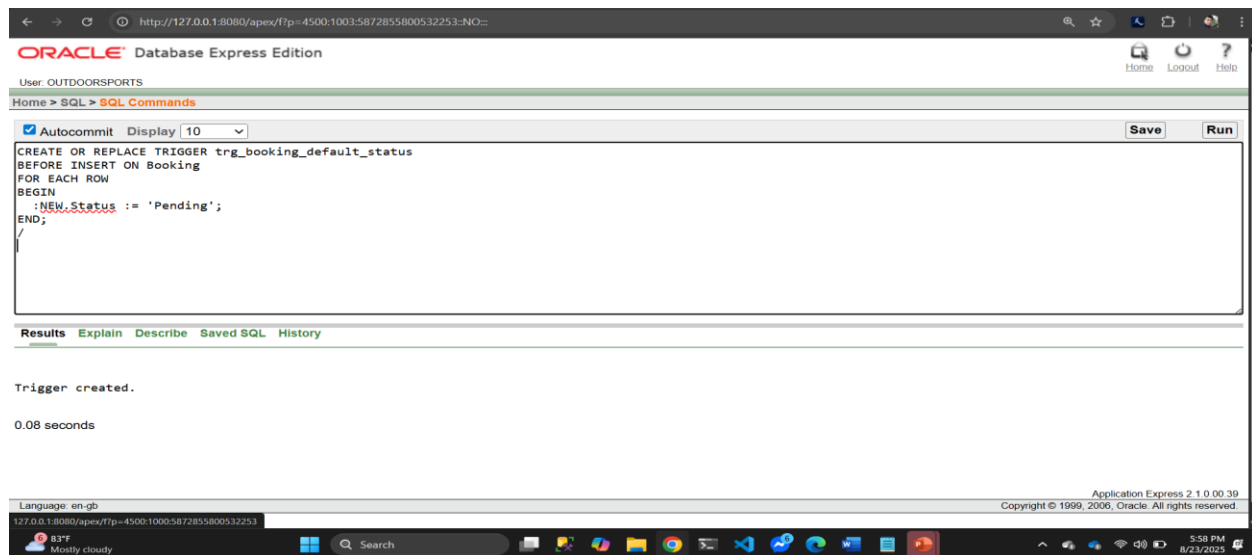
Answer:

```

CREATE OR REPLACE TRIGGER trg_booking_default_status
BEFORE INSERT ON Booking
FOR EACH ROW
BEGIN
    :NEW.Status := 'Pending';
END;

```


/



Question-2: Write a **row-level trigger** on the Players table that displays the name of a newly inserted player.

CREATE OR REPLACE TRIGGER trg_player_insert

AFTER INSERT ON Players

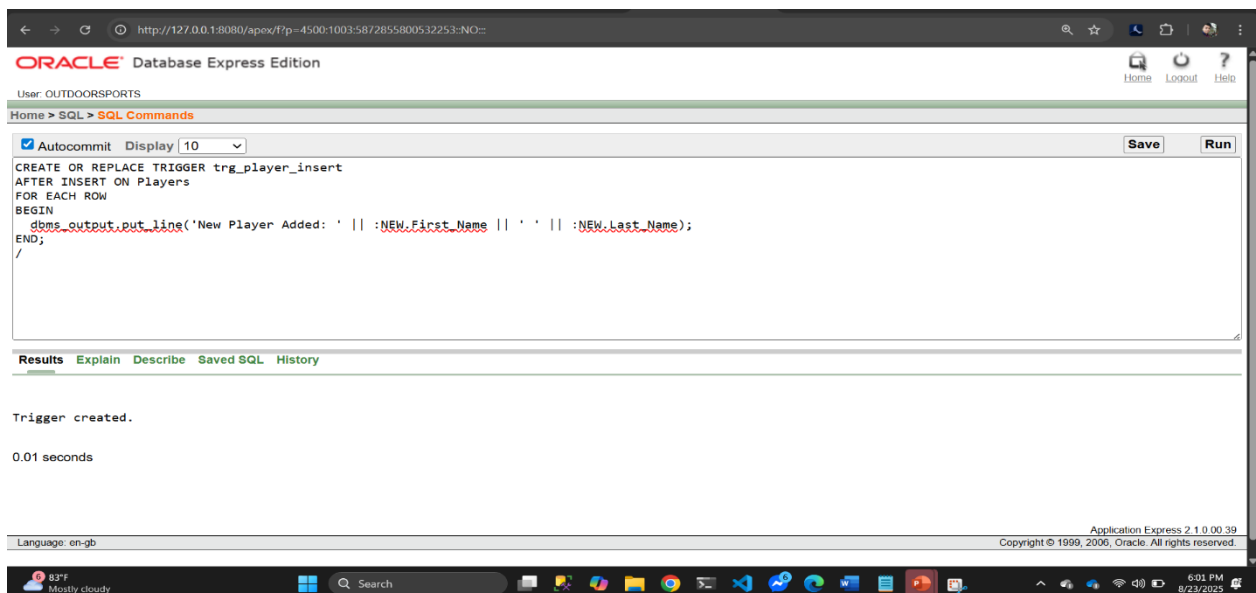
FOR EACH ROW

BEGIN

dbms_output.put_line('New Player Added: ' || :NEW.First_Name || ' ' || :NEW.Last_Name);

END;

/



7. Statement level trigger

Question-1: Write a statement-level trigger that prints a message whenever a new event is inserted into the Events table.

```
CREATE OR REPLACE TRIGGER trg_event_insert_stmt
```

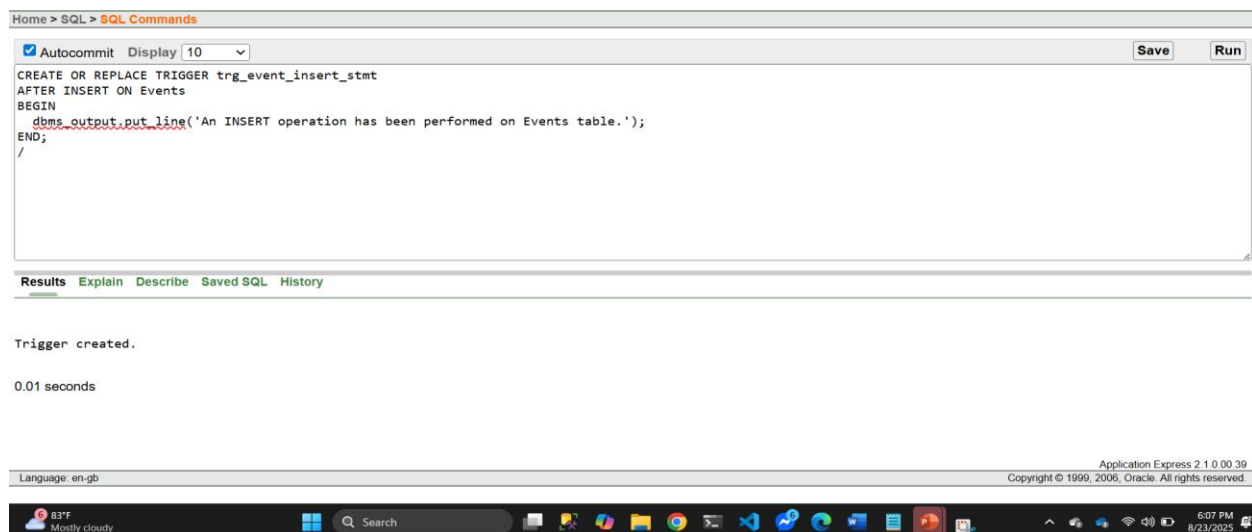
```
AFTER INSERT ON Events
```

```
BEGIN
```

```
    dbms_output.put_line('An INSERT operation has been performed on Events table.');
```

```
END;
```

```
/
```



Question-2: Write a statement-level trigger that prints a message before any row is deleted from the Team table

```
CREATE OR REPLACE TRIGGER trg_team_delete_stmt
```

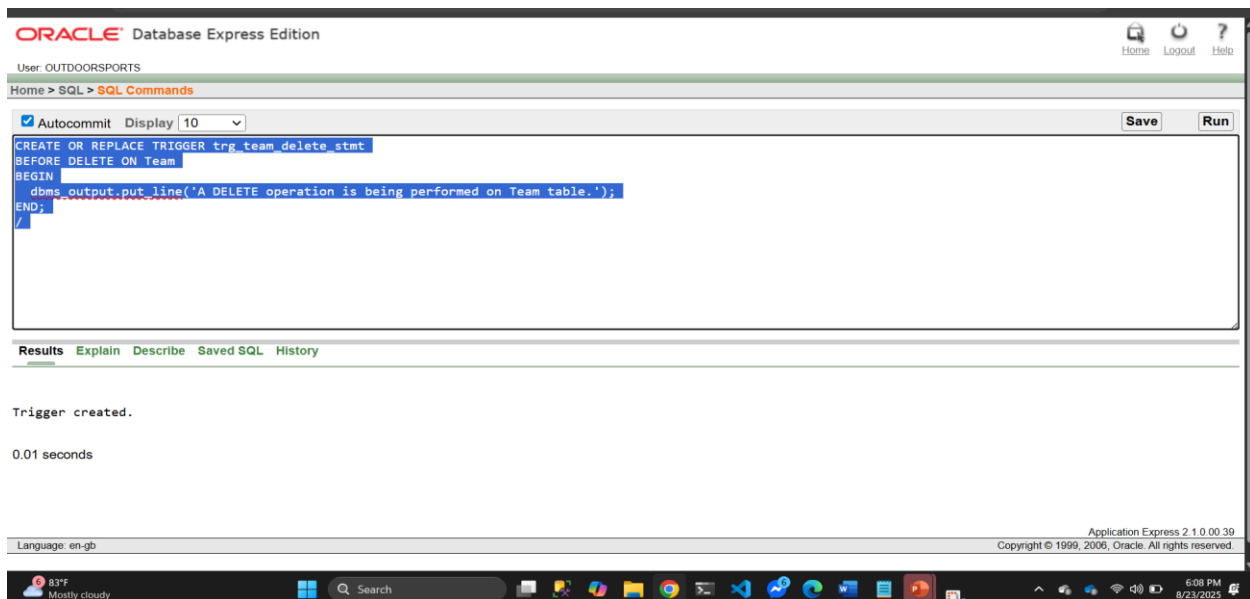
```
BEFORE DELETE ON Team
```

```
BEGIN
```

```
    dbms_output.put_line('A DELETE operation is being performed on Team table.');
```

```
END;
```

```
/
```



8. package

Question-1: Create a PL/SQL package named `player_pack` that contains procedures to display a player's full name and date of birth from the `Players` table.

Specification

```

CREATE OR REPLACE PACKAGE player_pack AS

    PROCEDURE show_player_name(p_id Players.Player_ID%TYPE);

    PROCEDURE show_player_dob(p_id Players.Player_ID%TYPE);

END player_pack;

/
```

Body

```

CREATE OR REPLACE PACKAGE BODY player_pack AS

    PROCEDURE show_player_name(p_id Players.Player_ID%TYPE) IS

        p_name VARCHAR2(100);

    BEGIN

        SELECT First_Name || ' ' || Last_Name INTO p_name

        FROM Players WHERE Player_ID = p_id;

        dbms_output.put_line('Player Name: ' || p_name);

    END;
```

```

PROCEDURE show_player_dob(p_id Players.Player_ID%TYPE) IS
    p_dob DATE;
BEGIN
    SELECT DOB INTO p_dob
    FROM Players WHERE Player_ID = p_id;
    dbms_output.put_line('Date of Birth: ' || TO_CHAR(p_dob,'DD-MON-YYYY'));
END;
END player_pack;
/

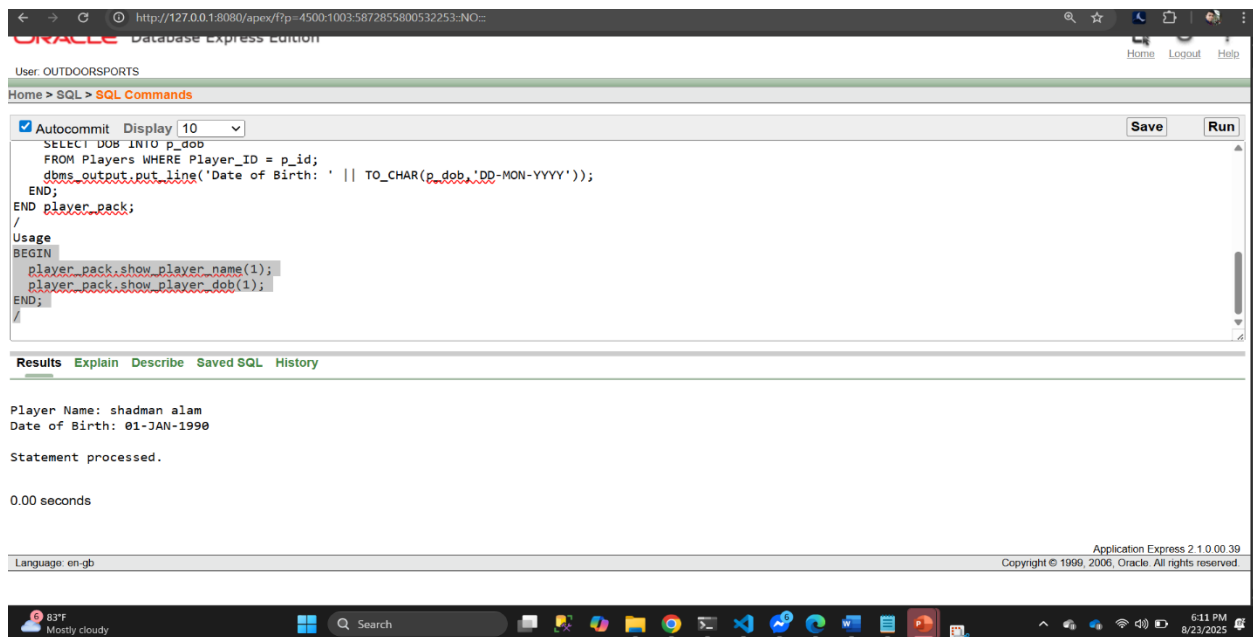
```

Usage

```

BEGIN
    player_pack.show_player_name(1);
    player_pack.show_player_dob(1);
END;
/

```



The screenshot shows the Oracle Database Express Edition interface. The SQL Command window contains the following code:

```

SELECT DOB INTO p_dob
FROM Players WHERE Player_ID = p_id;
dbms_output.put_line('Date of Birth: ' || TO_CHAR(p_dob,'DD-MON-YYYY'));
END;
END player_pack;
/
Usage
BEGIN
    player_pack.show_player_name(1);
    player_pack.show_player_dob(1);
END;
/

```

The Results tab shows the output of the execution:

```

Player Name: shadman alam
Date of Birth: 01-JAN-1990
Statement processed.
0.00 seconds

```

The footer of the window displays "Application Express 2.1.0.0.39" and "Copyright © 1999, 2006, Oracle. All rights reserved." The system tray at the bottom shows the date and time as 8/23/2025, 6:11 PM.

Question-2: Create a PL/SQL package named booking_pack that allows checking and updating the booking status in the Booking table.

Specification

```
CREATE OR REPLACE PACKAGE booking_pack AS

    PROCEDURE show_booking_status(b_id Booking.Booking_ID%TYPE);

    PROCEDURE update_booking_status(b_id Booking.Booking_ID%TYPE, new_status
    VARCHAR2);

END booking_pack;

/
```

Body

```
CREATE OR REPLACE PACKAGE BODY booking_pack AS

    PROCEDURE show_booking_status(b_id Booking.Booking_ID%TYPE) IS

        v_status VARCHAR2(50);

    BEGIN

        SELECT Status INTO v_status

        FROM Booking WHERE Booking_ID = b_id;

        dbms_output.put_line('Booking Status: ' || v_status);

    END;

    PROCEDURE update_booking_status(b_id Booking.Booking_ID%TYPE, new_status
    VARCHAR2) IS

    BEGIN

        UPDATE Booking SET Status = new_status WHERE Booking_ID = b_id;

        dbms_output.put_line('Booking ' || b_id || ' updated to ' || new_status);

    END;

END booking_pack;

/
```

Usage

BEGIN

```
booking_pack.show_booking_status(1);
```

```
booking_pack.update_booking_status(2, 'Confirmed');
```

END;

/

The screenshot shows the Oracle Database Express Edition SQL Command window. The user is 'OUTDOORSPORTS'. The SQL Command window contains the following code:

```
Autocommit Display 10
dms_output.put_line('Booking ' || b_id || ' updated to ' || new_status);
END;
END booking_pack;
/
```

Below the code, the 'Usage' section shows the following code:

```
BEGIN
booking_pack.show_booking_status(1);
booking_pack.update_booking_status(2, 'Confirmed');
END;
```

The 'Results' tab is selected, showing the following output:

```
Booking Status: Confirmed
Booking 2 updated to Confirmed

Statement processed.

0.00 seconds
```

The bottom of the window shows the Oracle Database Express Edition 2.1.0.0.39 Copyright © 1999, 2006, Oracle. All rights reserved. The system tray shows the date and time as 8/23/2025 6:14 PM.

Conclusion:

The Outdoor Sports Facility Management System project successfully implements a comprehensive solution for managing turf bookings, team creation, event scheduling, and player profile management. By utilizing advanced database management techniques and PL/SQL for business logic, the system optimizes key functions such as real-time turf availability, seamless booking, and secure payments. The user interface design ensures an intuitive experience for both sports organizations and individual players.

Looking ahead, the system can be further enhanced by integrating optimization techniques such as AI for predicting turf usage and recommending events or teams, ensuring more efficient operations. Improving mobile app integration will allow users to manage bookings and teams on-the-go, adding greater flexibility. Additionally, expanding payment options, implementing multilingual support, and strengthening data security will make the platform more accessible and secure for a wider audience.

In future iterations, the system could explore cloud-based infrastructure for scalability, and integration with machine learning could help in data-driven decision-making for sports organizations, optimizing resource usage and event planning. With these future enhancements, the Outdoor Sports Facility Management System will evolve into a fully scalable, secure, and user-friendly platform, capable of meeting the growing demands of both sports organizations and participants in the industry.